Outline

Axion Verification & Certification Suite (AVCS)

Clean System Outline

AVCS-00 — Purpose

Define AVCS as the system that verifies whether a build kit produced by Build Mode is actually enterprise quality, operationally safe, and deployment ready.

It exists to answer:

- Does the build work?
- Is it secure?
- Is it usable?
- Is it performant?
- Is it complete enough for release?
- What evidence proves that?

AVCS-01 — Objectives

State the core goals of the suite.

AVCS should:

- validate build quality across all critical domains
- surface failures, weaknesses, and regressions
- collect evidence for every important result
- score release readiness in a structured way

- prevent low-quality or unsafe builds from being deployed
- give operators a clear command center for certification decisions

---

AVCS-02 — Scope

Define what AVCS is responsible for testing.

Primary scope:

- build integrity
- functional correctness
- UI quality
- UX workflow quality
- security posture
- performance characteristics
- accessibility
- deployment readiness
- enterprise governance/readiness evidence

Out of scope:

- generating the build itself
- changing production systems directly
- auto-approving deployment without operator control
- replacing Build Mode
- replacing CI entirely

AVCS-03 — System Role in Axion

Place AVCS in the wider Axion lifecycle.

Recommended role:

- Build Mode produces a build kit
- AVCS executes verification and certification runs against that build
- AVCS records evidence, findings, and scores
- AVCS returns a release recommendation
- operator decides whether to proceed

So AVCS is the **quality and release gate layer** for build outputs.

AVCS-04 — Core Testing Domains

Define the major coverage categories AVCS must evaluate.

1. **Build Integrity**
    - install/build/type/lint readiness
    - required files/configs present
    - dependency resolution
    - environment contract validity
2. **Functional Coverage**
    - core flows execute correctly
    - expected outputs are produced
    - error handling behaves correctly

    - critical paths are not broken

3. **UI Coverage**
    - layouts render properly
    - screens/components display correctly
    - state transitions are visible and coherent
    - responsive behavior holds up
4. **UX Workflow Coverage**
    - end-to-end operator/user flows complete successfully
    - flows are efficient and understandable
    - friction and dead ends are surfaced
    - empty/loading/error states are usable
5. **Security Coverage**
    - auth and permissions checks
    - secret leakage detection
    - dependency vulnerability checks
    - unsafe input handling
    - insecure config/header exposure
6. **Performance Coverage**
    - startup and load timing
    - route/interaction latency
    - bundle/resource budgets
    - runtime stability under repeated use

    - regression against prior runs

7. **Accessibility Coverage**
    - keyboard navigation
    - semantic structure
    - focus behavior
    - labeling/roles
    - contrast and interaction clarity
8. **Deployment Readiness**
    - startup viability
    - env completeness
    - health/readiness checks
    - migration/config readiness
    - observability hooks
9. **Enterprise Readiness**
    - auditability
    - supportability
    - logging quality
    - operational clarity
    - release evidence completeness

AVCS-05 — Test Run Types

Define the major run modes of the suite.

1. **Smoke Run**
    - quick confidence check
    - catches obvious failures fast
2. **Functional Run**
    - validates critical workflows and system behavior
3. **UI/UX Run**
    - validates screens, navigation, and workflow usability
4. **Security Run**
    - focuses on hardening and exposure risks
5. **Performance Run**
    - validates speed, stability, and budget compliance
6. **Accessibility Run**
    - validates usability for assistive interaction patterns
7. **Regression Run**
    - compares current results to prior successful baselines
8. **Full Certification Run**
    - complete release-grade validation across all major domains
9. **Pre-Deployment Gate Run**
    - final decision run before release

AVCS-06 — Test Execution Model

Define how AVCS performs verification.

AVCS should:

- target a specific build kit/version
- execute selected run modes
- evaluate each test domain with explicit criteria
- collect artifacts and evidence during execution
- compute findings, severity, and coverage
- produce a final certification result

Execution should support:

- isolated runs
- repeatable runs
- selective reruns by domain
- comparison against previous runs
- reproducible evidence outputs

AVCS-07 — Evidence Model

Define what proof AVCS must collect.

Each run should produce structured evidence such as:

- run manifest
- domain results
- logs
- screenshots

- traces
- console output
- performance metrics
- security findings
- flow step results
- coverage summaries
- final certification report

The rule should be:
no important verdict without evidence.

AVCS-08 — Findings Model

Define how failures and issues are represented.

Every finding should include:

- domain
- severity
- status
- affected flow/page/module/surface
- failure description
- probable cause or signal
- evidence links
- remediation notes
- block/warn classification

Suggested severity levels:

- critical
- high
- medium
- low
- info

Suggested impact classes:

- release blocker
- conditional blocker
- warning
- observation

AVCS-09 — Certification Decision Model

Define the release outcome states.

Recommended result states:

- **PASS**
- **PASS_WITH_WARNINGS**
- **CONDITIONAL_PASS**
- **FAIL**
- **BLOCKED**

Meaning:

- **PASS** = deployable
- **PASS_WITH_WARNINGS** = deployable with tracked issues
- **CONDITIONAL_PASS** = operator override required
- **FAIL** = not deployable
- **BLOCKED** = incomplete or invalid certification run

This section should also define hard-stop failures that override score, such as:

- app does not boot
- critical flow broken
- auth boundary failure
- exposed secret or unsafe config
- severe accessibility failure in primary workflow
- evidence missing for required domains

AVCS-10 — Coverage Model

Define how AVCS measures “full coverage.”

Coverage should be measured across:

- domains
- workflows
- screens/pages
- APIs/endpoints
- critical user/operator actions

- risk classes
- release requirements

Coverage states can be:

- covered
- partially covered
- failed
- not tested
- blocked

This section is important because “full coverage” should mean something concrete, not just “a lot of tests ran.”

AVCS-11 — Scoring Model

Define how readiness is quantified.

AVCS should use weighted scoring by domain, but scoring should never replace hard-stop gates.

Example structure:

- functional correctness
- security
- performance
- UI/UX
- accessibility
- deployment readiness

- enterprise readiness

The score is useful for:

- release confidence
- trend tracking
- comparing builds
- prioritizing remediation

But the real release decision should always respect critical blockers.

AVCS-12 — Operator Experience

Define the command-center UI for AVCS.

Core operator surfaces:

1. **Certification Command Center**
    - choose build
    - choose run type
    - start and monitor runs
    - see status by domain
2. **Coverage Matrix**
    - domains vs workflows/features
    - covered/failed/partial/not-tested views
3. **Failure Triage Board**
    - findings by severity/domain

    - evidence preview
    - affected areas
    - remediation view
4. **UI/UX Flow Viewer**
    - step-by-step workflow outcomes
    - screenshots/states/friction points
5. **Security Workbench**
    - vulnerabilities
    - auth/config issues
    - leak/exposure findings
6. **Performance Workbench**
    - timings
    - budgets
    - regressions
    - stability metrics
7. **Release Gate Panel**
    - final verdict
    - blockers
    - warnings
    - evidence completeness
    - operator signoff

AVCS-13 — Audit and History

Define historical tracking.

AVCS should preserve:

- prior runs
- result deltas
- score trends
- repeated failures
- release decisions
- operator overrides
- evidence history

This makes AVCS useful not only for one run, but for long-term build quality management.

AVCS-14 — Policy and Gating

Define enforcement rules.

This section covers:

- which run types are required for which build classes
- which domains are mandatory for certification
- when overrides are allowed
- who can approve conditional passes
- what counts as a release blocker
- minimum evidence requirements
- minimum coverage thresholds

This is where AVCS becomes enterprise-grade rather than just informative.

AVCS-15 — Integration Boundaries

Define what AVCS connects to.

Likely integrations:

- Build Mode outputs
- build manifests
- kit metadata
- run history
- artifact storage
- release/deployment workflows
- observability/logging surfaces
- operator dashboards

AVCS should consume build outputs and emit certification outputs cleanly.

AVCS-16 — Minimum Viable AVCS

Define the first realistic version.

MVP should include:

- Smoke Run
- Functional Run
- Security Run
- Performance Run

- certification verdicts
- evidence collection
- failure triage
- release gate summary

That gets you an actually useful first version without bloating the system too early.

AVCS-17 — Future Expansion

Define what comes later.

Future layers:

- visual regression baselines
- richer UX scoring
- deeper accessibility automation
- policy packs by build type
- multi-build comparison
- certification trend analytics
- predictive release risk
- operator override workflows with formal justification

Clean one-line summary

AVCS is the post-build verification and certification system that determines whether a Build Mode output is secure, functional, usable, performant, and truly deployment ready.

AVCS-00

AVCS-00 — Purpose

1. What AVCS is

Axion Verification & Certification Suite (AVCS) is the post-build quality assurance and release-readiness system for build kits produced by Axion Build Mode.

Its job is not to generate the build kit. Its job is to **verify, certify, and document** whether that build kit meets Axion's required standard for enterprise deployment.

AVCS exists because a build being "produced" is not the same as a build being:

- correct
- secure
- usable
- stable
- supportable
- deployment ready

AVCS is the system that closes that gap.

---

2. Core purpose

The purpose of AVCS is to determine, with evidence, whether a build output is ready to move forward as a trustworthy enterprise-grade deliverable.

That means AVCS must answer five core questions:

1. **Does the build work as intended?**
2. **Is the build safe enough for enterprise use?**
3. **Is the build usable and coherent in real workflows?**
4. **Is the build operationally and technically ready for deployment?**

5. **What evidence proves the verdict?**

The system should never rely on assumption, optimism, or surface-level “pass” signals alone. Its purpose is to turn build validation into a **structured certification process**.

---

3. Why AVCS exists

Build Mode can assemble or produce a kit, but a produced kit can still have serious defects such as:

- broken flows
- incomplete configuration
- poor UX
- security weaknesses
- unstable runtime behavior
- missing release evidence
- deployment blockers
- enterprise-readiness gaps

Without AVCS, the system risks confusing **generation success** with **release quality**.

AVCS exists to prevent that failure mode.

It acts as the quality barrier between:

- “the build completed”
  and
- “the build is ready to trust”

---

4. Primary mission

The primary mission of AVCS is to provide a formal, repeatable, evidence-backed certification layer for all meaningful build outputs.

That mission has four parts:

4.1 Verify

Evaluate the build against explicit criteria across defined quality domains.

4.2 Detect

Identify failures, weaknesses, regressions, missing pieces, and hidden risks before release.

4.3 Prove

Collect artifacts, logs, metrics, screenshots, traces, and structured findings so every major result is backed by evidence.

4.4 Decide

Produce a clear certification outcome that tells the operator whether the build is:

- ready
- conditionally ready
- not ready
- blocked from decision

---

5. Strategic value inside Axion

AVCS gives Axion a formal quality gate after build generation.

That makes the overall Axion system stronger in several ways:

- it reduces the chance of shipping weak or unsafe outputs
- it standardizes what “deployment ready” means
- it turns subjective review into structured certification

- it preserves evidence for audits, remediation, and trend analysis
- it gives operators confidence that a build was actually validated, not just assembled

In other words, AVCS is not a convenience layer.
It is a **trust layer**.

---

6. What “certification” means in this system

In AVCS, certification does not mean legal certification or third-party accreditation.

It means an **Axion-governed internal release-quality verdict** based on:

- defined test coverage
- defined gating criteria
- required evidence
- explicit findings
- controlled release decision states

So when AVCS certifies a build, it is saying:

> This build was examined against Axion’s verification standards, required evidence was collected, and the resulting verdict reflects those standards.

That makes certification a structured internal quality contract.

---

7. What AVCS must protect against

The purpose section should explicitly state the failure classes AVCS is meant to catch before release.

AVCS is designed to protect against:

- false confidence from successful generation alone

- hidden functional breakage
- incomplete or misleading UI flows
- insecure defaults or exposures
- regressions against prior working outputs
- degraded performance that harms usability
- accessibility failures in primary workflows
- missing deployment requirements
- incomplete evidence for critical release decisions

This matters because AVCS is not just validating presence of code.
It is protecting release integrity.

---

8. Decision philosophy

AVCS should be built on the principle that **important release decisions require evidence, not assumption**.

That means:

- no major pass result without proof
- no critical risk ignored because a total score looks good
- no deployment recommendation based only on one test class
- no “green” status without minimum coverage and evidence completeness

Its purpose is to create disciplined release judgment.

---

9. Relationship to operators

AVCS is not meant to replace human judgment completely.

Its purpose is to:

- structure the validation process
- reduce ambiguity
- highlight blockers and risks
- present clear evidence
- support operator-controlled release decisions

The operator remains the final authority where policy requires human signoff, but AVCS should make that decision far more informed and much harder to make blindly.

---

10. Definition of success

AVCS succeeds when it consistently does the following:

- catches important quality issues before deployment
- measures build quality across all required domains
- produces repeatable certification outcomes
- gives operators a clear release recommendation
- preserves evidence for every meaningful verdict
- raises trust in Build Mode outputs over time

If AVCS only reports test results but does not help determine deployability, it is incomplete.
If it produces verdicts without evidence, it is unreliable.
If it allows weak builds through because they were merely generated successfully, it has failed its purpose.

---

11. Canonical purpose statement

Use a stable purpose statement for the section:

AVCS is the Axion post-build verification and certification system that evaluates build outputs for enterprise quality, collects evidence across required assurance domains, and produces a controlled release-readiness verdict before deployment.

---

12. Section close

AVCS-00 should end by establishing the system's identity clearly:

- **Build Mode produces**
- **AVCS verifies**
- **AVCS certifies**
- **operators decide using AVCS evidence and verdicts**

That is the foundation for the rest of the system.

AVCS-01

AVCS-01 — Objectives

1. Purpose of this section

This section defines what **Axion Verification & Certification Suite (AVCS)** is expected to accomplish in practice.

If **AVCS-00** explains why the system exists, **AVCS-01** defines the concrete outcomes the system must reliably produce.

These objectives should act as the operating targets for the entire suite. Every major AVCS component, run type, report, gate, and UI surface should map back to one or more of these objectives.

---

2. Primary objective

The primary objective of AVCS is to determine whether a build kit produced by Build Mode is genuinely ready for enterprise deployment, based on structured verification, required evidence, and controlled certification logic.

That means AVCS is not trying to answer only whether tests ran.
It is trying to answer whether the build is ready to be trusted.

---

3. Core system objectives

3.1 Validate enterprise quality across required assurance domains

AVCS must evaluate the build across the major domains that define enterprise readiness, not just code correctness.

At minimum, those domains include:

- build integrity
- functional correctness
- UI quality
- UX workflow quality

- security posture
- performance characteristics
- accessibility
- deployment readiness
- enterprise supportability and governance readiness

This objective ensures that certification is multidimensional rather than narrowly technical.

3.2 Detect failures, weaknesses, and release risks before deployment

AVCS must surface issues before a build reaches deployment or operator approval.

This includes:

- direct failures
- degraded behavior
- incomplete implementation
- unsafe configuration
- regression against prior working states
- weak usability in critical workflows
- missing operational requirements

The objective is preventive, not merely descriptive.
AVCS should catch issues while they are still controllable.

3.3 Convert test activity into a certification decision

AVCS must transform raw test execution into a structured release-readiness outcome.

That means the suite should not stop at:

- test logs
- scattered pass/fail results
- tool-specific outputs

Instead, it must consolidate those into a final certification state such as:

- pass
- pass with warnings
- conditional pass
- fail
- blocked

This objective is critical because enterprise systems need a decision surface, not just an evidence pile.

3.4 Require evidence for meaningful verdicts

AVCS must ensure that every important result is backed by verifiable evidence.

Evidence may include:

- logs
- screenshots
- traces
- performance captures
- security findings
- flow records
- coverage summaries

- run manifests
- artifact metadata

The objective here is to prevent unsupported conclusions.
A verdict without supporting evidence should be considered weak or invalid.

3.5 Standardize what “deployment ready” means inside Axion

AVCS must define and enforce a repeatable internal standard for release readiness.

Without this, different builds may be judged inconsistently based on opinion, speed, optimism, or incomplete review.

This objective gives Axion:

- consistent gating logic
- consistent coverage expectations
- consistent severity interpretation
- consistent release terminology
- consistent minimum evidence requirements

This is one of the most important institutional objectives of the system.

3.6 Provide clear operator visibility and control

AVCS must give operators a clean, high-signal view of:

- what was tested
- what passed
- what failed
- what is missing

- what is risky
- what is blocking release
- what evidence supports the decision

This objective means the system cannot be a black box.
Operators should be able to inspect the certification basis clearly and act on it with confidence.

3.7 Support repeatable and reproducible verification

AVCS must make verification runs repeatable enough that the same build, run mode, and conditions produce comparable outcomes.

This includes:

- stable run definitions
- consistent domain criteria
- preserved evidence structure
- explicit run manifests
- deterministic reporting where practical

This objective matters because certification loses value if results are noisy, opaque, or inconsistent.

3.8 Measure coverage, not just execution

AVCS must determine not only that tests ran, but what they actually covered.

That includes coverage across:

- required domains
- critical workflows

- important screens
- APIs/endpoints
- risk classes
- release requirements
- operator actions

This objective prevents “many tests ran” from being mistaken for “the system was meaningfully validated.”

3.9 Prevent weak builds from passing due to score masking

AVCS must ensure that critical failures cannot be hidden by strong performance in unrelated areas.

For example:

- excellent performance must not excuse auth failure
- strong UI polish must not excuse broken core flows
- broad functional success must not excuse secret exposure
- good overall score must not override hard release blockers

This objective requires hard-stop logic and blocker classification in addition to weighted scoring.

3.10 Support remediation, not just rejection

AVCS must help teams understand how to fix issues, not merely tell them that something failed.

That means findings should be structured in a way that makes them actionable:

- clear description
- impacted area

- severity
- evidence
- probable cause signal
- remediation guidance or next-step direction

This objective improves the system's usefulness as a quality-improvement engine, not just a gate.

3.11 Preserve certification history over time

AVCS must maintain historical records of runs, verdicts, evidence, and drift over time.

This supports:

- trend analysis
- regression detection
- release comparisons
- recurring issue identification
- audit review
- operator confidence building

The objective is not only to judge a single build, but to strengthen the overall build ecosystem through accumulated certification knowledge.

3.12 Fit cleanly into the Axion lifecycle

AVCS must integrate naturally with Build Mode and adjacent Axion systems without replacing them.

Its objective is to act as the post-build verification and certification layer by:

- consuming build outputs
- executing selected validation runs
- storing results and evidence
- returning a certification verdict
- informing the next release decision stage

This keeps system responsibilities clean.

---

4. Operational objectives

Beyond the high-level goals, AVCS should meet several practical operational objectives.

4.1 Fast confidence when needed

AVCS should support lightweight run types that provide quick confidence for obvious breakage detection.

4.2 Deep validation when required

AVCS should also support full release-grade certification runs for high-trust decisions.

4.3 Selective reruns

AVCS should allow focused reruns by domain or failure area so operators do not need to repeat unnecessary work.

4.4 Comparative review

AVCS should make it possible to compare current outcomes with previous successful certification runs.

4.5 Scalable evaluation structure

AVCS should be able to expand coverage depth over time without needing a total redesign.

---

5. Governance objectives

AVCS is also a governance system, not only a testing system.

Its governance objectives are to:

- enforce minimum validation expectations
- preserve structured release evidence
- support policy-based gating
- make override points explicit
- improve accountability for deployment decisions
- reduce ambiguous or informal release judgment

These objectives are what make AVCS enterprise-grade.

---

6. Anti-objectives

This section should also state what AVCS is not trying to do, so the system stays disciplined.

AVCS is not intended to:

- generate the build kit
- replace Build Mode
- silently deploy outputs
- rely on operator intuition alone
- treat all tests as equally important
- accept evidence-free pass states
- reduce certification to one flat score
- auto-approve unsafe releases for convenience

These anti-objectives help prevent design drift.

---

7. Success criteria for AVCS objectives

AVCS should be considered successful against its objectives when it consistently does the following:

- verifies all required assurance domains for the selected run type
- identifies meaningful failures before release
- produces evidence-backed findings and verdicts
- computes certification outcomes consistently
- prevents critical issues from being masked by aggregate scoring
- gives operators a clear release decision surface
- preserves run history and quality trends
- improves confidence in Build Mode outputs over time

---

8. Canonical objective statement

Use a stable summary statement for this section:

AVCS objectives are to validate enterprise quality across required assurance domains, detect release risks before deployment, collect evidence for every meaningful verdict, and produce a controlled certification decision that operators can trust.

---

9. Section close

AVCS-01 should leave the system with a clear mandate:

- test broadly enough to represent enterprise quality
- judge strictly enough to prevent weak releases
- prove results with evidence

- present outcomes clearly enough for operator decision-making

That gives the rest of the suite a concrete standard to build toward.

AVCS-02

AVCS-02 — Scope

1. Purpose of this section

This section defines the responsibility boundary of **Axion Verification & Certification Suite (AVCS)**.

If **AVCS-00** defines why the system exists and **AVCS-01** defines what it must achieve, **AVCS-02** defines what AVCS is and is not responsible for evaluating, enforcing, storing, and presenting.

This section matters because without a hard scope boundary, the suite can drift into:

- acting like Build Mode
- acting like deployment orchestration
- acting like general CI
- acting like a vague dashboard with no clear mission

AVCS must stay centered on **post-build verification and certification**.

---

2. Scope statement

AVCS is responsible for verifying and certifying the quality and release readiness of build kits produced by Axion Build Mode, using structured run types, domain-based evaluation, evidence collection, findings analysis, and controlled certification decisions.

Its scope begins when a meaningful build output exists and continues through certification outcome generation, evidence preservation, and operator-facing release-readiness reporting.

Its scope ends before autonomous deployment execution or uncontrolled downstream action.

---

3. In-scope responsibilities

AVCS is in scope for the following responsibility classes.

3.1 Build-output verification

AVCS is responsible for evaluating a produced build kit or build artifact set after generation.

This includes verifying whether the build is:

- complete enough to test
- structurally valid enough to certify
- stable enough to execute required checks against
- ready for release evaluation

AVCS is not responsible for generating that build, but it is responsible for determining whether the generated result is valid as a certification target.

3.2 Domain-based quality evaluation

AVCS is responsible for assessing the build across defined assurance domains.

These domains include:

- build integrity
- functional correctness
- UI quality
- UX workflow quality
- security posture
- performance behavior
- accessibility
- deployment readiness
- enterprise supportability and governance readiness

This means AVCS owns the evaluation layer across those domains, including the rules, coverage logic, pass/fail interpretation, and result aggregation for certification purposes.

3.3 Run orchestration inside certification boundaries

AVCS is responsible for orchestrating verification and certification runs against a selected build output.

This includes:

- selecting run type
- selecting target build/version
- executing domain-specific evaluation flows
- tracking run state
- collecting artifacts during execution
- allowing reruns within suite rules
- preserving run metadata and results

This is certification-run orchestration, not general pipeline orchestration.

3.4 Evidence collection and preservation

AVCS is responsible for collecting and preserving the evidence needed to support meaningful certification outcomes.

Evidence may include:

- logs
- traces
- screenshots
- console output
- performance measurements

- finding artifacts
- coverage summaries
- run manifests
- domain reports
- certification reports

AVCS owns the rule that major outcomes require evidence and that evidence must remain linked to the run and finding structure.

3.5 Findings generation and severity classification

AVCS is responsible for identifying findings and representing them in a structured way.

This includes:

- failure detection
- warning detection
- blocker classification
- severity assignment
- impacted-surface labeling
- evidence linking
- remediation-oriented presentation

AVCS is not merely a passive collector of raw tool output. It is responsible for converting validation signals into certification-relevant findings.

3.6 Coverage modeling

AVCS is responsible for defining and reporting what was actually covered during verification.

This includes coverage across:

- test domains
- workflows
- screens and surfaces
- APIs or service edges
- risk classes
- release requirements
- mandatory certification criteria

This keeps the suite from equating test execution volume with meaningful certification completeness.

3.7 Certification decision production

AVCS is responsible for computing and presenting a certification outcome for a build.

This includes:

- using findings, gates, coverage, and evidence completeness
- applying hard-stop logic
- applying score or weighted readiness logic where appropriate
- producing a final release-readiness state
- explaining the basis of the verdict clearly

The certification decision is one of the central scope items of AVCS.

3.8 Operator-facing certification experience

AVCS is responsible for presenting verification and certification information to operators through usable control surfaces.

This includes:

- run launch and monitoring
- domain status visibility
- failure triage
- evidence inspection
- coverage review
- historical comparison
- final gate review

So the suite is responsible not only for evaluation logic, but for the operator experience around certification.

3.9 Certification history and trend visibility

AVCS is responsible for preserving past runs and making historical certification context available.

This includes:

- previous run outcomes
- evidence history
- failure recurrence
- regressions over time
- score and readiness trends
- comparison between build versions

This is in scope because certification value increases when outcomes are historically interpretable.

3.10 Policy-driven release gating inputs

AVCS is responsible for applying and reporting policy-relevant gate conditions inside the certification process.

This includes:

- required domains by run type
- minimum coverage thresholds
- required evidence completeness
- hard blockers
- conditional-pass criteria
- override-needed states

AVCS may not execute deployment itself, but the gate logic that informs release approval is in scope.

---

4. Primary in-scope test surfaces

The core surfaces AVCS is allowed and expected to evaluate include the following.

4.1 Build/package surface

- build artifacts
- manifests
- package integrity
- config presence
- environment requirements

- startup/build readiness

4.2 Application runtime surface

- routes
- pages
- components
- workflows
- client behavior
- runtime failure conditions

4.3 Service/API surface

- endpoint behavior
- request/response correctness
- failure handling
- integration edges exposed by the build

4.4 UX interaction surface

- navigation
- task completion flow
- loading/error/empty states
- clarity and friction in primary operator or user journeys

4.5 Security and configuration surface

- auth boundaries
- permission behavior

- exposed secrets
- vulnerable dependencies
- unsafe defaults
- insecure headers/configuration surfaces

4.6 Performance surface

- load/start timing
- route latency
- interaction responsiveness
- stability under repeated execution
- size/budget compliance where applicable

4.7 Accessibility surface

- keyboard interaction
- semantic structure
- focus order
- assistive interpretation support
- interaction clarity and accessibility blockers

4.8 Deployment-readiness surface

- runtime boot viability
- required environment variables
- operational hooks
- health/readiness paths

- migration or startup assumptions
- supportability indicators

---

5. What AVCS explicitly does not own

This section is just as important as the in-scope list.

5.1 Build generation

AVCS does not generate the build kit.
That belongs to Build Mode.

AVCS may reject, block, or fail a generated build, but it does not own producing it.

5.2 Autonomous deployment execution

AVCS does not autonomously deploy certified builds.

It may output:

- pass states
- release recommendations
- gate results
- evidence packages

But actual deployment execution is outside AVCS scope unless another controlled system explicitly consumes those outputs.

5.3 General-purpose CI replacement

AVCS is not a full replacement for all repository CI or developer-local testing.

It may overlap with some CI signals, but its focus is:

- build-output certification
- release-readiness validation
- enterprise quality gating

General repo hygiene automation may exist elsewhere.

5.4 Arbitrary product analytics

AVCS is not a broad analytics platform for all product behavior.

It may capture metrics relevant to certification, but it does not own ongoing business analytics, user analytics, or unrelated telemetry interpretation.

5.5 Unbounded code review

AVCS is not a human-equivalent code review system for all architectural choices or code style issues.

It may flag structural or security concerns relevant to certification, but its mission is not to become a universal code-quality commentary engine.

5.6 Silent release approval

AVCS does not silently approve release without policy and operator visibility.

Even when the verdict is positive, the suite's role is to produce a controlled certification output, not an invisible automatic "ship it" action.

5.7 Production operations ownership

AVCS does not own live production incident response, uptime management, or production remediation workflows.

It can evaluate operational readiness indicators before release, but it is not the runtime operations control plane.

6. Dependency boundaries

AVCS depends on adjacent Axion systems, but those systems remain distinct.

6.1 Upstream dependencies

AVCS depends on upstream systems for:

- build production
- build metadata
- target selection
- artifact availability
- version identity

6.2 Internal dependencies

AVCS depends internally on:

- run definitions
- evaluation criteria
- evidence collection mechanisms
- scoring and gate logic
- finding models
- operator UI surfaces

6.3 Downstream consumers

AVCS outputs may be consumed by:

- release decision workflows
- operators

- audit/review processes
- maintenance systems
- historical trend systems

These consumers use AVCS outputs, but they are not AVCS itself.

---

7. Boundary rules

To keep scope stable, AVCS should follow a small number of explicit rules.

7.1 AVCS evaluates outputs, not intentions

It judges the produced build, not what the build was supposed to become.

7.2 AVCS certifies based on evidence, not optimism

A likely-good build is not enough. Evidence is required.

7.3 AVCS informs deployment decisions, but does not become deployment

It is a gate and certification layer, not the release executor.

7.4 AVCS can fail or block incomplete builds

If a build cannot be meaningfully certified, AVCS must be allowed to stop the path forward.

7.5 AVCS scope is post-build and pre-release

That is the cleanest system boundary.

---

8. Scope by lifecycle phase

A clean way to phrase the boundary is by lifecycle position.

Before AVCS

- requirements exist
- build logic runs

- build kit is generated
- target artifact is prepared

During AVCS

- target build is selected
- verification runs execute
- evidence is collected
- findings are created
- coverage is measured
- certification logic is applied
- verdict is produced

After AVCS

- operator reviews result
- downstream release process decides next step
- deployment may proceed or be blocked
- remediation work may begin

This framing keeps the suite from drifting into unrelated lifecycle ownership.

---

9. Scope control principle

Use a stable control principle for this section:

AVCS scope covers post-build verification, evidence-backed certification, and release-readiness reporting for build outputs, but excludes build generation, autonomous deployment, and unrelated platform responsibilities.

---

10. Canonical scope statement

Use a stable summary statement for the section:

AVCS is scoped to evaluate produced build outputs across required assurance domains, collect evidence, generate findings, measure coverage, and issue controlled certification outcomes that inform release decisions.

---

11. Section close

AVCS-02 should leave no ambiguity about the boundary:

- AVCS starts after a build exists
- AVCS evaluates the build across enterprise-quality domains
- AVCS produces evidence-backed certification outputs
- AVCS informs release decisions
- AVCS does not generate the build
- AVCS does not own deployment execution

That gives the rest of the system a hard frame.

AVCS-03

AVCS-03 — System Role in Axion

1. Purpose of this section

This section defines where **Axion Verification & Certification Suite (AVCS)** sits inside the larger Axion system and what role it plays relative to adjacent systems.

If **AVCS-00** defines why AVCS exists, **AVCS-01** defines its objectives, and **AVCS-02** defines its scope, then **AVCS-03** defines its **position in the Axion lifecycle** and its relationship to the rest of the platform.

This section is important because AVCS should not be treated as an isolated test tool. It is a formal system role inside the broader Axion operating model.

---

2. Core role statement

AVCS is the **post-build verification and certification layer** of Axion.

Its role is to take a build output produced by Build Mode, evaluate it across required assurance domains, collect supporting evidence, and return a controlled release-readiness verdict that operators and downstream systems can trust.

That makes AVCS the system that sits between:

- **build production**
  and
- **release approval / deployment progression**

It is the quality and certification barrier in that path.

---

3. Lifecycle position

AVCS belongs in the Axion lifecycle **after build generation and before release progression**.

A clean lifecycle view looks like this:

1. requirements / plans / instructions define what must be built

2. Build Mode produces the build kit
3. AVCS verifies and certifies the build output
4. operators review the certification outcome
5. downstream release or deployment processes act based on that result

This placement gives Axion a clear transition from:

- creation
  to
- validation
  to
- controlled release judgment

---

4. Why AVCS must exist as its own system role

AVCS should be a distinct system role because build production and build certification are not the same function.

A build system is optimized to:

- assemble
- generate
- compose
- produce outputs

A certification system is optimized to:

- inspect
- challenge

- measure
- prove
- gate
- document release readiness

If those roles are merged too tightly, the platform risks:

- self-validating outputs too loosely
- over-trusting generation success
- under-documenting failure evidence
- allowing weak builds to advance too far

AVCS exists as a separate role so Axion has an independent quality judgment layer.

---

5. AVCS as the trust layer

Inside Axion, AVCS should be understood as the **trust layer** for Build Mode outputs.

Build Mode answers:

- Was something produced?
- What was produced?
- How was it assembled?

AVCS answers:

- Does the output hold up under verification?
- Is it safe to trust?
- Is it ready for release?

- What evidence proves that?

This trust-layer role is what turns Axion from a build-producing system into a build-validating platform.

---

6. Relationship to Build Mode

Build Mode and AVCS are closely related, but their responsibilities must stay separate.

6.1 Build Mode responsibility

Build Mode is responsible for:

- generating the build kit
- assembling outputs
- packaging the result
- preserving build metadata
- exposing the produced target for validation

6.2 AVCS responsibility

AVCS is responsible for:

- selecting or receiving that build output
- executing verification runs against it
- measuring quality across defined domains
- collecting certification evidence
- generating findings and verdicts
- returning release-readiness outcomes

6.3 Boundary between them

Build Mode should never be the final authority on whether its own output is enterprise-ready.

That judgment belongs to AVCS.

So the boundary is:

- **Build Mode produces**
- **AVCS judges**

That is one of the most important role separations in the system.

---

7. Relationship to operators

AVCS is not only a backend validation system. It is also an operator-facing decision support layer.

Its role relative to operators is to:

- make verification visible
- make risk legible
- make evidence inspectable
- make release-readiness understandable
- make blockers explicit
- reduce subjective guesswork

Operators should not need to infer release safety from scattered logs or intuition. AVCS should concentrate that judgment into a structured control surface.

So in Axion, AVCS is both:

- a verification engine
- an operator certification console

---

8. Relationship to release decisions

AVCS does not replace the final release decision authority where human approval or policy approval is required.

Its role is to provide the formal basis for that decision.

That basis includes:

- run completion state
- domain-level outcomes
- findings and severity
- evidence completeness
- coverage completeness
- hard blockers
- certification verdict

This means AVCS is a **decision-enabling system**, not an uncontrolled decision-maker.

It should shape release decisions strongly, but it should do so through transparent outputs, not opaque automation.

---

9. Relationship to deployment systems

AVCS is upstream of deployment execution.

A deployment system may later consume AVCS outputs, such as:

- certification verdict
- blocker state
- required override state
- evidence package

- release recommendation

But deployment orchestration is not the role of AVCS.

This matters because the certification layer must remain disciplined. Once AVCS starts acting like a deployment engine, the boundary becomes unstable and the system becomes harder to trust.

AVCS should therefore be treated as:

- a gate
- a verdict source
- an evidence source
- a release-readiness interpreter

Not as the deployer itself.

---

10. Relationship to CI and test infrastructure

AVCS may overlap with ordinary test tooling, but its role is broader and more structured than generic CI.

Typical CI often answers:

- did the code compile?
- did unit tests pass?
- did linting succeed?
- did the pipeline complete?

AVCS must answer:

- is the produced build enterprise-ready?
- were critical workflows actually validated?

- are there security or UX blockers?
- is evidence complete enough to support release?
- what is the certification state of this build?

So AVCS should consume or coordinate test infrastructure where useful, but its system role is **certification**, not just automation of test commands.

---

11. Relationship to maintenance and historical quality systems

AVCS also plays a role in Axion's longer-term quality management.

Its outputs are useful beyond the immediate certification run because they provide:

- history of build quality
- recurring failure patterns
- regressions over time
- changing readiness trends
- evidence for remediation priorities

This means AVCS is not only a point-in-time gate. It is also part of Axion's quality memory.

That said, AVCS is still not the entire maintenance system. It contributes high-value certification data into that wider quality ecosystem.

---

12. Functional role breakdown

A clean way to define the system role is by the functions AVCS performs inside Axion.

12.1 Verification function

Checks whether the build works across required domains.

12.2 Certification function

Converts verification results into a controlled readiness verdict.

12.3 Evidence function

Captures the proof that supports each important result.

12.4 Gate function

Prevents weak or incomplete builds from advancing without visibility.

12.5 Operator support function

Gives humans a clear command center for certification review.

12.6 Historical quality function

Preserves run outcomes for comparison, regression tracking, and trend analysis.

These functions together define AVCS's system role much better than simply calling it "the test suite."

---

13. Role in the Axion control model

Within Axion's control model, AVCS should function as a **controlled quality authority**.

That means:

- it applies defined standards
- it evaluates against explicit criteria
- it records evidence and decisions
- it exposes overrides where policy allows
- it does not make silent or unverifiable release judgments

This makes AVCS part of the governance fabric of Axion, not just part of the engineering toolchain.

---

14. Input and output role framing

Another useful way to define AVCS's role is by what it receives and what it emits.

Inputs

AVCS receives:

- build kit target
- build metadata
- run selection
- certification policy context
- prior baseline or history where relevant

Outputs

AVCS emits:

- run state
- domain results
- findings
- evidence pack
- coverage summary
- certification verdict
- release gate signals
- historical record entry

This framing makes it clear that AVCS is a transformation layer: it turns build outputs into trusted certification outputs.

---

15. Role constraints

To protect the role from drift, AVCS should follow these constraints.

15.1 It must remain post-build

AVCS should not take over creation responsibilities.

15.2 It must remain evidence-driven

Its authority comes from proof, not internal assumption.

15.3 It must remain policy-aware

Its role includes enforcement of certification rules and blocker logic.

15.4 It must remain operator-legible

Its outputs must be understandable and inspectable.

15.5 It must remain distinct from deployment

It can inform progression, but it does not become progression itself.

These constraints keep the role stable as the system grows.

---

16. Canonical system-role statement

Use a stable summary statement for this section:

AVCS serves as Axion's post-build verification, evidence, and certification layer, transforming produced build outputs into controlled release-readiness verdicts that operators and downstream release systems can rely on.

---

17. Section close

AVCS-03 should leave the role unmistakable:

- Build Mode creates the output
- AVCS verifies the output
- AVCS certifies the output
- operators and downstream release systems consume that certification result

That is the clean role of AVCS in Axion.

AVCS-04

AVCS-04 — Core Testing Domains

1. Purpose of this section

This section defines the major assurance domains that **Axion Verification & Certification Suite (AVCS)** must evaluate when certifying a build output.

If earlier sections define why AVCS exists, what it must accomplish, what it owns, and where it sits in Axion, this section defines **what quality dimensions actually make up certification**.

This is one of the most important sections in the system because it prevents AVCS from becoming shallow or one-dimensional. A build is not enterprise-ready just because it compiles, just because it looks polished, or just because a few flows pass. AVCS must certify across a set of domains that together represent real deployment trust.

---

2. Domain model principle

AVCS should evaluate builds through a **multi-domain assurance model**.

That means:

- each domain represents a distinct class of release risk
- each domain has its own evaluation criteria
- each domain can produce its own findings and evidence
- each domain contributes to certification readiness
- some domains may contain hard blockers that override scoring

This model is necessary because enterprise quality is composite. Different failure classes live in different layers of the build.

---

3. Canonical domain set

The core testing domains for AVCS should be:

1. **Build Integrity**
2. **Functional Correctness**
3. **UI Quality**
4. **UX Workflow Quality**
5. **Security Posture**
6. **Performance & Stability**
7. **Accessibility**
8. **Deployment Readiness**
9. **Enterprise Readiness**

These nine domains give AVCS broad enough coverage to support a real certification verdict without drifting into an unbounded catch-all system.

---

4. Domain definitions

AVCS-04.1 — Build Integrity

4.1.1 Domain purpose

Build Integrity verifies that the produced build kit is structurally valid, internally coherent, and executable as a certification target.

This is the first domain because if the build is not fundamentally sound at the package/config/runtime-startup level, deeper certification is either impossible or misleading.

4.1.2 What this domain evaluates

Build Integrity should evaluate whether:

- required files and manifests exist
- dependency declarations resolve correctly

- install steps succeed
- typecheck/build/lint surfaces are valid where required
- environment contracts are present and coherent
- startup routines can execute
- configuration references are not broken
- artifact structure matches expected build-kit shape

4.1.3 Risks this domain is meant to catch

This domain protects against:

- incomplete build output
- broken package structure
- invalid config linkage
- missing env requirements
- non-bootable artifact states
- certification attempts against unusable build targets

4.1.4 Example findings

- missing required runtime config
- build target cannot install dependencies
- manifest references non-existent entry file
- startup command fails
- type surface broken in required area

4.1.5 Why it matters

A polished-looking application is irrelevant if the build itself is incomplete or structurally unsound. This domain establishes whether the output is even fit to undergo meaningful certification.

AVCS-04.2 — Functional Correctness

4.2.1 Domain purpose

Functional Correctness verifies that the build behaves according to expected product and system behavior across critical actions, logic paths, and response conditions.

This domain is the core “does it actually work?” layer.

4.2.2 What this domain evaluates

Functional Correctness should evaluate:

- critical user and operator flows
- form submission behavior
- CRUD operations where applicable
- state transitions
- API request/response handling
- expected output generation
- failure-state behavior
- validation rules
- data persistence behavior where relevant

4.2.3 Risks this domain is meant to catch

This domain protects against:

- broken primary workflows
- false-success states

- invalid output behavior
- endpoint failures
- missing action handling
- bad validation or state logic
- functional regressions from prior working builds

4.2.4 Example findings

- save flow reports success but does not persist data
- approval workflow breaks on final step
- API returns invalid payload shape
- error state traps user in unusable screen
- required action path fails under standard input

4.2.5 Why it matters

Enterprise readiness is impossible without functional trust. If the build does not reliably do what it claims, nothing else can compensate for that failure.

AVCS-04.3 — UI Quality

4.3.1 Domain purpose

UI Quality verifies that the visible interface renders clearly, consistently, and correctly across required surfaces and states.

This domain focuses on the presentation layer as an operational surface, not just aesthetics.

4.3.2 What this domain evaluates

UI Quality should evaluate:

- layout integrity

- rendering correctness
- component visibility and coherence
- responsive behavior
- consistency across pages and modules
- loading, empty, success, and error states
- visual breakage or overlap
- hierarchy and screen clarity
- high-friction visual defects

4.3.3 Risks this domain is meant to catch

This domain protects against:

- broken layout
- unreadable interface states
- invisible controls
- clipped or overlapping UI
- broken responsive behavior
- inconsistent visual state logic
- visually misleading success or failure cues

4.3.4 Example findings

- primary CTA hidden below broken layout region
- modal renders off-screen on standard viewport
- table columns overlap and obscure data

- loading state has no visible progress cue
- error banner visually conflicts with surrounding controls

4.3.5 Why it matters

A functionally correct build can still fail certification if the interface is visually unstable, confusing, or broken in actual use. UI quality directly affects operator trust and execution reliability.

AVCS-04.4 — UX Workflow Quality

4.4.1 Domain purpose

UX Workflow Quality verifies that real workflows can be completed efficiently, coherently, and without avoidable friction across the intended operator or user journey.

This is distinct from UI Quality. UI asks whether screens render correctly. UX Workflow asks whether the flow as a whole makes sense and can be completed well.

4.4.2 What this domain evaluates

UX Workflow Quality should evaluate:

- end-to-end workflow completion
- navigation clarity
- step ordering and coherence
- task friction
- dead ends and recovery paths
- empty/loading/error state usability
- operator comprehension during flow execution
- number of unnecessary steps or confusion points
- success-path continuity

4.4.3 Risks this domain is meant to catch

This domain protects against:

- workflows that technically work but are hard to complete
- dead-end navigation
- confusing sequencing
- missing recovery actions
- overly fragile interaction paths
- task abandonment due to poor flow design

4.4.4 Example findings

- operator must backtrack through unrelated screens to complete primary task
- workflow loses context between steps
- error recovery path is missing
- success state does not guide next action
- empty state gives no actionable route forward

4.4.5 Why it matters

Enterprise quality depends on more than logic correctness. If the product creates friction, confusion, or workflow breakdown during real use, it is not fully deployment-ready.

AVCS-04.5 — Security Posture

4.5.1 Domain purpose

Security Posture verifies that the build does not expose obvious or material security weaknesses that would make deployment unsafe or non-compliant with Axion release expectations.

This is one of the highest-priority domains because some security failures must immediately block certification.

4.5.2 What this domain evaluates

Security Posture should evaluate:

- auth boundary behavior
- permission enforcement
- secret exposure
- dependency vulnerabilities
- unsafe input handling
- insecure config defaults
- header and transport security surfaces where applicable
- debug/admin exposure
- unsafe storage or logging of sensitive material

4.5.3 Risks this domain is meant to catch

This domain protects against:

- unauthorized access
- privilege escalation
- leaked credentials or secrets
- exploitable dependency risk
- insecure defaults
- exposed internal surfaces
- unsafe handling of user-supplied data
- security regressions from prior safe baselines

4.5.4 Example findings

- admin surface accessible without required authorization
- secret value committed into build output
- vulnerable dependency above allowed severity threshold
- unsafe input accepted into sensitive execution path
- debug endpoint exposed in deployable profile

4.5.5 Why it matters

A build that is fast, polished, and functionally rich can still be entirely non-deployable if the security posture is weak. This domain carries hard-stop authority for many failure classes.

AVCS-04.6 — Performance & Stability

4.6.1 Domain purpose

Performance & Stability verifies that the build responds within acceptable thresholds and remains operationally stable under expected usage patterns.

This domain is about actual runtime quality, not just theoretical efficiency.

4.6.2 What this domain evaluates

Performance & Stability should evaluate:

- startup time
- initial load time
- route transition time
- action latency
- API response timing where relevant
- repeated-use stability
- memory or resource pressure signals

- size or budget compliance
- regression against prior successful baselines

4.6.3 Risks this domain is meant to catch

This domain protects against:

- slow startup
- laggy interaction
- resource spikes
- instability under repetition
- performance regressions
- unacceptable load budgets
- poor responsiveness in critical workflows

4.6.4 Example findings

- dashboard route exceeds allowed load threshold
- repeated action causes memory growth or UI degradation
- build bundle exceeds policy budget
- key flow latency regresses sharply from prior certified run
- startup sequence is too slow for required operational expectation

4.6.5 Why it matters

A system that technically works but performs poorly can still fail enterprise use. Poor performance erodes trust, harms workflow execution, and can create operational risk at scale.

AVCS-04.7 — Accessibility

4.7.1 Domain purpose

Accessibility verifies that the build remains usable for people relying on accessible interaction patterns and that key workflows are not blocked by preventable accessibility failures.

This is not an optional polish domain. It is part of release quality.

4.7.2 What this domain evaluates

Accessibility should evaluate:

- keyboard navigation
- focus order and focus visibility
- labels and roles
- semantic structure
- screen-reader interpretable surfaces
- contrast and readability issues
- dialog and modal accessibility
- form accessibility
- primary workflow accessibility blockers

4.7.3 Risks this domain is meant to catch

This domain protects against:

- inaccessible primary actions
- trapped focus states
- unlabeled controls
- unreadable contrast
- inaccessible forms or modals
- broken assistive interpretation of key interface regions

4.7.4 Example findings

- primary submission control unreachable by keyboard
- modal traps focus without exit path
- required form input has no accessible label
- low-contrast state obscures important message
- workflow completion blocked for non-pointer interaction pattern

4.7.5 Why it matters

Accessibility failures can make a system unusable for real users and can represent a serious enterprise readiness gap. Primary workflows must be accessible enough to certify.

AVCS-04.8 — Deployment Readiness

4.8.1 Domain purpose

Deployment Readiness verifies that the build contains the operational prerequisites needed to move from certification into a controlled deployment path.

This domain focuses on whether the build is practically releasable, not just theoretically good.

4.8.2 What this domain evaluates

Deployment Readiness should evaluate:

- boot viability
- environment variable completeness
- startup scripts and commands
- health and readiness checks
- migration or initialization readiness
- failure handling on startup

- config documentation sufficiency
- required runtime assumptions
- deploy-profile integrity

4.8.3 Risks this domain is meant to catch

This domain protects against:

- builds that pass app-level tests but cannot deploy cleanly
- missing env definitions
- incomplete startup procedures
- broken health surfaces
- migration failure risk
- undocumented runtime dependencies
- operational ambiguity at release time

4.8.4 Example findings

- required environment variable missing from deploy contract
- app boots locally but fails defined production-start path
- readiness endpoint absent for required deploy profile
- initialization step undocumented and mandatory
- migration prerequisite missing for release path

4.8.5 Why it matters

Certification should not greenlight a build that cannot realistically survive handoff into deployment. This domain makes release readiness concrete.

AVCS-04.9 — Enterprise Readiness

4.9.1 Domain purpose

Enterprise Readiness verifies whether the build meets the broader supportability, governance, observability, and operational clarity expectations required for enterprise-grade release confidence.

This domain captures the quality layers that are often forgotten when teams focus only on code and screens.

4.9.2 What this domain evaluates

Enterprise Readiness should evaluate:

- auditability of major actions
- logging quality
- supportability signals
- operator clarity
- error transparency
- release evidence completeness
- configuration explicitness
- role and permission clarity
- operational handoff quality where required

4.9.3 Risks this domain is meant to catch

This domain protects against:

- systems that are hard to support after release
- missing audit evidence
- weak or unclear operational signals
- poor error observability

- undocumented role assumptions
- incomplete release documentation or evidence trails

4.9.4 Example findings

- critical action produces no auditable record
- error logs are missing key diagnostic context
- operator-facing failure state provides no remediation direction
- role assumptions are implicit and undocumented
- release evidence pack incomplete for required class of build

4.9.5 Why it matters

A build can appear complete but still fail enterprise standards if it is difficult to support, govern, debug, or review responsibly after release. This domain captures that final layer of maturity.

---

5. Domain interaction rules

5.1 Domains must remain distinct but connected

The domains should not collapse into each other, but they also should not be treated as fully isolated.

For example:

- a broken permission check belongs primarily to Security Posture
- a confusing recovery flow belongs primarily to UX Workflow Quality
- a hidden primary button may affect both UI Quality and Accessibility
- a startup failure may affect both Build Integrity and Deployment Readiness

AVCS should allow cross-domain relationships while still assigning a primary finding domain.

5.2 Domains do not carry equal blocker power

Some domains will naturally contain more hard-stop failure classes than others.

Typical high-blocker domains:

- Build Integrity
- Functional Correctness
- Security Posture
- Deployment Readiness

Strongly important but not always immediate hard-stop domains:

- UI Quality
- UX Workflow Quality
- Performance & Stability
- Accessibility
- Enterprise Readiness

This does not mean the second group is optional. It means blocker logic should be designed intentionally rather than uniformly.

5.3 Domain coverage must be explicit by run type

Not every run type has to evaluate every domain at the same depth, but AVCS should always define:

- which domains are included
- which are partial

- which are required for certification
- which are deferred
- which are blockers for the selected run

This prevents ambiguity in certification meaning.

---

6. Domain evaluation principles

Each domain in AVCS should follow the same high-level evaluation structure:

- defined purpose
- defined target surfaces
- defined pass conditions
- defined failure conditions
- evidence requirements
- finding severity rules
- optional score contribution
- hard-blocker rules where applicable

That consistency makes the suite easier to operate and evolve.

---

7. Minimum expectation across all domains

Every core domain should be able to answer these questions:

- What was evaluated?
- What standard was applied?

- What passed?
- What failed?
- What evidence supports that result?
- Does the result affect certification?
- Is the issue a warning, blocker, or informational observation?

If a domain cannot answer those questions clearly, it is not mature enough for certification use.

---

8. Canonical domain statement

Use a stable summary statement for this section:

AVCS certifies build outputs across nine core assurance domains: Build Integrity, Functional Correctness, UI Quality, UX Workflow Quality, Security Posture, Performance & Stability, Accessibility, Deployment Readiness, and Enterprise Readiness.

---

9. Section close

AVCS-04 should establish that certification in Axion is not one-dimensional.

A build must be evaluated as:

- a package
- a functioning system
- a visible interface
- a usable workflow environment
- a security surface
- a runtime experience

- an accessible product
- a deployable artifact
- an enterprise-supportable release candidate

That is the full assurance frame AVCS should carry.

AVCS-05

AVCS-05 — Test Run Types

1. Purpose of this section

This section defines the major verification and certification run types that **Axion Verification & Certification Suite (AVCS)** must support.

If **AVCS-04** defines *what* domains AVCS evaluates, **AVCS-05** defines *how those domains are packaged into executable run modes* for different operational needs.

This section is critical because AVCS cannot operate as one giant undifferentiated test button. Different moments in the lifecycle require different depths of evaluation, different time costs, different evidence expectations, and different release authority.

Run types give the suite operational structure.

---

2. Run-type model principle

AVCS should use a **tiered run model**.

That means:

- each run type exists for a specific purpose
- each run type includes a defined subset of domains
- each run type has its own depth, speed, and evidence expectations
- each run type produces a known certification meaning
- not every run type is sufficient for release approval

This matters because:

- quick confidence checks are useful
- deep certification runs are necessary
- regression comparison is different from first-pass validation

- pre-release gating needs stricter authority than exploratory runs

So AVCS run types should be explicit, named, and policy-aware.

---

3. Canonical run-type set

The core AVCS run types should be:

1. **Smoke Run**
2. **Functional Run**
3. **UI/UX Run**
4. **Security Run**
5. **Performance Run**
6. **Accessibility Run**
7. **Regression Run**
8. **Full Certification Run**
9. **Pre-Deployment Gate Run**

These nine run types give AVCS enough flexibility to serve day-to-day validation, deeper review, historical comparison, and final release gating.

---

4. Run definitions

AVCS-05.1 — Smoke Run

4.1.1 Run purpose

Smoke Run is the fast-confidence verification mode.

Its purpose is to determine whether the build is sufficiently intact and operational to justify deeper testing, or whether it is obviously broken and should fail early.

4.1.2 When it should be used

Smoke Run should be used:

- immediately after build production
- after major build changes
- before spending time on deeper certification
- as a quick health check after remediation
- in rapid iteration loops where full certification is too heavy

4.1.3 What it should cover

Smoke Run should lightly evaluate:

- basic build integrity
- startup viability
- critical route/surface availability
- core dependency/config sanity
- obvious runtime failures
- at least one or two primary flow entry points

4.1.4 What it is meant to catch

Smoke Run is meant to catch:

- app will not boot
- missing critical files/config
- broken startup path
- immediate crash states
- obviously unusable build outputs

- major route or shell breakage

4.1.5 What it is not

Smoke Run is not a release-certification run.
It is not meant to prove deep quality.
It is a fast rejection or fast confidence layer.

4.1.6 Expected outcome meaning

A passing Smoke Run means:

- the build is probably viable enough for deeper evaluation

A failing Smoke Run means:

- deeper certification should usually stop until the basic failure is resolved

---

AVCS-05.2 — Functional Run

4.2.1 Run purpose

Functional Run validates that the build behaves correctly across critical actions, workflows, and system behaviors.

This is the main “does the system actually work?” run mode.

4.2.2 When it should be used

Functional Run should be used:

- after a successful Smoke Run
- after logic changes
- before release decisions
- after bug fixes affecting workflows
- when validating core feature completeness

4.2.3 What it should cover

Functional Run should evaluate:

- critical end-to-end flows
- primary actions and expected outputs
- form submissions
- API behavior and error handling
- state transitions
- business logic surfaces
- core failure-state behavior

4.2.4 What it is meant to catch

Functional Run is meant to catch:

- broken critical workflows
- false success states
- invalid outputs
- API/interaction failures
- logic regressions
- incomplete feature behaviors
- unhandled errors in core paths

4.2.5 Expected outcome meaning

A passing Functional Run means:

- the build likely satisfies baseline behavioral correctness in the validated flows

A failing Functional Run means:

- certification is materially weakened, and release may be blocked depending on severity and affected flow class

---

AVCS-05.3 — UI/UX Run

4.3.1 Run purpose

UI/UX Run validates the visible interface quality and the usability of primary workflows.

This run type exists because a build can be technically functional while still being frustrating, visually unstable, or operationally confusing.

4.3.2 When it should be used

UI/UX Run should be used:

- after interface changes
- when validating operator-facing workflows
- before release of major UI surfaces
- during polish and readiness review
- when quality expectations extend beyond raw functionality

4.3.3 What it should cover

UI/UX Run should evaluate:

- layout integrity
- responsive behavior
- loading/empty/error states
- workflow clarity
- navigation continuity
- step completion friction

- visual coherence of major screens
- action discoverability and task completion usability

4.3.4 What it is meant to catch

UI/UX Run is meant to catch:

- broken layouts
- hidden or inaccessible controls
- confusing workflow sequences
- dead-end states
- poor recovery patterns
- visual instability
- friction in primary operator journeys

4.3.5 Expected outcome meaning

A passing UI/UX Run means:

- major visual and workflow surfaces appear sufficiently coherent and usable for intended operation

A failing UI/UX Run means:

- the build may still function technically, but certification confidence is reduced and release may be blocked for primary-surface issues

---

AVCS-05.4 — Security Run

4.4.1 Run purpose

Security Run validates whether the build exposes material security weaknesses that would make release unsafe.

This is one of the highest-authority run types in AVCS because certain failures here should block certification immediately.

4.4.2 When it should be used

Security Run should be used:

- before any meaningful release decision
- after auth or permission changes
- after dependency shifts
- after config changes affecting exposure
- as part of full certification
- as part of any pre-deployment gate

4.4.3 What it should cover

Security Run should evaluate:

- auth boundary behavior
- role/permission enforcement
- secret leakage
- vulnerable dependencies
- unsafe inputs
- exposed admin/debug surfaces
- insecure defaults/configuration
- sensitive logging/storage behavior where relevant

4.4.4 What it is meant to catch

Security Run is meant to catch:

- unauthorized access
- privilege escalation
- exposed secrets
- severe vulnerability conditions
- unsafe execution surfaces
- insecure production configuration
- material security regressions

4.4.5 Expected outcome meaning

A passing Security Run means:

- no material security blockers were found within the scope of the run

A failing Security Run means:

- release should usually be blocked unless policy explicitly distinguishes the issue as non-blocking

---

AVCS-05.5 — Performance Run

4.5.1 Run purpose

Performance Run validates that the build performs within acceptable thresholds and remains stable under expected usage patterns.

This run type exists because responsiveness and runtime stability are part of enterprise quality, not optional refinements.

4.5.2 When it should be used

Performance Run should be used:

- before release for performance-sensitive builds

- after major frontend/runtime changes
- after large dependency or bundle shifts
- after infrastructure-affecting changes
- when a prior baseline exists and regressions matter

4.5.3 What it should cover

Performance Run should evaluate:

- startup/load timing
- route transition timing
- interaction latency
- repeated action stability
- resource pressure signals
- performance budgets
- regression against baseline
- size/bundle budget conditions where relevant

4.5.4 What it is meant to catch

Performance Run is meant to catch:

- slow startup
- laggy workflows
- poor responsiveness
- memory/resource growth
- unacceptable regressions

- budget violations
- instability under repeat usage

4.5.5 Expected outcome meaning

A passing Performance Run means:

- the build is operating within acceptable performance and stability bounds for the tested surfaces

A failing Performance Run means:

- release quality may be materially reduced, especially if critical workflows exceed policy thresholds

---

AVCS-05.6 — Accessibility Run

4.6.1 Run purpose

Accessibility Run validates that the build remains usable for accessible interaction patterns and that primary workflows are not blocked by preventable accessibility failures.

4.6.2 When it should be used

Accessibility Run should be used:

- before release of major UI surfaces
- after interface or interaction changes
- during full certification
- during pre-deployment gate review for required build classes

4.6.3 What it should cover

Accessibility Run should evaluate:

- keyboard navigation
- focus behavior
- labels and roles
- semantic structure
- forms/modals/dialogs
- contrast and readability
- primary workflow accessibility blockers

4.6.4 What it is meant to catch

Accessibility Run is meant to catch:

- blocked keyboard workflows
- unlabeled critical controls
- focus traps
- inaccessible dialogs/forms
- unreadable contrast states
- assistive-usage blockers in key flows

4.6.5 Expected outcome meaning

A passing Accessibility Run means:

- the tested interface surfaces meet the suite's minimum accessible usability expectations

A failing Accessibility Run means:

- the build may not be acceptable for release if key workflows are blocked or materially degraded

---

AVCS-05.7 — Regression Run

4.7.1 Run purpose

Regression Run compares the current build against a previous successful or approved baseline to detect quality drift, new failures, or degraded readiness.

This run type is about change detection, not only isolated validation.

4.7.2 When it should be used

Regression Run should be used:

- after meaningful changes to a previously certified build
- before release when a prior certified baseline exists
- after remediation of known issues
- when trend tracking and comparative confidence matter

4.7.3 What it should cover

Regression Run should compare:

- findings count and severity
- critical workflow status
- performance metrics
- UI stability on tracked surfaces
- security posture deltas
- coverage deltas
- certification score or readiness changes where used

4.7.4 What it is meant to catch

Regression Run is meant to catch:

- reintroduced bugs
- worsened latency
- newly broken workflows
- degraded coverage
- increased severity footprint
- drift from previously accepted quality levels

4.7.5 Expected outcome meaning

A passing Regression Run means:

- the current build is materially aligned with or stronger than the selected baseline

A failing Regression Run means:

- the new build has degraded in one or more meaningful ways and requires review before progression

---

AVCS-05.8 — Full Certification Run

4.8.1 Run purpose

Full Certification Run is the complete release-grade validation mode.

It is the main comprehensive run type used to determine whether a build meets Axion's certification standard across all required assurance domains.

4.8.2 When it should be used

Full Certification Run should be used:

- before major release decisions
- for enterprise-significant builds

- for milestone or handoff builds
- when full evidence is required
- when policy requires full validation coverage

4.8.3 What it should cover

Full Certification Run should evaluate all core AVCS domains at the required policy depth:

- Build Integrity
- Functional Correctness
- UI Quality
- UX Workflow Quality
- Security Posture
- Performance & Stability
- Accessibility
- Deployment Readiness
- Enterprise Readiness

4.8.4 What it is meant to catch

Full Certification Run is meant to catch:

- cross-domain weakness
- hidden blockers
- incomplete evidence
- policy noncompliance
- release-readiness gaps that lighter runs would miss

4.8.5 Expected outcome meaning

A passing Full Certification Run means:

- the build has satisfied the required certification standard for the selected policy context

A failing Full Certification Run means:

- the build is not ready for certification approval without remediation or controlled override policy where allowed

---

AVCS-05.9 — Pre-Deployment Gate Run

4.9.1 Run purpose

Pre-Deployment Gate Run is the final release-decision run used immediately before the build is allowed to progress into deployment or formal release handoff.

This is the strictest operational run type in AVCS.

4.9.2 When it should be used

Pre-Deployment Gate Run should be used:

- immediately before deployment progression
- as the final release gate for policy-controlled builds
- when final confidence and blocker review are required
- after all major remediation is complete

4.9.3 What it should cover

Pre-Deployment Gate Run should include:

- all mandatory release domains
- final blocker review

- evidence completeness validation
- policy threshold checks
- final coverage sufficiency check
- final verdict computation

It may reuse outputs from recent runs where policy allows, but the gate result itself must be explicit and fresh enough to trust.

4.9.4 What it is meant to catch

Pre-Deployment Gate Run is meant to catch:

- unresolved blockers
- stale evidence
- incomplete required coverage
- unreviewed regressions
- policy violations before final progression
- false-positive readiness from earlier partial runs

4.9.5 Expected outcome meaning

A passing Pre-Deployment Gate Run means:

- the build has cleared the final AVCS gate for deployment progression under the applicable policy

A failing or blocked Pre-Deployment Gate Run means:

- release progression should stop until the gate issue is resolved or formal override policy is applied

---

5. Run hierarchy

5.1 Lightweight runs

These support fast confidence and targeted review:

- Smoke Run
- Functional Run
- UI/UX Run
- Security Run
- Performance Run
- Accessibility Run

5.2 Comparative run

This supports drift detection:

- Regression Run

5.3 Comprehensive release runs

These support certification authority:

- Full Certification Run
- Pre-Deployment Gate Run

This hierarchy helps AVCS scale from rapid iteration to release-grade control.

---

6. Run authority levels

Not all run types should carry the same certification authority.

6.1 Informational or preparatory authority

These runs provide strong signals but are usually not sufficient alone for release approval:

- Smoke Run
- UI/UX Run
- Performance Run
- Accessibility Run

6.2 Substantive domain authority

These runs can materially affect certification and may block progression in their domain:

- Functional Run
- Security Run
- Regression Run

6.3 Final certification authority

These runs are intended to support or determine release readiness:

- Full Certification Run
- Pre-Deployment Gate Run

This prevents misuse of lighter runs as substitutes for full certification.

---

7. Run composition rules

Each run type should declare:

- included domains
- excluded domains
- domain depth

- required evidence types
- blocker logic
- scoring participation
- reuse rules for prior evidence if allowed
- certification meaning

Without this, run names become vague labels instead of enforceable operating modes.

---

8. Rerun rules

AVCS should allow reruns, but reruns should be controlled.

8.1 Allowed rerun classes

AVCS should support:

- full rerun of same run type
- selective rerun by domain
- targeted rerun for failed workflows
- post-remediation rerun
- baseline comparison rerun

8.2 Rerun requirements

Each rerun should preserve:

- linkage to original run where relevant
- reason for rerun
- changed scope if narrowed

- fresh evidence for rerun surfaces
- updated verdict state if certification meaning changes

This prevents reruns from becoming confusing or evidence-destructive.

---

9. Policy linkage

Run types should connect directly to policy.

Policy should define:

- which run types are required for which build classes
- which run types are optional
- which run types can block release directly
- how recent a run must be to count toward a gate
- whether evidence from earlier runs can be reused
- who can accept conditional outcomes

This is what turns run types into enforceable certification machinery.

---

10. Canonical run-type statement

Use a stable summary statement for this section:

AVCS supports tiered verification and certification through nine run types: Smoke, Functional, UI/UX, Security, Performance, Accessibility, Regression, Full Certification, and Pre-Deployment Gate.

---

11. Section close

AVCS-05 should establish that verification in Axion is operationally structured, not ad hoc.

Different runs exist for different purposes:

- fast confidence
- domain-specific validation
- drift detection
- full certification
- final release gating

That gives AVCS the execution model it needs to be practical and enterprise-grade at the same time.

AVCS-06

AVCS-06 — Test Execution Model

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** actually executes verification and certification against a build output.

If **AVCS-05** defines the run types AVCS supports, **AVCS-06** defines the operating mechanics behind those runs:

- how a run starts
- what a run targets
- how domains execute
- how evidence is collected
- how findings are produced
- how verdict inputs are assembled

This section is where AVCS stops being just a conceptual quality system and becomes an operational one.

---

2. Core execution statement

AVCS executes certification through **structured runs** against a selected build target, using declared run scope, domain-specific evaluators, controlled evidence collection, and deterministic result assembly to produce a certification-ready output.

That means every run should have:

- a target
- a mode
- a scope
- an execution plan

- a set of domain evaluators
- a result set
- an evidence pack
- a final assembled outcome

AVCS should never operate as an unstructured “run some checks and hope” system.

---

3. Execution model principle

The AVCS execution model should follow six principles:

3.1 Targeted

Every run must target a specific build kit, version, or certification artifact set.

3.2 Structured

Every run must execute according to a declared run type and explicit scope.

3.3 Evidence-backed

Every meaningful result must be supported by captured evidence.

3.4 Repeatable

The same target and same run definition should produce materially comparable outcomes.

3.5 Isolatable

Runs and reruns should support narrow scope where policy allows.

3.6 Assembled

Raw domain outputs must be normalized into findings, coverage, and verdict inputs.

These principles keep AVCS reliable and operator-trustworthy.

---

4. High-level execution flow

A clean AVCS execution flow should look like this:

1. target build is selected
2. run type is selected
3. run manifest is created
4. execution prerequisites are checked
5. domain plan is resolved
6. evaluators execute
7. evidence is collected during execution
8. domain outcomes are normalized
9. findings are generated and classified
10. coverage is computed
11. certification inputs are assembled
12. verdict engine runs
13. outputs are stored and presented

This sequence should remain stable even as AVCS grows.

---

5. Execution stages

AVCS-06.1 — Stage 1: Target Resolution

5.1.1 Purpose

The first stage resolves exactly what AVCS is testing.

5.1.2 Required target identity

A run should always resolve a concrete target such as:

- build kit ID
- build version
- artifact manifest
- commit/build reference
- environment/profile under certification
- baseline reference if regression comparison is requested

5.1.3 Rules

AVCS should not allow ambiguous targets.
If the build target cannot be identified clearly, the run should be blocked before deeper execution begins.

5.1.4 Why it matters

Certification has no meaning if the system cannot say exactly what was certified.

AVCS-06.2 — Stage 2: Run Initialization

5.2.1 Purpose

This stage creates the formal run record and locks the intended execution scope.

5.2.2 Run manifest contents

The run manifest should include at minimum:

- run ID
- target ID
- run type
- requested domains
- policy context

- operator or initiating actor
- start timestamp
- baseline reference if applicable
- environment/profile context
- evidence requirements
- rerun linkage if applicable

5.2.3 Rules

Once execution begins, the manifest should be treated as the authoritative identity for that run.

Changes to scope after start should either:

- be disallowed
  or
- create an explicit updated manifest event

5.2.4 Why it matters

The manifest is the backbone of traceability and reproducibility.

AVCS-06.3 — Stage 3: Preflight and Prerequisite Checks

5.3.1 Purpose

Before running deeper checks, AVCS should confirm that the target is testable and that the requested run can execute meaningfully.

5.3.2 What preflight should validate

Preflight should confirm:

- target artifacts are available
- required runtime/config inputs are present

- selected run type is valid for the target class
- required evaluators are available
- required evidence collectors can run
- baseline exists when regression mode requires it
- policy prerequisites are met

5.3.3 Failure behavior

If preflight fails, the run should not proceed blindly.
It should enter a clear blocked or failed-initialization state with reasons recorded.

5.3.4 Why it matters

This prevents false-negative chaos and prevents AVCS from pretending to certify an untestable target.

AVCS-06.4 — Stage 4: Domain Plan Resolution

5.4.1 Purpose

This stage converts the selected run type into the actual domain execution plan.

5.4.2 What domain planning should decide

AVCS should determine:

- which domains are included
- which domains are required
- which domains are partial-depth
- execution order
- evidence collectors per domain
- blocker-sensitive checks

- scoring participation
- reuse permissions for prior evidence if policy allows

5.4.3 Execution-order principle

AVCS should generally execute from foundational domains toward higher-layer domains.

A common order could be:

1. Build Integrity
2. Deployment Readiness prechecks
3. Functional Correctness
4. Security Posture
5. UI Quality
6. UX Workflow Quality
7. Accessibility
8. Performance & Stability
9. Enterprise Readiness assembly

Exact ordering can vary, but foundational failure should be visible early.

5.4.4 Why it matters

A declared run type needs to become a real execution plan, not just a label.

AVCS-06.5 — Stage 5: Domain Evaluation Execution

5.5.1 Purpose

This is the core evaluation stage where domain-specific verification actually runs.

5.5.2 Domain evaluator model

Each domain should execute through a defined evaluator or evaluator set.

A domain evaluator should know:

- what surfaces it tests
- what pass/fail criteria apply
- what evidence it must collect
- what outputs it emits
- what blocker logic applies
- how to report incomplete or indeterminate states

5.5.3 Domain output shape

Every domain evaluation should return a normalized output that includes at least:

- domain name
- execution status
- pass/fail/partial state
- raw observations
- finding candidates
- evidence references
- coverage notes
- timing/metadata
- blocker flags where relevant

5.5.4 Partial execution handling

A domain may complete as:

- passed

- failed
- partial
- blocked
- skipped by design
- indeterminate

AVCS should preserve these distinctions clearly.

5.5.5 Why it matters

Without normalized domain execution, AVCS becomes a loose wrapper around unrelated checks instead of a certification system.

AVCS-06.6 — Stage 6: Evidence Collection

5.6.1 Purpose

Evidence collection runs alongside execution and preserves proof for all meaningful outcomes.

5.6.2 Evidence classes

AVCS should collect evidence such as:

- logs
- screenshots
- traces
- console output
- timing metrics
- route/action captures
- security artifacts
- coverage artifacts

- evaluator notes
- environment and run metadata

5.6.3 Evidence rules

Evidence should be:

- linked to the run
- linked to domain where applicable
- linked to specific findings where possible
- timestamped
- immutable once finalized
- preserved even when a run fails

5.6.4 Minimum evidence principle

No major blocker, pass claim, or certification outcome should exist without supporting evidence.

5.6.5 Why it matters

Evidence is what gives AVCS authority.

AVCS-06.7 — Stage 7: Normalization and Finding Generation

5.7.1 Purpose

This stage converts raw evaluation outputs into structured certification findings.

5.7.2 What normalization should do

Normalization should:

- consolidate raw tool signals
- remove duplicate or overlapping issue noise

- assign primary domain ownership
- classify severity
- classify impact level
- attach evidence references
- identify blocker status
- preserve raw context for drill-down

5.7.3 Finding generation rules

A finding should only be created when the signal is strong enough to matter for certification or operator action.

Each finding should include:

- unique finding ID
- run linkage
- domain
- severity
- impact class
- title/summary
- affected surface
- description
- evidence references
- remediation note or next-action guidance
- blocker/conditional/warning classification

5.7.4 Why it matters

Operators need structured decisions and triage objects, not a flood of raw output.

AVCS-06.8 — Stage 8: Coverage Assembly

5.8.1 Purpose

This stage determines what the run actually covered and whether certification scope was fulfilled.

5.8.2 Coverage dimensions

Coverage should be assembled across:

- included domains
- required domains
- critical workflows
- pages/screens/surfaces
- APIs/endpoints where relevant
- risk classes
- certification requirements

5.8.3 Coverage states

Coverage elements should be labeled clearly as:

- covered
- partially covered
- failed during covered attempt
- not tested
- blocked
- deferred by run design

5.8.4 Why it matters

Certification requires knowing not just what failed, but what was never validated.

AVCS-06.9 — Stage 9: Certification Input Assembly

5.9.1 Purpose

This stage gathers all run outputs into the dataset used by the verdict engine.

5.9.2 Inputs to assemble

The certification input set should include:

- domain outcomes
- findings
- coverage status
- evidence completeness
- blocker flags
- scoring inputs if used
- policy checks
- baseline deltas if applicable
- run completeness state

5.9.3 Rules

This assembled certification object should be stable, reviewable, and preserved as a first-class run artifact.

5.9.4 Why it matters

This is the bridge between raw evaluation and final certification judgment.

AVCS-06.10 — Stage 10: Verdict Computation

5.10.1 Purpose

This stage computes the run's certification outcome.

5.10.2 Verdict inputs

Verdict computation should consider:

- hard blockers
- severity distribution
- required-domain status
- evidence completeness
- coverage sufficiency
- score/weight results where applicable
- policy rules
- freshness or validity of reused evidence if relevant

5.10.3 Possible outputs

The verdict stage should produce one of the controlled certification states, such as:

- PASS
- PASS_WITH_WARNINGS
- CONDITIONAL_PASS
- FAIL
- BLOCKED

5.10.4 Why it matters

This is where AVCS becomes a release-decision support system rather than only a test executor.

AVCS-06.11 — Stage 11: Finalization and Persistence

5.11.1 Purpose

After verdict computation, AVCS should finalize the run and preserve its outputs.

5.11.2 What should be stored

Finalized storage should include:

- run manifest
- domain outputs
- findings
- evidence inventory
- coverage summary
- certification input object
- final verdict
- timestamps
- rerun linkage
- operator-facing summary object

5.11.3 Finalization rules

Once finalized:

- results should be immutable except for explicit append-only annotations
- reruns should create new run records, not overwrite history
- any later override or review should be logged as a separate event

5.11.4 Why it matters

A certification run must remain auditable after it completes.

---

6. Execution states

AVCS runs should move through clear lifecycle states.

Recommended states:

- queued
- initializing
- preflight
- executing
- evidence-collecting
- normalizing
- assembling
- verdict-ready
- finalized
- failed
- blocked
- canceled if policy allows manual cancellation

These states should be visible to operators and recorded in run history.

---

7. Execution modes

7.1 Full execution

All included domains run according to the selected run type.

7.2 Selective execution

Only chosen domains or scopes run, where policy allows.

7.3 Rerun execution

A new run is created to retest failed, changed, or targeted surfaces.

7.4 Comparative execution

Current results are assembled alongside a previous baseline for regression-aware interpretation.

These modes give AVCS flexibility without sacrificing structure.

---

8. Isolation and reproducibility rules

AVCS should make reasonable effort to ensure comparable results across runs.

That means:

- stable target identity
- stable run definition
- explicit environment/profile context
- explicit baseline selection
- preserved evaluator versions where relevant
- preserved policy version or rule snapshot where relevant

Absolute determinism is not always possible, but traceable reproducibility should be the standard.

---

9. Failure-handling rules

The execution model should define what happens when execution itself breaks.

9.1 Preflight failure

Run becomes blocked or failed-initialization with reasons.

9.2 Domain crash or interruption

Affected domain should be marked blocked, failed, or indeterminate with evidence preserved.

9.3 Evidence collector failure

Certification should degrade appropriately; required evidence loss may block final verdict.

9.4 Incomplete run

If required domains do not complete, final verdict may become BLOCKED rather than PASS/FAIL.

This prevents AVCS from producing false confidence after broken execution.

---

10. Operator visibility during execution

Operators should be able to see:

- current run state
- completed domains
- active domain
- early blockers
- evidence being collected
- partial findings as they emerge where appropriate
- estimated completeness by scope, not time promise

The goal is high visibility without making the system noisy or misleading.

---

11. Canonical execution statement

Use a stable summary statement for this section:

AVCS executes certification through structured runs that resolve a target, validate prerequisites, evaluate selected assurance domains, collect evidence, normalize findings, assemble coverage and policy inputs, and compute a controlled release-readiness verdict.

---

12. Section close

AVCS-06 should make the execution model unmistakable:

- every run targets a specific build
- every run starts from an explicit manifest
- every run resolves domain scope before execution
- every meaningful result is backed by evidence
- raw outputs are normalized into findings and coverage
- certification verdicts are assembled, not improvised
- run history is preserved immutably

That gives AVCS an execution backbone strong enough for enterprise-grade use.

AVCS-07

AVCS-07 — Evidence Model

1. Purpose of this section

This section defines the evidence system used by **Axion Verification & Certification Suite (AVCS)**.

If **AVCS-06** defines how runs execute, **AVCS-07** defines how AVCS proves what happened during those runs and what supports each result, finding, and certification verdict.

This section is foundational because AVCS is not supposed to be a trust-me system. Its authority depends on evidence. A run result without evidence may be interesting, but it is not strong certification material.

---

2. Core evidence statement

AVCS must require **structured, attributable, preserved evidence** for all meaningful certification outcomes.

That means:

- every important domain result should have supporting proof
- every major finding should link to evidence
- every verdict should be explainable through evidence-backed inputs
- missing required evidence should weaken or block certification

Evidence is not a side effect of testing.
It is a first-class certification asset.

---

3. Evidence model principle

The AVCS evidence model should follow seven principles:

3.1 Attributable

Every evidence item must be traceable to a specific run and, where relevant, to a domain, workflow, surface, or finding.

3.2 Contextual

Evidence should preserve enough surrounding context to explain what happened, not just that something happened.

3.3 Durable

Evidence should remain available after run completion for audit, review, comparison, and remediation.

3.4 Immutable after finalization

Once a run is finalized, evidence should not be silently altered.

3.5 Structured

Evidence should not exist only as loose blobs. It should have metadata, type, timestamps, and linkage.

3.6 Proportional

Higher-severity findings and stronger certification claims should require stronger evidence.

3.7 Reviewable

Evidence should be easy for operators to inspect through AVCS surfaces without digging through raw internals unnecessarily.

These principles make evidence useful as a certification substrate rather than a storage dump.

---

4. Why AVCS needs a formal evidence model

Without a formal evidence model, AVCS would degrade into:

- raw test output without certification trust
- pass/fail claims that cannot be defended
- findings that are hard to triage
- historical runs that are not auditable

- verdicts that feel subjective instead of disciplined

A formal evidence model ensures that AVCS can answer:

- what exactly was tested
- what happened during evaluation
- what supports this finding
- what supports this final verdict
- what changed compared to prior runs

That is what turns validation into certification.

---

5. Evidence classes

AVCS should support a defined set of evidence classes rather than treating all artifacts the same.

AVCS-07.1 — Run Identity Evidence

5.1.1 Purpose

Run identity evidence proves the identity and framing of the certification event itself.

5.1.2 Examples

This includes:

- run manifest
- run ID
- target build ID
- target version or artifact reference
- run type

- policy context
- environment/profile context
- initiating actor
- timestamps
- rerun linkage
- baseline linkage if applicable

5.1.3 Why it matters

Without identity evidence, the rest of the evidence cannot be trusted in context because it is unclear what run it belongs to.

AVCS-07.2 — Execution Evidence

5.2.1 Purpose

Execution evidence proves what the system actually did during the run.

5.2.2 Examples

This includes:

- evaluator start and end records
- execution logs
- domain execution status transitions
- command/runtime outputs
- route/action traces
- step-completion records
- preflight results

- failure interruption records

5.2.3 Why it matters

Execution evidence shows the mechanics of the run rather than only the final conclusion.

AVCS-07.3 — Observation Evidence

5.3.1 Purpose

Observation evidence captures what AVCS saw during testing.

5.3.2 Examples

This includes:

- screenshots
- DOM or UI state captures
- response payload captures
- console logs
- interaction traces
- error messages
- workflow state snapshots
- visible failure captures
- captured comparison deltas

5.3.3 Why it matters

Observation evidence is often the most operator-legible proof because it shows the state or behavior directly.

AVCS-07.4 — Metric Evidence

5.4.1 Purpose

Metric evidence captures quantitative signals used in evaluation or gating.

5.4.2 Examples

This includes:

- load times
- route transition timings
- action latency
- bundle size
- memory pressure signals
- repeated-run performance variance
- response time distributions
- stability counters
- error frequency measures

5.4.3 Why it matters

Some domains, especially performance and stability, cannot be certified well using screenshots or log snippets alone.

AVCS-07.5 — Security Evidence

5.5.1 Purpose

Security evidence captures proof relevant to security posture evaluation and findings.

5.5.2 Examples

This includes:

- vulnerability scan outputs

- auth boundary test results
- permission test results
- leaked-secret detections
- exposed endpoint captures
- config exposure findings
- unsafe input handling records
- security policy check output

5.5.3 Why it matters

Security findings are often release-blocking, so the evidence must be strong, attributable, and reviewable.

AVCS-07.6 — Coverage Evidence

5.6.1 Purpose

Coverage evidence proves what parts of the certification scope were actually exercised.

5.6.2 Examples

This includes:

- domain coverage records
- workflow coverage maps
- page/surface coverage lists
- endpoint coverage traces
- requirement-to-test mapping
- partial/blocked/not-tested annotations

- deferred-scope declarations

5.6.3 Why it matters

Coverage evidence prevents the suite from implying that “everything was validated” when only selected parts were touched.

AVCS-07.7 — Finding Evidence

5.7.1 Purpose

Finding evidence is the subset of evidence directly attached to a specific issue, blocker, or warning.

5.7.2 Examples

This includes:

- error screenshot for a broken workflow
- trace for a failing action step
- vulnerability artifact for a security blocker
- metric record for a performance regression
- log excerpt linked to a startup failure
- inaccessible control capture linked to accessibility issue

5.7.3 Why it matters

Operators need issue-level proof, not just run-level proof.

AVCS-07.8 — Verdict Evidence

5.8.1 Purpose

Verdict evidence supports the final certification outcome.

5.8.2 Examples

This includes:

- required-domain completion state
- blocker summary
- evidence completeness summary
- coverage sufficiency summary
- policy check results
- scoring summary where applicable
- baseline comparison outcome where relevant

5.8.3 Why it matters

The final certification state should be defensible at the decision level, not only at the individual test level.

---

6. Evidence schema expectations

Each evidence item should have structured metadata.

At minimum, an evidence record should include:

- evidence ID
- run ID
- evidence type
- domain linkage if applicable
- workflow/surface linkage if applicable
- finding linkage if applicable

- timestamp
- source/evaluator origin
- storage reference
- summary or descriptor
- integrity/finalization status

Optional but valuable fields:

- severity relevance
- policy relevance
- baseline-comparison tag
- environment/profile tag
- operator-visibility hint

This structure makes evidence queryable and reviewable.

---

7. Evidence strength model

Not all evidence carries equal strength. AVCS should define levels of evidence strength.

7.1 Strong evidence

Evidence is strong when it is:

- direct
- attributable
- clear
- reproducible or corroborated

- linked tightly to the affected surface or finding

Examples:

- screenshot of broken UI state with step trace
- captured unauthorized access result
- exact failing route trace with visible error
- measured latency exceeding threshold with baseline comparison

7.2 Moderate evidence

Evidence is moderate when it is useful but less conclusive alone.

Examples:

- partial logs without full trace context
- symptom screenshot without precise step chain
- warning signal without reproduced failure sequence

7.3 Weak evidence

Evidence is weak when it suggests a problem but does not strongly support certification-grade judgment alone.

Examples:

- vague console noise
- isolated comment without capture
- incomplete metric snapshot
- unlabeled artifact with unclear provenance

Critical verdicts should not rely primarily on weak evidence.

---

8. Evidence requirements by result class

AVCS should scale evidence requirements based on the importance of the claim.

8.1 Pass claims

Important pass claims should be backed by evidence showing:

- the relevant surface was actually exercised
- the required condition completed successfully
- no blocker condition was present for that surface

8.2 Warning findings

Warnings should usually have enough evidence for triage and prioritization.

8.3 Blocker findings

Blockers should require strong and explicit evidence whenever feasible.

8.4 Final verdicts

Final verdicts should require:

- run identity evidence
- domain outcome evidence
- blocker evidence where blockers exist
- coverage evidence
- evidence completeness summary
- policy-result evidence

This prevents the system from using the same proof standard for low-value observations and release-blocking conclusions.

---

9. Evidence linkage rules

Evidence should be linked in multiple directions.

9.1 Run linkage

Every evidence item must link to the run it belongs to.

9.2 Domain linkage

Where applicable, evidence should link to the relevant domain.

9.3 Surface linkage

Where possible, evidence should link to:

- workflow
- screen/page
- route
- endpoint
- module
- action step

9.4 Finding linkage

Evidence supporting a finding should be attached directly to that finding.

9.5 Verdict linkage

Evidence used in final verdict logic should be referenced by the verdict summary object.

This creates a navigable evidence graph rather than a loose bucket of files.

---

10. Evidence completeness rules

AVCS should evaluate not only the presence of evidence, but whether evidence is complete enough for certification.

10.1 Minimum completeness expectations

A run should have at minimum:

- manifest evidence
- execution trace of included domains
- outcome evidence for each required domain
- finding evidence for material failures
- coverage evidence
- final verdict evidence summary

10.2 Incomplete evidence behavior

If required evidence is missing:

- affected findings may be downgraded to indeterminate or blocked for review
- affected domain results may be weakened
- final certification may become BLOCKED if evidence gaps undermine trust

10.3 Why it matters

A run with missing proof should not silently retain full authority.

---

11. Evidence preservation rules

Evidence should remain durable after finalization.

11.1 Preservation expectations

AVCS should preserve:

- all evidence tied to finalized runs
- evidence references used by findings and verdicts
- enough raw context to support later review
- historical evidence needed for regression comparison

11.2 Finalization rule

After finalization:

- evidence content should be immutable
- annotations should be append-only
- reruns should generate new evidence records instead of replacing old ones

11.3 Why it matters

Certification history must remain reviewable over time.

---

12. Evidence reviewability rules

AVCS should present evidence at multiple levels.

12.1 Summary view

Operators should see:

- evidence availability by domain
- missing required evidence
- strongest evidence attached to major findings
- high-level verdict support summary

12.2 Drill-down view

Operators should be able to inspect:

- screenshots
- traces
- logs
- metric charts
- workflow step captures
- security artifacts
- coverage records

12.3 Why it matters

Evidence that cannot be reviewed efficiently is almost as bad as evidence that was never collected.

---

13. Evidence misuse protections

AVCS should explicitly guard against bad evidence practices.

It should prevent:

- orphaned artifacts with no run linkage
- unlabeled screenshots with unclear relevance
- verdicts based on stale evidence without disclosure
- silent replacement of evidence after finalization
- overreliance on raw logs when clearer evidence exists
- pass claims unsupported by any direct execution proof

These controls are essential for operator trust.

---

14. Evidence and reruns

Reruns should produce new evidence, not overwrite prior evidence.

When a rerun occurs, AVCS should preserve:

- original evidence set
- rerun evidence set
- reason for rerun
- linkage between old and new findings
- comparison visibility where relevant

This keeps remediation history intact and supports regression analysis.

---

15. Canonical evidence statement

Use a stable summary statement for this section:

AVCS requires structured, attributable, and preserved evidence for all meaningful domain outcomes, findings, and certification verdicts, so that release-readiness decisions remain reviewable, defensible, and audit-capable.

---

16. Section close

AVCS-07 should make one thing very clear:

- results are not enough
- evidence is not optional
- findings must be provable

- certification verdicts must be defensible
- historical runs must remain inspectable

That is what gives AVCS its authority as a certification system rather than just a test runner.

AVCS-08

AVCS-08 — Findings Model

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** represents problems, risks, warnings, and notable observations discovered during verification and certification runs.

If **AVCS-07** defines the evidence system, **AVCS-08** defines how AVCS turns evaluation signals into structured, operator-usable issue objects.

This section matters because raw failures, logs, screenshots, metrics, and scan outputs are not enough on their own. AVCS needs a disciplined findings model so operators can understand:

- what went wrong
- how serious it is
- what area is affected
- whether release is blocked
- what evidence supports the issue
- what should happen next

Without a proper findings model, AVCS becomes noisy, hard to triage, and weak as a certification system.

---

2. Core findings statement

AVCS must convert meaningful verification signals into **structured findings** that describe the issue, classify its seriousness, link the relevant evidence, indicate certification impact, and support remediation and release decision-making.

A finding is not just an error message.
It is the certification-facing representation of a meaningful quality signal.

---

3. Findings model principle

The AVCS findings model should follow eight principles:

3.1 Structured

Every meaningful issue should be represented in a standard format.

3.2 Evidence-linked

A finding must link to the evidence that supports it.

3.3 Actionable

A finding should be understandable enough to guide triage or remediation.

3.4 Scoped

A finding should clearly identify what domain, workflow, surface, or artifact it affects.

3.5 Severity-aware

A finding should indicate how serious the issue is.

3.6 Certification-aware

A finding should indicate whether it affects the release outcome and how.

3.7 Deduplicated

AVCS should avoid generating multiple redundant findings for the same underlying issue without reason.

3.8 Historically traceable

A finding should be comparable across reruns and future runs where possible.

These principles make findings useful as release-control objects rather than mere diagnostic fragments.

---

4. What counts as a finding

AVCS should not create a finding for every tiny signal. A finding should exist when the signal is meaningful enough to matter for certification, operator awareness, remediation, or historical comparison.

A finding is justified when one or more of the following is true:

- it indicates a failure in a required domain
- it reveals a release risk
- it represents a policy violation
- it blocks or conditions certification
- it materially degrades quality in a tested surface
- it indicates drift or regression from a prior accepted baseline
- it is important enough that operators should review or track it

This keeps the model from becoming bloated.

---

5. Finding classes

AVCS should distinguish different kinds of findings rather than treating everything as the same.

AVCS-08.1 — Failure Findings

5.1.1 Definition

A failure finding represents a condition where an evaluated surface did not meet its expected standard or did not complete successfully.

5.1.2 Examples

- primary save flow fails on submit
- route crashes during load
- startup command exits unexpectedly
- modal traps focus and cannot close
- required env contract is incomplete

5.1.3 Why it matters

Failure findings are the most direct representation of nonconforming behavior.

AVCS-08.2 — Risk Findings

5.2.1 Definition

A risk finding represents a condition that may not be an immediate execution failure but introduces a meaningful release concern.

5.2.2 Examples

- insecure default config present
- dependency vulnerability over allowed threshold
- missing observability hook for critical path
- poor recovery path in operator workflow
- performance margin dangerously close to policy limit

5.2.3 Why it matters

Enterprise certification depends on identifying risk, not only visible breakage.

AVCS-08.3 — Regression Findings

5.3.1 Definition

A regression finding represents a deterioration compared to a previous successful or accepted baseline.

5.3.2 Examples

- workflow that previously passed now fails
- route latency increased beyond allowed delta
- coverage decreased in required surface

- finding severity footprint worsened relative to baseline
- UI layout previously stable now breaks on standard viewport

5.3.3 Why it matters

Regression findings help AVCS judge build quality over time rather than only in isolation.

AVCS-08.4 — Policy Findings

5.4.1 Definition

A policy finding represents a violation of AVCS certification policy, gating rules, or minimum evidence or coverage requirements.

5.4.2 Examples

- required domain not completed for selected run type
- required evidence missing for blocker-class issue
- required pre-deployment run is stale
- minimum coverage threshold not met
- forbidden override path attempted without authorization

5.4.3 Why it matters

Some certification failures are about process and compliance, not only product behavior.

AVCS-08.5 — Informational Findings

5.5.1 Definition

An informational finding represents a notable condition that should be visible to operators but does not materially affect current certification status.

5.5.2 Examples

- minor UI inconsistency in non-critical surface
- low-priority log noise
- non-blocking performance observation
- secondary accessibility improvement opportunity
- low-impact documentation gap

5.5.3 Why it matters

Not every useful signal should be escalated into a warning or blocker.

---

6. Required finding fields

Every AVCS finding should follow a stable schema.

At minimum, a finding should include:

- finding ID
- run ID
- target build ID
- finding class
- primary domain
- severity
- impact class
- status
- title
- summary

- detailed description
- affected surface or surfaces
- workflow/page/route/module linkage where applicable
- evidence references
- evaluator/source origin
- certification effect
- created timestamp

Strongly recommended additional fields:

- baseline comparison reference
- probable cause signal
- remediation note
- owner/team field if routing exists
- policy rule reference if relevant
- duplicate-of or related-finding linkage
- rerun resolution linkage

This schema gives findings enough structure to be actionable and historically useful.

---

7. Severity model

AVCS should classify findings by severity so operators can prioritize correctly.

7.1 Critical

Used when the issue represents an immediate or severe release problem.

Typical cases:

- auth bypass
- app fails to boot
- critical workflow completely broken
- exposed secret
- severe blocker in required deployment path

Critical findings should usually block certification.

7.2 High

Used when the issue materially harms release readiness or a required surface, but may not always be as catastrophic as critical.

Typical cases:

- major workflow degradation
- serious performance regression in core path
- inaccessible primary action
- missing required startup contract piece
- significant permission failure short of total bypass

High findings often block or strongly condition certification.

7.3 Medium

Used when the issue is meaningful and should be addressed, but may not automatically block release unless policy or accumulation rules say otherwise.

Typical cases:

- broken secondary workflow
- significant but recoverable UX friction
- moderate stability issue
- important but non-primary accessibility issue
- non-trivial supportability gap

Medium findings often produce warnings or conditional outcomes.

7.4 Low

Used when the issue is real but limited in impact.

Typical cases:

- minor UI misalignment
- non-blocking copy or state issue
- low-risk observability gap
- cosmetic inconsistency in secondary surface

Low findings usually do not block certification on their own.

7.5 Info

Used when the signal is worth preserving but is not really a defect or release issue.

Typical cases:

- note about baseline variance
- optional improvement opportunity
- informative trace from rerun context

Info findings should not affect verdict directly.

---

8. Impact class model

Severity and certification impact should not be the same field. AVCS should also classify the release effect of a finding.

Recommended impact classes:

8.1 Release Blocker

The finding directly prevents certification approval under current policy.

8.2 Conditional Blocker

The finding does not force total failure, but release requires explicit override, exception handling, or additional review.

8.3 Warning

The finding affects quality and should be visible, but does not independently block release.

8.4 Observation

The finding is informative and preserved for visibility or trend tracking.

This distinction matters because two high-severity findings may not always carry the same gate effect.

---

9. Finding status model

Findings should also have lifecycle status so AVCS can track them cleanly across reruns and review.

Recommended statuses:

- open
- confirmed

- under review
- mitigated
- resolved
- accepted by override
- superseded
- false positive
- informational only

This makes AVCS useful over time, not just at run completion.

---

10. Domain ownership rules

Every finding should have a **primary domain owner**, even if it touches multiple domains.

Examples:

- hidden primary button with keyboard inaccessibility may touch UI Quality and Accessibility, but AVCS should assign one primary domain and optionally list related domains
- startup failure may involve Build Integrity and Deployment Readiness, but one should be designated as primary

This avoids confusion in triage, scoring, and reporting while still allowing cross-domain relationships.

---

11. Finding creation rules

AVCS should create findings through explicit normalization logic.

A finding should be created when:

- a domain evaluator returns a meaningful nonconformance
- a policy rule is violated
- a regression delta exceeds allowed tolerance
- a risk signal crosses a configured threshold
- evidence shows a material issue in a required surface

A finding should usually not be created when:

- the signal is too weak or ambiguous
- the issue is duplicate noise from the same root cause
- the evaluator did not collect enough evidence to support meaningful representation
- the signal is outside scope and not relevant to certification

This keeps findings high-signal.

---

12. Deduplication and grouping

AVCS should avoid flooding operators with many copies of the same underlying issue.

12.1 Deduplication rule

If multiple raw signals point to the same root problem, AVCS should prefer one consolidated finding with related evidence references rather than many near-duplicate findings.

12.2 Grouping rule

Closely related findings may be grouped under a parent issue cluster where useful, especially for:

- repeated workflow failures from one backend contract issue
- many UI symptoms from one layout system break

- multiple affected routes from one auth rule defect

12.3 Why it matters

Operators need triage clarity, not volume inflation.

---

13. Finding-to-evidence rules

Every meaningful finding must link to supporting evidence.

At minimum, a finding should have:

- direct supporting evidence
- enough context to explain the issue
- evidence strong enough for its severity and impact class

Critical and blocker findings should have especially strong evidence whenever feasible.

A finding without evidence should not carry full certification authority.

---

14. Finding-to-verdict rules

Findings are one of the primary inputs to certification verdict computation.

AVCS should ensure that findings influence verdict through:

- severity distribution
- blocker count
- affected required domains
- impacted critical workflows
- policy violation status

- evidence completeness
- regression significance

This means findings are not just report objects. They are control inputs.

---

15. Finding resolution and reruns

Findings should remain historically traceable even after remediation.

When a rerun occurs, AVCS should preserve:

- the original finding
- the rerun result for that issue
- whether the finding remained, changed, or resolved
- evidence attached to the rerun
- linkage between old and new status

Suggested resolution relationships:

- persists
- resolved in rerun
- partially mitigated
- worsened
- replaced by related finding
- determined false positive

This gives AVCS strong remediation visibility.

---

16. Finding presentation rules

Operators should be able to review findings in several useful views.

Recommended views:

- by severity
- by impact class
- by domain
- by workflow or surface
- by open vs resolved state
- by blocker status
- by regression vs net-new
- by run or target build

Each finding view should expose:

- concise title
- severity
- impact
- affected surface
- evidence preview
- certification effect
- next-action or remediation hint

This makes the model operationally useful.

---

17. Findings misuse protections

AVCS should guard against bad finding behavior.

It should prevent:

- evidence-free blocker findings
- duplicate issue spam
- severity inflation without criteria
- weak informational signals being presented like hard failures
- findings with unclear affected surface
- hidden policy findings that affect verdict but are not visible to operators

These protections preserve trust in the suite.

---

18. Canonical findings statement

Use a stable summary statement for this section:

AVCS findings are structured, evidence-linked issue objects that classify failures, risks, regressions, and policy violations by domain, severity, and certification impact so operators can triage quality problems and understand release consequences clearly.

---

19. Section close

AVCS-08 should leave the findings model clear:

- raw signals become structured findings
- findings have severity and certification impact
- findings must link to evidence

- findings should be deduplicated and actionable
- findings feed directly into verdict logic
- findings remain traceable across reruns and history

That gives AVCS a usable triage and release-control language.

AVCS-09

AVCS-09 — Certification Decision Model

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** converts domain outcomes, findings, evidence, coverage, and policy results into a controlled certification verdict.

If **AVCS-08** defines how issues are represented, **AVCS-09** defines how AVCS decides what those issues mean for release readiness.

This section is critical because AVCS is not just a reporting system. It is a certification system. That means it must produce a disciplined release decision model rather than leaving operators to guess from raw outputs.

---

2. Core decision statement

AVCS must produce a **controlled certification outcome** for each run by evaluating:

- required-domain status
- blocker conditions
- finding severity and impact
- coverage sufficiency
- evidence completeness
- policy requirements
- score or weighted readiness inputs where used

The certification decision should never be based on one flat number alone, one isolated test result alone, or operator intuition alone.

It should be a structured verdict assembled from defined decision inputs.

---

3. Decision model principle

The AVCS decision model should follow eight principles:

3.1 Controlled

Verdicts must come from declared rules, not ad hoc judgment.

3.2 Multi-input

Verdicts must consider multiple certification signals, not only pass/fail counts.

3.3 Evidence-dependent

A decision should weaken or block when required proof is missing.

3.4 Coverage-aware

A build should not be certified as fully ready when meaningful required scope was never tested.

3.5 Blocker-sensitive

Critical issues must be able to override aggregate strength elsewhere.

3.6 Policy-aware

Decision logic must honor run-type and build-class policy requirements.

3.7 Explainable

The system must be able to state why a verdict was reached.

3.8 Override-disciplined

Where overrides are allowed, they must be explicit, visible, and governed.

These principles make the verdict trustworthy.

---

4. Canonical certification outcomes

AVCS should use a fixed, controlled set of certification states.

AVCS-09.1 — PASS

4.1.1 Meaning

The build is certified as ready to progress under the selected run type and policy context.

4.1.2 Typical conditions

PASS generally requires:

- all required domains completed successfully or within allowed tolerance
- no release blockers
- required coverage met
- required evidence complete
- no policy violations that prevent release
- readiness score, if used, above threshold

4.1.3 Interpretation

PASS means the build is deployable or progression-ready within current policy.

AVCS-09.2 — PASS_WITH_WARNINGS

4.2.1 Meaning

The build is acceptable for progression, but non-blocking issues remain and must be visible.

4.2.2 Typical conditions

PASS_WITH_WARNINGS generally requires:

- no hard release blockers
- required domains sufficiently complete
- warnings or medium/low issues exist
- evidence remains complete enough for trust
- policy permits progression with visible concerns

4.2.3 Interpretation

This is a controlled green-with-visibility state, not a hidden soft fail.

AVCS-09.3 — CONDITIONAL_PASS

4.3.1 Meaning

The build may proceed only if explicit operator review, override, or additional controlled acceptance occurs.

4.3.2 Typical conditions

CONDITIONAL_PASS generally applies when:

- there are non-ideal but not automatic-fail conditions
- policy allows controlled exceptions
- quality concerns are significant enough to require explicit acknowledgement
- required domains mostly completed, but issues remain with meaningful impact
- evidence is sufficient to understand the risk

4.3.3 Interpretation

This is a policy-mediated release state, not a normal pass.

AVCS-09.4 — FAIL

4.4.1 Meaning

The build is not certified and is not ready to progress under current conditions.

4.4.2 Typical conditions

FAIL generally applies when:

- one or more release blockers are present
- critical required workflows fail

- required domains fail materially
- security or deployment blockers exist
- score falls below allowed threshold and policy does not permit override
- policy-required quality conditions are not met

4.4.3 Interpretation

The build requires remediation before meaningful progression.

AVCS-09.5 — BLOCKED

4.5.1 Meaning

AVCS cannot responsibly issue a normal certification verdict because the run or its evidence is incomplete, invalid, or non-certifiable.

4.5.2 Typical conditions

BLOCKED generally applies when:

- target identity is unclear
- required domains did not execute
- required evidence is missing
- preflight failed
- run was interrupted in a way that undermines trust
- required policy inputs were unavailable
- coverage insufficiency makes certification invalid

4.5.3 Interpretation

BLOCKED is not the same as FAIL.
FAIL says the build was tested and did not meet the standard.
BLOCKED says AVCS could not responsibly complete the certification judgment.

---

5. Decision inputs

AVCS should compute verdicts from explicit input classes.

AVCS-09.6 — Domain Outcome Inputs

These include:

- pass/fail/partial/blocked state by domain
- required-domain completion
- domain-specific blocker flags
- cross-domain consistency issues

These are the core quality inputs.

AVCS-09.7 — Finding Inputs

These include:

- number of findings
- severity distribution
- impact-class distribution
- blocker-class findings
- critical workflow findings
- policy findings
- regression findings

Findings give the verdict model its risk awareness.

AVCS-09.8 — Coverage Inputs

These include:

- required domain coverage
- workflow coverage
- surface coverage
- risk-class coverage
- deferred/not-tested/blocked coverage states
- minimum threshold compliance

Coverage prevents false confidence.

AVCS-09.9 — Evidence Inputs

These include:

- run identity completeness
- required evidence presence
- strength of evidence for major findings
- evidence linkage completeness
- verdict-support evidence availability

Evidence determines how much authority the run should carry.

AVCS-09.10 — Policy Inputs

These include:

- required run types
- freshness windows
- mandatory domains
- gating thresholds
- override rules
- build-class-specific certification requirements
- mandatory blocker rules

Policy transforms a general validation result into an enforceable release decision.

AVCS-09.11 — Scoring Inputs

If AVCS uses weighted scoring, the verdict model may also consume:

- domain scores
- total readiness score
- threshold compliance
- trend or regression delta
- score floor by required domain

Scoring should support the decision model, not replace it.

---

6. Hard-stop decision logic

AVCS must define hard-stop rules that override score and broad pass patterns.

A build should usually not receive PASS or PASS_WITH_WARNINGS if any of the following exists:

- app cannot boot or initialize in required target profile
- critical workflow failure in a required path
- auth boundary failure or severe permission defect
- exposed secret or severe security weakness
- required deployment-readiness blocker
- missing required evidence for a blocker-class issue
- required domain not executed
- policy-required gate not satisfied
- certification scope too incomplete to trust

These are examples of conditions that should force FAIL or BLOCKED depending on the reason.

---

7. Difference between FAIL and BLOCKED

This distinction should be explicit because it matters operationally.

7.1 FAIL

Use FAIL when:

- AVCS had enough valid information to judge the build
- the build did not meet the required standard

7.2 BLOCKED

Use BLOCKED when:

- AVCS could not complete a trustworthy judgment

- run incompleteness or evidence deficiency prevents valid certification

This distinction helps operators know whether they are dealing with:

- a product quality problem
  or
- a certification process completeness problem

Sometimes both exist, but the final state should reflect what most accurately describes the gating condition.

---

8. Conditional-pass logic

AVCS should explicitly define when CONDITIONAL_PASS is allowed.

Typical use cases:

- issue severity is meaningful but policy permits explicit acceptance
- non-primary surfaces have unresolved but bounded issues
- evidence is complete and risk is understood
- required domains passed, but one or more conditions require acknowledgement
- override authority exists and is properly scoped

CONDITIONAL_PASS should not be used to hide:

- severe security defects
- critical workflow breakage
- missing required evidence
- major required-domain incompleteness

This state must remain disciplined or it will erode trust.

---

9. Pass-with-warnings logic

PASS_WITH_WARNINGS should represent a real pass state with visible defects, not a loophole.

Typical use cases:

- low/medium non-blocking issues remain
- minor UX/UI issues exist outside critical paths
- there are supportability or documentation warnings that do not block current release
- performance margin is acceptable but worth flagging
- informational regressions exist below blocking threshold

This state is useful when the build is genuinely releasable, but operators should not lose visibility into quality debt.

---

10. Decision assembly sequence

AVCS should compute verdicts in a stable order.

Recommended sequence:

1. validate run completeness
2. validate evidence completeness
3. validate required-domain completion
4. check hard blockers
5. check policy violations
6. evaluate coverage sufficiency

7. incorporate score/readiness results if used
8. determine highest-valid verdict state
9. attach reasoning summary
10. finalize certification result object

This sequence ensures that foundational invalidity is considered before softer optimization signals.

---

11. Decision reasoning summary

Every certification result should include a concise reasoning summary.

That summary should answer:

- what verdict was issued
- what most strongly drove that verdict
- whether blockers were present
- whether coverage and evidence were sufficient
- whether policy conditions affected the decision
- whether override is required or permitted

This is critical for operator trust.

---

12. Decision object structure

The certification result should be represented as a structured object.

At minimum, it should include:

- verdict state
- run ID
- target build ID
- run type
- decision timestamp
- required-domain status summary
- blocker summary
- warning summary
- coverage sufficiency status
- evidence completeness status
- policy compliance status
- score summary if used
- override-required flag
- reasoning summary
- linked major findings
- linked evidence summary

This makes the verdict auditable and machine-usable.

---

13. Decision confidence concept

AVCS may optionally represent **decision confidence** as a secondary signal.

This should never override the main verdict, but it can help explain result strength.

Confidence may be influenced by:

- evidence strength
- coverage completeness
- stability of evaluator outputs
- consistency with prior runs
- degree of ambiguity in the findings

Example:

- PASS with high confidence
- CONDITIONAL_PASS with moderate confidence
- BLOCKED with low confidence due to incomplete evidence

This can be useful, but it must remain secondary to the actual verdict state.

---

14. Override model linkage

AVCS should support policy-controlled overrides, but the base decision should remain intact.

That means:

- the original verdict should remain preserved
- override action should be a separate append-only decision event
- override authority should be explicit
- override reason should be required
- overridden blockers should remain visible

This ensures that overrides do not erase certification truth.

---

15. Multi-run and freshness considerations

For release decisions that rely on multiple runs or prior runs, AVCS should define freshness rules.

This may include:

- how recent a Security Run must be to count
- whether prior Accessibility or Performance evidence can be reused
- how Full Certification Run and Pre-Deployment Gate Run interact
- when regression comparison is mandatory

A build should not receive a strong verdict using stale supporting runs without disclosure.

---

16. Decision misuse protections

AVCS should guard against weak certification behavior.

It should prevent:

- full PASS with missing required evidence
- PASS based only on score despite hard blockers
- warnings being hidden from operator view
- CONDITIONAL_PASS being used as a catch-all escape hatch
- FAIL being used when BLOCKED is the more truthful state
- operator-facing verdicts with no reasoning summary
- silent override of blocked or failed states

These protections preserve integrity.

---

17. Canonical decision statement

Use a stable summary statement for this section:

AVCS determines certification outcomes by combining domain results, findings, coverage sufficiency, evidence completeness, and policy rules into a controlled verdict state of PASS, PASS_WITH_WARNINGS, CONDITIONAL_PASS, FAIL, or BLOCKED.

---

18. Section close

AVCS-09 should make the certification logic clear:

- verdicts are rule-based
- blockers can override scores
- missing proof can block certification
- incomplete scope weakens authority
- policy shapes what counts as releasable
- the system must explain why it decided what it decided

That gives AVCS a real release-decision backbone.

AVCS-10

AVCS-10 — Coverage Model

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** measures and represents coverage during verification and certification.

If **AVCS-09** defines how verdicts are computed, **AVCS-10** defines how AVCS answers a separate but equally important question:

What was actually tested, to what depth, and against which required certification surfaces?

This section matters because “a lot of tests ran” is not the same as “the build was fully covered.” AVCS needs a formal coverage model so certification can distinguish between:

- validated scope
- partially validated scope
- failed validated scope
- untouched required scope
- intentionally deferred scope
- blocked scope that could not be evaluated

Without this, PASS states become easy to overstate.

---

2. Core coverage statement

AVCS must model coverage as the **measured relationship between intended certification scope and actually evaluated scope** across domains, workflows, surfaces, risk areas, and release requirements.

Coverage is not merely execution count.
Coverage is proof of scope fulfillment.

That means AVCS should measure not only:

- whether checks executed
  but also:
- what those checks touched
- whether required areas were included
- whether tested areas completed successfully
- whether gaps remain in required certification territory

---

3. Coverage model principle

The AVCS coverage model should follow eight principles:

3.1 Scope-aware

Coverage must be measured against declared intended scope, not against an undefined idea of completeness.

3.2 Multi-dimensional

Coverage must include more than test-domain participation; it must span workflows, surfaces, and requirements.

3.3 Stateful

Coverage items must reflect whether they passed, failed, were partial, blocked, or were not tested.

3.4 Policy-aware

Required coverage thresholds must vary appropriately by run type and build class.

3.5 Explainable

Operators must be able to see exactly what is covered and what is missing.

3.6 Non-inflationary

AVCS must not over-credit shallow checks as full coverage.

3.7 Historically comparable

Coverage should be trackable across runs to detect regressions or drift.

3.8 Verdict-relevant

Coverage must affect certification authority, not merely appear in a side panel.

These principles keep coverage meaningful.

---

4. What coverage means in AVCS

In AVCS, coverage should be interpreted as:

the extent to which required certification scope has been evaluated with sufficient depth and evidence for the selected run type and policy context

That means coverage must always be understood relative to:

- the target build
- the selected run type
- the policy requirements
- the declared included domains
- the expected critical workflows and surfaces
- the evidence and completion state of those validations

So coverage is not universal. It is contextual and run-scoped.

---

5. Coverage dimensions

AVCS should measure coverage across multiple dimensions, not just one aggregate percentage.

AVCS-10.1 — Domain Coverage

5.1.1 Definition

Domain Coverage measures whether each included or required assurance domain was evaluated to the necessary depth.

5.1.2 Examples

- Build Integrity covered
- Security Posture partially covered
- Accessibility not included by run design
- Deployment Readiness blocked during execution

5.1.3 Why it matters

This is the top-level coverage layer used in certification summaries.

AVCS-10.2 — Workflow Coverage

5.2.1 Definition

Workflow Coverage measures whether critical end-to-end user or operator flows were exercised.

5.2.2 Examples

- create-project flow covered and passed
- review-and-approve flow covered but failed
- export flow not tested
- onboarding flow partially covered

5.2.3 Why it matters

A build can look broadly tested while still missing key real-world usage paths.

AVCS-10.3 — Surface Coverage

5.3.1 Definition

Surface Coverage measures whether relevant UI, API, route, page, component, and operational surfaces were included in testing.

5.3.2 Examples

- dashboard page covered
- settings route not tested
- auth callback surface failed during execution
- admin control panel intentionally deferred

5.3.3 Why it matters

This helps operators see where the tested footprint actually was.

AVCS-10.4 — Requirement Coverage

5.4.1 Definition

Requirement Coverage measures whether certification-relevant requirements and release expectations were validated.

5.4.2 Examples

- required startup viability validated
- required auth boundary validation completed
- required evidence completeness check missing
- required health endpoint verification passed

5.4.3 Why it matters

Certification must connect back to required release conditions, not just test artifacts.

AVCS-10.5 — Risk-Class Coverage

5.5.1 Definition

Risk-Class Coverage measures whether high-risk and high-impact areas received appropriate attention.

5.5.2 Examples

- critical security risk class covered
- primary workflow risk class covered
- high-risk admin surface not tested
- low-risk cosmetic surfaces intentionally deferred

5.5.3 Why it matters

Not all untested areas are equally important. Risk weighting matters.

AVCS-10.6 — Evidence Coverage

5.6.1 Definition

Evidence Coverage measures whether covered scope also has the evidence needed to support its claimed result.

5.6.2 Examples

- workflow marked covered with screenshots and trace
- domain marked passed but evidence incomplete
- policy check executed but missing linked artifact
- blocker recorded with full evidence pack

5.6.3 Why it matters

A scope item should not count as strong coverage if the supporting proof is weak or missing.

---

6. Coverage states

Each coverage item should be represented with a clear state rather than a vague boolean.

Recommended states:

6.1 Covered

The item was evaluated at the required depth and completed with sufficient evidence.

6.2 Covered-Failed

The item was evaluated, but the evaluated result failed.

6.3 Partially Covered

The item was only partly evaluated, or depth/evidence was below full expectation.

6.4 Blocked

The item could not be evaluated due to execution, environment, dependency, or prerequisite failure.

6.5 Not Tested

The item was in relevant scope but was not exercised.

6.6 Deferred by Design

The item was intentionally excluded for the selected run type or policy path.

These states are much more useful than a single yes/no.

---

7. Covered vs passed

AVCS must keep a hard distinction between **coverage** and **success**.

A workflow can be:

- covered and passed
- covered and failed
- partially covered
- blocked
- not tested

This distinction is essential because a failed workflow still counts as evaluated scope, while an untested workflow does not.

That means:

- coverage answers “was it exercised?”
- results answer “what happened when it was exercised?”

Both matter separately.

---

8. Coverage depth model

Not all coverage is equal. AVCS should define depth tiers.

8.1 Shallow coverage

The surface was touched lightly, often for existence or initial response only.

Examples:

- route loads
- page renders shell
- basic endpoint responds

8.2 Standard coverage

The surface was exercised through normal expected use at meaningful depth.

Examples:

- critical workflow executed successfully
- form submitted with validation path
- auth boundary checked for normal role conditions

8.3 Deep coverage

The surface was exercised through normal and adverse conditions with richer evidence and edge-path validation.

Examples:

- workflow tested across success, error, and recovery states
- performance measured repeatedly
- permissions tested across role boundaries
- accessibility tested through full primary path

AVCS should avoid scoring shallow touches as full equivalent to deep validation.

---

9. Coverage requirements by run type

Coverage expectations should vary by run type.

9.1 Smoke Run

Expected coverage:

- foundational build surfaces
- startup path
- minimal critical-route coverage

Not expected:

- deep workflow coverage
- complete accessibility or enterprise-readiness coverage

9.2 Functional Run

Expected coverage:

- critical workflow coverage
- primary behavior paths
- important error handling paths

9.3 UI/UX Run

Expected coverage:

- major interface surfaces
- workflow clarity paths
- state coverage for loading/empty/error/success

9.4 Security Run

Expected coverage:

- auth surfaces
- permission surfaces
- config exposure surfaces
- vulnerability and secret-risk surfaces

9.5 Performance Run

Expected coverage:

- performance-sensitive routes and interactions
- key timing surfaces
- repeated stability paths

9.6 Accessibility Run

Expected coverage:

- primary interactive paths
- forms, dialogs, controls
- keyboard and focus-critical surfaces

9.7 Regression Run

Expected coverage:

- previously certified or baseline-critical surfaces
- changed or high-risk surfaces
- drift-sensitive metrics and findings

9.8 Full Certification Run

Expected coverage:

- all required domains
- all critical workflows
- required release surfaces
- required risk classes
- required evidence layers

9.9 Pre-Deployment Gate Run

Expected coverage:

- all policy-mandated final-release surfaces
- final blocker-sensitive scope
- required freshness and completeness checks

This keeps coverage aligned with operational intent.

---

10. Coverage thresholds

AVCS should support threshold rules rather than assuming all runs need perfect completeness.

Examples of threshold concepts:

- minimum domain completion count
- 100% coverage of critical workflows
- no untested high-risk admin surfaces
- no missing required security coverage
- allowed deferred scope only in non-mandatory areas
- evidence completeness threshold for covered required surfaces

Thresholds should come from policy and build class, not arbitrary intuition.

---

11. Coverage computation rules

Coverage should be computed from declared scope plus observed execution.

A coverage item should require:

- an identifiable scope target

- execution or evaluation evidence
- mapped outcome state
- domain/workflow/surface linkage
- final state assignment

Coverage should not be inflated by:

- duplicate checks on same narrow surface
- shallow route touches treated as full workflow coverage
- rerunning the same passing micro-check and counting it repeatedly
- evidence-free assumptions that a nearby surface must also have worked

This protects quality integrity.

---

12. Coverage gaps

AVCS should explicitly represent gaps rather than hiding them.

A coverage gap exists when:

- a required item was not tested
- a required item was blocked
- a required item only has shallow validation where deeper validation is required
- required evidence is missing for a claimed covered item
- a run type failed to satisfy its declared scope

Coverage gaps should be first-class certification signals.

They can lead to:

- warnings
- conditional passes
- blocked certification
- outright failure depending on severity and policy

---

13. Coverage and verdict interaction

Coverage must directly influence certification decisions.

Typical interactions:

- missing required domain coverage can force BLOCKED or FAIL
- missing critical workflow coverage should usually prevent strong PASS
- blocked high-risk surface coverage may force conditional or blocked outcomes
- low-risk deferred scope may be acceptable under policy
- strong score should not compensate for missing mandatory coverage

This is what makes coverage operationally significant.

---

14. Coverage reporting views

Operators should be able to inspect coverage in multiple ways.

Recommended views:

- by domain
- by workflow
- by page/route/surface

- by risk class
- by requirement
- by passed vs failed vs blocked vs not tested
- by current run vs baseline run

This is where the **Coverage Matrix** becomes one of the key AVCS control surfaces.

---

15. Historical coverage comparison

AVCS should preserve and compare coverage over time.

Historical comparison should help answer:

- did coverage shrink?
- did a critical workflow fall out of validation scope?
- did a domain move from deep to shallow coverage?
- did required evidence completeness decline?
- did new features enter the build without corresponding certification coverage?

This makes coverage a strategic quality metric, not just a per-run snapshot.

---

16. Coverage misuse protections

AVCS should guard against weak coverage practices.

It should prevent:

- shallow checks being labeled as full coverage
- counting failed execution as equivalent to passed required coverage

- hiding not-tested required areas
- aggregating many low-risk checks to mask missing high-risk scope
- using stale historical coverage as if it were current without disclosure
- presenting deferred scope as accidental omission

These protections are necessary for honest certification.

---

17. Canonical coverage statement

Use a stable summary statement for this section:

AVCS measures coverage as the fulfilled certification scope of a run across domains, workflows, surfaces, requirements, risk classes, and evidence layers, using explicit states such as covered, covered-failed, partial, blocked, not tested, and deferred.

---

18. Section close

AVCS-10 should establish that coverage is not just test volume.

Coverage in AVCS means:

- what was supposed to be validated
- what was actually exercised
- how deeply it was exercised
- whether that exercise succeeded
- what was never tested
- what gaps remain relevant to certification

That gives AVCS an honest way to talk about “full coverage” instead of using it as a vague promise.

AVCS-11

AVCS-11 — Scoring Model

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** quantifies overall build readiness through structured scoring.

If **AVCS-10** defines what AVCS covered, **AVCS-11** defines how AVCS can summarize quality strength across domains without reducing certification to a simplistic pass/fail count.

This section matters because scoring is useful, but dangerous if designed poorly. A good scoring model helps operators:

- compare builds
- spot quality trends
- understand relative strength across domains
- prioritize remediation
- track improvement over time

A bad scoring model hides serious problems behind a nice-looking aggregate.

So the AVCS scoring model must be useful, but subordinate to blocker logic, evidence requirements, coverage sufficiency, and policy rules.

---

2. Core scoring statement

AVCS should use scoring as a **structured readiness summary**, not as the sole authority for certification.

A score should help answer:

- how strong is this build overall
- which domains are strongest or weakest
- how does this run compare to prior runs

- where is quality debt accumulating
- how close is the build to expected release quality

A score should **never** override:

- hard blockers
- missing required evidence
- missing required coverage
- failed required domains
- policy violations

In AVCS, scoring supports judgment.
It does not replace judgment.

---

3. Scoring model principle

The AVCS scoring model should follow eight principles:

3.1 Secondary to gating

Scoring must never outrank hard-stop release logic.

3.2 Domain-based

Scores should be computed by assurance domain, not only as one blended number.

3.3 Explainable

Operators should be able to see how the score was produced.

3.4 Coverage-aware

A high score should not be possible when required scope is missing.

3.5 Evidence-aware

Unsupported pass assumptions should not earn full score credit.

3.6 Risk-weighted

High-risk domains and high-risk surfaces should matter more than low-risk polish items.

3.7 Historically comparable

Scores should be useful for trend comparison across runs.

3.8 Non-maskable

Strong performance in one area should not fully erase weakness in another critical area.

These principles keep scoring honest.

---

4. What scoring is for

AVCS scoring should serve five main purposes.

4.1 Readiness summarization

Give operators a compact signal of overall quality strength.

4.2 Domain comparison

Show where the build is strong and where it is weak.

4.3 Trend tracking

Make it possible to compare one build’s quality profile against past runs.

4.4 Remediation prioritization

Help teams focus on the domains causing the biggest readiness drag.

4.5 Portfolio visibility

If AVCS later compares many kits or releases, scoring becomes a useful management surface.

These are the correct uses of scoring.

---

5. What scoring is not for

AVCS should explicitly reject bad uses of scoring.

Scoring is not for:

- ignoring critical blockers because the total number looks good
- labeling incomplete runs as strong
- compensating for failed security or auth boundaries with strong UI polish
- replacing human review where policy requires review
- collapsing all quality nuance into one vanity metric

This should be stated clearly because scoring drift is common in complex systems.

---

6. Scoring structure

AVCS should use a **layered scoring structure**.

Recommended layers:

1. **Domain scores**
2. **Weighted overall readiness score**
3. **Confidence or evidence modifier**
4. **Trend delta**
5. **Blocker override layer outside score**

This gives the system enough nuance without making it unreadable.

---

7. Domain score model

Each core assurance domain should receive its own score.

Recommended scored domains:

- Build Integrity
- Functional Correctness
- UI Quality
- UX Workflow Quality
- Security Posture
- Performance & Stability
- Accessibility
- Deployment Readiness
- Enterprise Readiness

Each domain score should reflect:

- pass/fail state of covered items
- severity and volume of findings
- coverage depth and completeness
- evidence completeness
- regression impact where relevant
- policy expectations for that domain

This makes scores diagnostically useful.

---

8. Recommended weighting model

AVCS should weight domains based on release importance rather than treating all domains equally.

A strong default weighting model could be:

- **Functional Correctness** — 22%
- **Security Posture** — 20%
- **Build Integrity** — 12%
- **Deployment Readiness** — 12%
- **Performance & Stability** — 10%
- **UX Workflow Quality** — 8%
- **UI Quality** — 6%
- **Accessibility** — 6%
- **Enterprise Readiness** — 4%

This is only a starting model, but it reflects a reasonable priority structure:

- the build must work
- it must be safe
- it must be structurally sound
- it must be releasable
- then the rest refines enterprise quality

This weighting should be policy-adjustable by build class if needed.

---

9. Domain score inputs

A domain score should not be based on one signal alone.

Each domain score should consume a combination of inputs such as:

- domain coverage ratio

- depth of coverage
- pass/fail ratio across evaluated items
- severity-adjusted findings
- blocker presence
- evidence completeness
- regression delta from baseline
- policy-specific subrequirements

This prevents shallow score inflation.

---

10. Suggested domain score formula shape

AVCS does not need a single rigid formula yet, but it should define the shape of the computation.

A domain score can be thought of as:

Domain Score = Coverage Quality × Outcome Quality × Evidence Quality × Risk Adjustment

Where:

- **Coverage Quality** reflects whether the right surfaces were evaluated at the right depth
- **Outcome Quality** reflects pass/fail balance and severity footprint
- **Evidence Quality** reflects how well-supported the claims are
- **Risk Adjustment** reduces score for critical or high-risk issues even if other signals look good

This formula shape is better than simple pass percentage.

---

11. Coverage effect on scoring

Coverage must directly affect score.

Examples:

- a domain with shallow or partial coverage should not receive full credit
- a required workflow not tested should depress associated domain score
- blocked coverage in a high-risk area should materially reduce score
- deferred low-risk scope should not destroy score if policy allows deferral

This ensures scores reflect meaningful validation, not mere optimism.

---

12. Evidence effect on scoring

Evidence completeness should also shape score.

Examples:

- a domain with strong observed success but weak evidence should lose score credit
- a claimed pass without direct execution proof should receive reduced confidence credit
- blocker findings with strong evidence should apply full penalty

This prevents unsupported pass states from inflating readiness.

---

13. Severity penalty model

AVCS should penalize scores based on severity and certification impact, not only on count of findings.

A sensible penalty approach:

13.1 Critical findings

- major domain score reduction
- likely overall score suppression
- may trigger blocker override outside score

13.2 High findings

- strong penalty, especially in required domains

13.3 Medium findings

- moderate penalty, especially when clustered in critical workflows

13.4 Low findings

- light penalty, mainly affecting polish and warning posture

13.5 Info findings

- usually no direct penalty, but preserved for visibility

This is better than treating ten cosmetic findings as worse than one auth defect.

---

14. Risk-adjusted scoring

AVCS should weight issues more heavily when they affect:

- critical workflows
- high-risk admin surfaces
- auth and permission boundaries
- deployment-critical paths

- primary operator journeys
- required policy surfaces

That means the same severity may carry different score impact depending on where it occurs.

Example:

- a medium issue in a low-risk settings screen should hurt less than a medium issue in the primary approval workflow

This keeps scoring aligned with real release risk.

---

15. Overall readiness score

AVCS may compute an overall readiness score from the weighted domain scores.

A clean model is:

Overall Readiness Score = Sum of Weighted Domain Scores, subject to coverage and evidence penalties

This overall score should be displayed alongside:

- verdict state
- blocker count
- coverage status
- evidence completeness

Never by itself.

---

16. Score bands

AVCS should use readable readiness bands rather than showing only raw numbers.

Suggested bands:

- **90–100** — Strong Readiness
- **75–89** — Acceptable Readiness
- **60–74** — Fragile Readiness
- **Below 60** — Insufficient Readiness

These bands help operators interpret the number quickly.

But score band must still defer to verdict logic. For example:

- a build can have an 88 and still be **FAIL** if a hard blocker exists
- a build can have a 72 and be **CONDITIONAL_PASS** if policy allows a narrow exception with full visibility

---

17. Blocker override rule

This is the most important rule in the scoring model:

A strong overall score must never override a blocker-class condition.

Examples:

- Security Posture score looks good overall, but one exposed secret exists → score cannot rescue the verdict
- Functional Correctness score is high, but a required end-to-end workflow fails → score cannot rescue the verdict
- Overall readiness is 91, but required evidence is missing for a certification-critical area → score cannot rescue the verdict

This is non-negotiable if the scoring model is to remain trustworthy.

---

18. Confidence modifier

AVCS may include a **confidence modifier** as a secondary layer on the score.

Confidence can reflect:

- evidence completeness
- stability of evaluator outputs
- depth of coverage
- rerun consistency
- baseline alignment

Example presentation:

- Overall Readiness: 86
- Confidence: High

or

- Overall Readiness: 86
- Confidence: Low due to partial evidence and blocked coverage in one required surface

This helps prevent over-reading a decent score from a weak run.

---

19. Trend and delta scoring

Scores should support comparison across time.

AVCS should show:

- current score
- prior comparable run score

- delta by domain
- delta overall
- trend direction

Examples:

- Security Posture: 92 → 84
- Performance & Stability: 71 → 79
- Overall Readiness: 81 → 83

This makes scoring strategically useful.

---

20. Score freshness and comparability

Not all scores should be compared blindly.

For comparison to be valid, AVCS should track:

- run type
- target class
- policy version
- evaluator version if relevant
- baseline reference
- scope differences
- deferred/untested areas

A Full Certification score should not be presented as directly comparable to a Smoke Run score without clear labeling.

---

21. Build-class scoring variation

AVCS may later allow score weighting to vary by build class.

Examples:

- a security-sensitive admin system may weight Security Posture and Access Control more heavily
- a workflow-heavy operator console may weight Functional Correctness and UX Workflow Quality more heavily
- a deployment-critical infrastructure UI may weight Deployment Readiness and Build Integrity more heavily

The core scoring framework should support this without losing consistency.

---

22. Score presentation rules

Operators should see scores in a clear hierarchy.

Recommended presentation:

1. verdict state
2. blocker / warning summary
3. overall readiness score and band
4. domain score breakdown
5. trend delta
6. confidence indicator
7. explanation of score depressors

This keeps the score informative without letting it dominate the truth.

---

23. Score misuse protections

AVCS should explicitly prevent bad scoring behavior.

It should prevent:

- showing score without verdict
- allowing incomplete required coverage to retain full score
- letting shallow checks count as full domain credit
- allowing many low-risk passes to mask one critical failure
- comparing incompatible run types without disclosure
- calculating a strong score from weak evidence
- using score as the only deployment decision surface

These protections are essential.

---

24. Canonical scoring statement

Use a stable summary statement for this section:

AVCS uses weighted domain-based scoring to summarize build readiness across assurance domains, but scoring remains subordinate to blocker logic, coverage sufficiency, evidence completeness, and certification policy.

---

25. Section close

AVCS-11 should leave the scoring model clear:

- score is useful
- score is not supreme

- domains are scored separately
- critical domains weigh more
- missing coverage and weak evidence reduce score strength
- blockers override aggregate strength
- trends matter more than vanity numbers

That gives AVCS a readiness metric without compromising certification integrity.

AVCS-12

AVCS-12 — Operator Experience

1. Purpose of this section

This section defines the operator-facing experience of **Axion Verification & Certification Suite (AVCS)**.

If the earlier sections define AVCS as a verification, evidence, findings, coverage, and certification system, this section defines how operators actually interact with it in practice.

This matters because AVCS cannot be enterprise-grade if its internal logic is strong but its control surface is weak. Operators need a system that makes certification:

- visible
- understandable
- actionable
- reviewable
- fast to navigate
- hard to misuse

The operator experience is where AVCS becomes a real command center rather than just a backend test framework.

---

2. Core operator experience statement

AVCS must provide a **high-clarity certification command environment** that allows operators to launch runs, monitor execution, inspect coverage, review findings, examine evidence, compare history, and make informed release decisions with minimal ambiguity.

The operator experience should not force users to reconstruct meaning from logs or scattered outputs.
It should present certification as a guided, legible workflow.

---

3. Operator experience principle

The AVCS operator experience should follow nine principles:

3.1 High signal

The UI should emphasize the most important certification information first.

3.2 Decision-oriented

The experience should help operators answer release-readiness questions quickly.

3.3 Drill-down capable

Operators should be able to move from summary to domain to finding to evidence without friction.

3.4 Evidence-visible

Important claims should be one or two steps away from proof.

3.5 Status-legible

Run state, blocker state, coverage state, and verdict state should always be easy to understand.

3.6 Triage-friendly

The system should make failures and risk clusters easy to sort and act on.

3.7 Historically aware

Operators should be able to compare current runs to prior runs without leaving the suite.

3.8 Policy-legible

Operators should be able to see why something passed, failed, or became conditional.

3.9 Controlled

The UI should reduce the chance of accidental misuse, hidden blockers, or ambiguous interpretation.

These principles should guide every AVCS screen.

---

4. Primary operator jobs

The operator experience should be designed around the actual jobs an operator needs to perform.

The main jobs are:

- choose a build target
- choose a run type
- start a certification run
- monitor execution progress
- identify blockers quickly
- inspect failing domains
- review evidence
- understand coverage gaps
- compare against previous runs
- determine whether release can proceed
- document or acknowledge conditional outcomes where policy allows

If a screen does not help with one of those jobs, it is probably secondary.

---

5. Core operator surfaces

AVCS should expose a small number of major control surfaces rather than scattering certification information across too many disconnected screens.

The canonical operator surfaces should be:

1. **Certification Command Center**
2. **Run Detail View**
3. **Coverage Matrix**

4. **Findings / Failure Triage Board**
5. **Evidence Explorer**
6. **Domain Workbenches**
7. **History & Regression View**
8. **Release Gate Panel**

These are the backbone of the operator experience.

---

6. Surface definitions

AVCS-12.1 — Certification Command Center

6.1.1 Purpose

This is the primary entry surface for AVCS. It should function as the main launch and status dashboard for certification work.

6.1.2 What it should allow operators to do

Operators should be able to:

- select a build target
- see build identity and metadata
- choose a run type
- view required domains for that run type
- understand estimated scope complexity
- start a run
- view active runs
- view recent verdicts

- see immediate blocker alerts or stale-run warnings

6.1.3 Key information it should show

The Command Center should prominently show:

- target build/version
- latest certification status
- latest run type and timestamp
- blocker count
- warning count
- overall readiness summary
- whether the build is currently releasable, conditional, failed, or blocked
- freshness of major runs

6.1.4 Why it matters

This is where the suite establishes confidence and orientation. Operators should understand the state of a build within seconds.

AVCS-12.2 — Run Detail View

6.2.1 Purpose

Run Detail View is the execution-centric screen for a single certification run.

6.2.2 What it should show

This view should show:

- run ID
- target build

- run type
- run state
- domain execution order and status
- domain-level results
- evidence collection status
- findings summary
- coverage summary
- verdict summary
- timestamps and rerun linkage

6.2.3 Live execution behavior

During active execution, operators should be able to see:

- current stage
- active domain
- completed domains
- blocked domains
- emerging findings where appropriate
- evidence collection activity
- completion by scope, not fake certainty by time

6.2.4 Why it matters

Operators need a stable place to watch a run unfold and inspect its final assembled result.

AVCS-12.3 — Coverage Matrix

6.3.1 Purpose

The Coverage Matrix is the main surface for understanding what was actually validated.

6.3.2 Recommended structure

A strong matrix view should allow operators to inspect coverage by:

- domain × workflow
- domain × surface
- workflow × state
- requirement × state
- risk class × state

6.3.3 Coverage states that should be visible

The matrix should clearly distinguish:

- covered
- covered-failed
- partial
- blocked
- not tested
- deferred by design

6.3.4 Why it matters

This is the screen that makes “full coverage” honest. Operators should immediately see what the run did and did not validate.

AVCS-12.4 — Findings / Failure Triage Board

6.4.1 Purpose

This surface is for issue review and remediation prioritization.

6.4.2 What it should support

Operators should be able to sort and filter findings by:

- severity
- impact class
- domain
- workflow
- blocker status
- regression vs net-new
- open/resolved state
- run ID
- target build

6.4.3 What each finding card or row should show

Each finding entry should show:

- title
- severity
- impact class
- primary domain
- affected surface
- concise summary
- certification effect
- evidence preview

- status
- rerun/change state if applicable

6.4.4 Why it matters

Operators need to move from “something failed” to “what should I care about first?” quickly.

AVCS-12.5 — Evidence Explorer

6.5.1 Purpose

This surface is the proof layer of AVCS.

6.5.2 What it should allow

Operators should be able to:

- inspect screenshots
- inspect traces
- inspect logs
- inspect metric outputs
- inspect security artifacts
- inspect coverage records
- navigate evidence by run, domain, workflow, or finding

6.5.3 Presentation principle

Evidence should be shown in a way that is:

- contextual
- linked
- previewable

- not overwhelming

Operators should not need to hunt through opaque file dumps.

6.5.4 Why it matters

Certification trust depends on proof being easy to inspect.

AVCS-12.6 — Domain Workbenches

6.6.1 Purpose

These are focused analysis views for major assurance domains.

6.6.2 Recommended workbenches

AVCS should likely have dedicated workbenches for:

- Security Posture
- Performance & Stability
- UI/UX
- Accessibility
- Deployment Readiness

6.6.3 What each workbench should do

Each workbench should provide:

- domain summary
- domain score/status
- domain findings
- domain-specific evidence
- domain-specific coverage view

- trend comparison where relevant
- policy or threshold indicators for that domain

6.6.4 Why it matters

Some domains need deeper review surfaces than the main dashboard can reasonably provide.

AVCS-12.7 — History & Regression View

6.7.1 Purpose

This surface allows operators to compare quality over time.

6.7.2 What it should show

Operators should be able to inspect:

- prior runs
- verdict history
- score deltas
- coverage deltas
- finding trend changes
- blocker recurrence
- resolved vs persistent issues
- regression findings by domain

6.7.3 Why it matters

AVCS should support long-term quality management, not only one isolated run at a time.

AVCS-12.8 — Release Gate Panel

6.8.1 Purpose

This is the final decision surface for release-readiness review.

6.8.2 What it should show

The Release Gate Panel should prominently display:

- current certification verdict
- blocker summary
- warning summary
- required-domain completion status
- coverage sufficiency
- evidence completeness
- policy compliance
- overall readiness score and confidence where used
- whether override is required or allowed
- reasoning summary for the verdict

6.8.3 Why it matters

This is where operators should be able to answer the central question:
Can this build move forward right now, and why or why not?

---

7. Information hierarchy

The AVCS UI should present information in a stable hierarchy.

The top hierarchy should generally be:

1. verdict state

2. blocker state
3. run state and freshness
4. required-domain status
5. coverage sufficiency
6. evidence completeness
7. findings summary
8. readiness score and trends
9. deep evidence and raw detail

This hierarchy prevents the interface from over-emphasizing vanity metrics or secondary detail before the operator understands release truth.

---

8. Visual status language

AVCS should use a consistent visual language for major states.

The operator experience should visually distinguish:

- PASS
- PASS_WITH_WARNINGS
- CONDITIONAL_PASS
- FAIL
- BLOCKED

It should also clearly distinguish:

- covered vs covered-failed
- open blocker vs warning

- active run vs finalized run
- current result vs baseline result
- required vs optional scope

Consistency here matters because certification systems depend on rapid recognition.

---

9. Navigation model

The UI should support natural movement between certification layers.

A healthy navigation flow looks like:

Command Center → Run Detail → Domain / Coverage / Findings → Evidence → Release Gate

Operators should also be able to move backward cleanly and pivot across related contexts, such as:

- from a finding to all evidence supporting it
- from a domain to all its blockers
- from a run to prior comparable runs
- from a verdict to the policy condition that shaped it

Navigation should feel investigative, not maze-like.

---

10. Live execution UX

During active runs, AVCS should feel operationally alive without becoming noisy.

10.1 What should update live

Operators should see:

- stage transitions
- domain completion
- newly surfaced blockers
- run interruption or blocked state
- evidence collector status
- emerging coverage completion

10.2 What should not happen

The system should avoid:

- fake precision time promises
- noisy low-value log spam in primary view
- unstable status language
- overwhelming the operator with raw artifacts before summary meaning exists

Live execution should be calm, legible, and precise.

---

11. Operator workflows that must be excellent

Certain workflows are core and should be treated as top-priority UX paths inside AVCS.

These include:

- starting a new run
- identifying why a run failed
- identifying whether release is blocked
- reviewing all blocker findings

- opening evidence for a specific finding
- understanding missing coverage
- comparing current run to last successful run
- confirming whether a conditional pass requires action
- reviewing deployment gate status

If these workflows are clumsy, the suite will feel weaker than it is.

---

12. Filtering and slicing

AVCS should support robust filtering because certification data gets dense quickly.

Operators should be able to filter by:

- build
- run type
- run date
- domain
- severity
- impact class
- blocker state
- workflow
- surface
- open/resolved status
- regression/net-new

- evidence type

Filters should persist reasonably within the operator session so the suite feels efficient for real triage work.

---

13. Comparison experience

Historical comparison should be built into the operator experience, not bolted on.

Useful comparison modes include:

- current run vs previous run
- current run vs last passing run
- current run vs selected baseline
- domain score delta
- coverage delta
- findings added / removed / worsened / resolved
- verdict change over time

This should be easy to access from Run Detail, Findings, and Release Gate surfaces.

---

14. Policy visibility

Operators should be able to see why the system is enforcing a certain outcome.

The UI should make visible:

- required domains for the run type
- policy-triggered blocker conditions

- freshness requirements
- evidence requirements
- override eligibility
- mandatory release gates

This prevents the suite from feeling arbitrary or mysterious.

---

15. Override and exception visibility

If policy allows overrides or conditional progression, the operator experience must make those states explicit.

It should show:

- original verdict
- why the verdict is conditional or failed
- whether override is permitted
- what authority is required
- what conditions must be acknowledged
- that override does not erase the underlying finding set

This must be visible but controlled. AVCS should not encourage casual override behavior.

---

16. Readability and tone of the UI

The AVCS interface should feel:

- precise

- premium
- controlled
- technical without being chaotic
- enterprise-grade
- command-center oriented

It should avoid:

- overly playful presentation
- cluttered developer-tool sprawl
- raw log-first design
- ambiguity in action labels
- soft language around blocker conditions

This should feel like a release-certification cockpit.

---

17. Minimum viable operator experience

The minimum viable AVCS operator experience should include:

- a Command Center
- a Run Detail page
- a Coverage Matrix
- a Findings / Failure Triage view
- evidence drill-down
- a Release Gate Panel

That is enough to make the system genuinely usable early.

---

18. Future operator experience expansion

Later versions of AVCS can expand with:

- richer trend dashboards
- more advanced domain workbenches
- inline remediation tracking
- smarter finding clustering
- certification timeline playback
- multi-build portfolio views
- operator notes and review history
- policy simulation views

These are valuable later, but the primary release workflow should come first.

---

19. Canonical operator experience statement

Use a stable summary statement for this section:

AVCS provides an operator-grade certification command environment with dedicated surfaces for run launch, run monitoring, coverage inspection, findings triage, evidence review, historical comparison, and final release-gate decision support.

---

20. Section close

AVCS-12 should leave the operator experience clear:

- operators need a command center, not a log dump
- the suite must make status, blockers, coverage, and evidence immediately legible
- deep review must be easy
- historical comparison must be built in
- the final release gate must be unmistakable

That gives AVCS the human-facing surface it needs to function as a real certification system.

AVCS-13

AVCS-13 — Audit and History

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** preserves certification history and auditability over time.

If **AVCS-12** defines how operators interact with AVCS in the moment, **AVCS-13** defines how AVCS remembers what happened, what was decided, what evidence existed, and how quality changed across runs.

This section matters because enterprise-grade certification cannot be purely present-tense. AVCS must not only answer:

- what is happening now
  but also:
- what happened before
- what changed
- who initiated or reviewed a run
- why a build passed, failed, or was blocked
- whether issues were resolved or repeated
- whether an override occurred
- whether certification history is trustworthy

Without audit and history, AVCS becomes a disposable test runner. With them, it becomes a durable certification record system.

---

2. Core audit and history statement

AVCS must preserve an **append-only, reviewable certification history** for each build target, run, finding set, evidence pack, and release-relevant decision event so that quality judgments remain traceable, comparable, and auditable over time.

This means AVCS should not only store the latest state.
It should preserve the decision trail.

---

3. Audit and history principles

The AVCS audit and history model should follow nine principles:

3.1 Append-only

Historical certification records should not be silently overwritten.

3.2 Traceable

Every important event should link back to a run, build target, actor, and time.

3.3 Comparable

Historical runs should be easy to compare across verdicts, scores, coverage, and findings.

3.4 Evidence-preserving

Audit history must preserve the proof associated with important outcomes.

3.5 Decision-legible

Later reviewers should be able to understand why a verdict occurred.

3.6 Override-visible

Exceptions, overrides, and conditional acceptances must remain visible in history.

3.7 Resolution-aware

History should show whether problems persisted, improved, or worsened across reruns.

3.8 Policy-aware

Historical records should reflect the policy context that shaped the run.

3.9 Durable

Certification history should remain available long enough to support review, remediation, governance, and trend analysis.

These principles make AVCS usable as a long-term quality memory.

---

4. What must be historically preserved

AVCS should preserve more than raw run rows.

At minimum, history should retain:

- run manifests
- target build identity
- run type
- execution timeline
- domain outcomes
- findings
- evidence references
- coverage state
- certification verdict
- score and confidence where used
- policy context
- rerun relationships
- override or review events
- major status transitions
- historical comparisons or deltas where computed

This creates a complete certification record rather than a partial one.

---

5. Audit object types

AVCS should preserve several classes of historical records.

AVCS-13.1 — Run History Records

5.1.1 Definition

A Run History Record represents one executed certification run.

5.1.2 What it should include

Each run record should include:

- run ID
- build target ID
- run type
- actor who initiated it
- start and end timestamps
- execution status
- included domains
- verdict
- summary counts for blockers/warnings/findings
- coverage summary
- evidence completeness summary
- rerun linkage
- baseline linkage if applicable

5.1.3 Why it matters

This is the anchor record for AVCS history.

AVCS-13.2 — Event History Records

5.2.1 Definition

An Event History Record captures meaningful state changes or actions that occur during or after a certification run.

5.2.2 Examples

This includes:

- run created
- preflight failed
- domain blocked
- verdict finalized
- override applied
- rerun initiated
- finding status changed
- evidence finalization completed

5.2.3 Why it matters

Events provide timeline-level traceability beyond the final snapshot.

AVCS-13.3 — Finding History Records

5.3.1 Definition

A Finding History Record preserves the lifecycle of a finding across runs and reviews.

5.3.2 What it should show

It should show:

- first appearance

- severity changes
- evidence changes
- persistence across reruns
- resolution state
- override acceptance if any
- supersession or false-positive determination

5.3.3 Why it matters

Operators need to know whether an issue is new, recurring, or resolved.

AVCS-13.4 — Evidence History Records

5.4.1 Definition

An Evidence History Record preserves the existence and linkage of certification proof artifacts.

5.4.2 What it should show

It should show:

- evidence ID
- originating run
- evidence type
- linked domain or finding
- timestamp
- finalization status
- storage reference
- whether it was reused or superseded in later runs where policy allows

5.4.3 Why it matters

Evidence history is essential for audit defensibility.

AVCS-13.5 — Decision History Records

5.5.1 Definition

A Decision History Record captures certification outcomes and any later decision events related to them.

5.5.2 Examples

This includes:

- original verdict
- reasoning summary
- policy basis
- override-required state
- override-applied event
- accepted warning state
- pre-deployment gate decision result

5.5.3 Why it matters

Release-relevant decisions must remain inspectable later.

---

6. History timeline model

AVCS should expose a coherent timeline for each target build and each run.

A healthy timeline should allow operators to see:

1. build target created or selected

2. certification run initiated
3. preflight result
4. domain execution milestones
5. major findings surfaced
6. verdict finalized
7. rerun initiated if needed
8. later comparison or review events
9. override or acceptance events if applicable
10. final release gate state

This timeline should be readable in both compact and detailed forms.

---

7. Append-only history rules

AVCS history should be append-only by default.

That means:

- finalized runs should not be overwritten
- verdict changes should create new decision events, not erase old ones
- reruns should create new run records
- finding resolution should update lifecycle state through historical events
- annotations should be additive
- evidence should not be silently replaced after finalization

This is one of the most important integrity rules in the entire system.

---

8. Audit fields for important records

AVCS should preserve standard audit metadata on major record types.

Recommended audit fields:

- record ID
- record type
- related run ID
- related target build ID
- actor
- timestamp
- source/action type
- prior state if relevant
- new state if relevant
- policy context/version
- reason/note field where relevant
- related evidence or finding linkage

These fields make the system searchable and reviewable.

---

9. Actor and action traceability

AVCS should preserve who did what.

For important events, history should show:

- who initiated the run

- who reviewed the result
- who triggered a rerun
- who applied an override if allowed
- who marked a finding resolved, false positive, or accepted
- what the action was
- when it happened
- what object it affected

This is critical for governance and accountability.

---

10. Historical comparison requirements

History is not useful if it is only archival. AVCS should actively support comparison.

Operators should be able to compare:

- verdict changes over time
- score changes by domain
- blocker recurrence
- finding count and severity shifts
- coverage expansion or shrinkage
- evidence completeness changes
- regression vs resolution trends
- stability of required workflows across runs

This should work for:

- current run vs previous run
- current run vs last passing run
- current run vs selected baseline
- build lineage over time

---

11. Rerun lineage

AVCS should explicitly preserve lineage between runs.

A rerun should record:

- which run it follows
- why it was rerun
- whether the rerun scope is full or partial
- which findings or domains it intended to re-evaluate
- what changed in the result

Lineage should make it easy to answer:

- did the remediation fix the blocker
- did new issues appear
- did confidence improve
- is this issue persistent across many attempts

This prevents reruns from becoming historically opaque.

---

12. Historical states that matter

AVCS history should preserve important certification states over time, not just the latest status.

Examples:

- latest verdict
- previous verdicts
- unresolved blockers
- resolved blockers
- conditional-pass history
- blocked-run history
- stale-run history
- accepted-by-override history
- persistent warning history

This supports smarter operator judgment.

---

13. Policy context in history

Historical records should include the policy context under which a run was evaluated.

This matters because the meaning of a verdict can change if:

- required domains changed
- coverage thresholds changed
- blocker rules changed
- freshness rules changed
- score weighting changed

So AVCS should preserve:

- policy version or snapshot reference
- run-type requirements at time of run
- gating rules that materially affected verdict

This protects historical comparability.

---

14. Auditability of overrides and exceptions

Overrides are one of the most sensitive parts of the system and must be fully auditable.

Whenever an override or exception occurs, AVCS should preserve:

- original verdict
- override action
- actor who applied it
- authority or role used
- timestamp
- reason/justification
- affected blockers or conditions
- resulting effective release state

Overrides should never erase the original certification truth.

---

15. History presentation surfaces

AVCS should make history visible in several practical ways.

Recommended views:

- build-level certification timeline
- run history table
- verdict history trend view
- findings persistence view
- rerun lineage graph
- blocker recurrence panel
- override history view
- score and coverage delta view

These views should be accessible from both the Command Center and Run Detail contexts.

---

16. Retention and preservation guidance

AVCS should preserve finalized certification history long enough to remain operationally and governance-useful.

At minimum, retention policy should distinguish:

- active recent runs
- long-term audit records
- evidence retention windows
- summarized historical records
- archived but queryable records

The exact retention length can be policy-defined later, but the design should assume that certification history is durable infrastructure, not disposable telemetry.

---

17. History misuse protections

AVCS should explicitly guard against bad audit and history behavior.

It should prevent:

- overwriting finalized verdicts
- deleting evidence references from historical runs without trace
- losing rerun lineage
- hiding override events
- showing only latest state with no history trail
- comparing runs without disclosing policy or scope differences
- allowing finding resolution to erase original occurrence

These protections are essential for trust.

---

18. Canonical audit and history statement

Use a stable summary statement for this section:

AVCS preserves append-only certification history across runs, findings, evidence, verdicts, reruns, and override events so that build quality, release decisions, and remediation progress remain traceable and auditable over time.

---

19. Section close

AVCS-13 should make the historical role of the suite clear:

- AVCS does not just evaluate builds
- it preserves the record of that evaluation

- it shows how quality changes over time
- it makes overrides and reruns visible
- it protects certification integrity through append-only history

That gives AVCS institutional memory, not just present-tense output.

AVCS-14

AVCS-14 — Policy and Gating

1. Purpose of this section

This section defines the policy framework and gating logic that control how **Axion Verification & Certification Suite (AVCS)** turns verification into enforceable release discipline.

If **AVCS-13** defines historical traceability, **AVCS-14** defines the rules that make AVCS authoritative in the present.

This section matters because AVCS should not be a passive reporting dashboard. It should be a governed certification system with explicit standards for:

- what must be tested
- what must be proven
- what blocks release
- what can proceed with warnings
- what requires override
- who is allowed to accept exceptions

Without policy and gating, certification becomes inconsistent and easy to weaken.

---

2. Core policy and gating statement

AVCS must apply **explicit certification policy and release gates** that determine when a build may pass, pass with warnings, require conditional acceptance, fail, or remain blocked.

Policy defines the rules.
Gating enforces those rules against each run.

Together, they ensure that certification is:

- consistent
- explainable
- enforceable
- auditable
- resistant to convenience-driven drift

---

3. Policy and gating principles

The AVCS policy and gating model should follow ten principles:

3.1 Explicit

Required certification conditions must be declared, not implied.

3.2 Role-aware

Different build classes and run types may require different gates.

3.3 Risk-sensitive

Higher-risk surfaces and release paths must face stricter gates.

3.4 Evidence-dependent

Claims without required proof must not clear gates.

3.5 Coverage-dependent

Untested required scope must influence gate outcome.

3.6 Blocker-respecting

Hard blockers must override otherwise strong signals.

3.7 Override-controlled

Exceptions must be formal, visible, and constrained.

3.8 Consistent

Equivalent cases should be treated equivalently under the same policy context.

3.9 Auditable

Every gate outcome and policy-driven decision must be historically traceable.

3.10 Evolvable

Policy must be structured so it can mature without destabilizing the entire suite.

These principles are what make AVCS a release-control system instead of just an evaluation engine.

---

4. What policy governs in AVCS

AVCS policy should govern at least the following areas:

- required run types by build class
- required domains by run type
- minimum coverage thresholds
- minimum evidence requirements
- severity-to-impact mapping rules
- blocker conditions
- pass / conditional / warning thresholds
- rerun and freshness rules
- override eligibility and authority
- final pre-deployment gate requirements

That means policy is not a single switch. It is the rule layer across the certification lifecycle.

---

5. Policy layers

AVCS should organize policy into layered scopes rather than one flat ruleset.

AVCS-14.1 — Global Certification Policy

5.1.1 Definition

Global Certification Policy defines the baseline rules that apply across all AVCS certification activity unless a narrower policy overrides them legitimately.

5.1.2 Typical contents

This may define:

- canonical verdict states
- default blocker classes
- minimum evidence rules
- standard severity taxonomy
- core run-type definitions
- general override requirements
- append-only audit expectations

5.1.3 Why it matters

This gives AVCS a stable common backbone.

AVCS-14.2 — Build-Class Policy

5.2.1 Definition

Build-Class Policy defines stricter or more tailored rules for different categories of build outputs.

5.2.2 Example build classes

Examples could include:

- internal operator console
- customer-facing web app
- admin/control surface
- security-sensitive workflow system
- deployment-critical infrastructure surface

5.2.3 Typical effects

Build-Class Policy may change:

- domain weighting
- mandatory runs
- blocker strictness
- required workflow coverage
- required security checks
- allowed override behavior

5.2.4 Why it matters

Not all builds have the same risk profile.

AVCS-14.3 — Run-Type Policy

5.3.1 Definition

Run-Type Policy defines what each run type must include and what authority that run type carries.

5.3.2 Typical contents

This may define:

- included domains

- required depth
- evidence expectations
- whether the run may contribute to final release gating
- freshness window for reuse
- whether deferred scope is allowed

5.3.3 Why it matters

This prevents run names from becoming vague labels.

AVCS-14.4 — Release Gate Policy

5.4.1 Definition

Release Gate Policy defines the conditions required for a build to clear the final progression gate.

5.4.2 Typical contents

This may define:

- mandatory pre-deployment run types
- freshness requirements
- no-open-blocker conditions
- minimum critical-workflow coverage
- required evidence completeness
- approval or override conditions

5.4.3 Why it matters

This is the rule layer directly tied to progression decisions.

---

6. Gating object model

AVCS should treat gates as first-class control objects, not hidden logic branches.

Each gate should have:

- gate ID
- gate name
- policy source
- applicable build class

- applicable run type or stage
- gate condition
- evaluation status
- result state
- failure reason if not cleared
- override eligibility flag
- timestamp
- related evidence and findings

This makes gate outcomes inspectable and machine-usable.

---

7. Canonical gate types

AVCS should support a clear set of major gate types.

AVCS-14.5 — Run Eligibility Gate

Determines whether the selected target and requested run can start meaningfully.

Examples:

- target build exists
- required metadata present
- selected run allowed for target class
- required prerequisites available

Failure here usually leads to **BLOCKED** before deeper execution.

AVCS-14.6 — Domain Completion Gate

Determines whether required domains for the run type actually completed.

Examples:

- Security Run must complete security domain checks
- Full Certification Run must complete all required domains
- Pre-Deployment Gate Run must not be missing required deployment checks

Failure here often leads to **BLOCKED** or **FAIL**, depending on cause.

AVCS-14.7 — Evidence Completeness Gate

Determines whether the run collected enough proof to support certification authority.

Examples:

- major findings have linked evidence
- required domains have result evidence
- verdict support summary exists
- no required proof class is missing

Failure here should usually prevent strong PASS states.

AVCS-14.8 — Coverage Sufficiency Gate

Determines whether required scope was exercised deeply enough to trust the run.

Examples:

- all critical workflows tested
- no untested required auth surface
- minimum domain coverage threshold met
- no blocked high-risk surface left unresolved

Failure here may lead to **CONDITIONAL_PASS**, **FAIL**, or **BLOCKED** depending on severity and policy.

AVCS-14.9 — Blocker Gate

Determines whether blocker-class conditions are present.

Examples:

- auth bypass detected
- app boot failure in required profile
- required workflow broken
- exposed secret found
- required deployment-readiness blocker present

Failure here should usually prevent PASS and PASS_WITH_WARNINGS.

AVCS-14.10 — Policy Compliance Gate

Determines whether the run and target satisfy the applicable certification policy rules.

Examples:

- required run freshness satisfied
- required run type executed
- forbidden stale evidence not used
- override path not misused
- minimum severity rules applied correctly

Failure here should materially affect certification authority.

AVCS-14.11 — Final Release Gate

Determines whether the build may progress under the current policy context.

This is the highest-level gate and should consider:

- required-domain completion
- blocker state
- coverage sufficiency
- evidence completeness
- freshness
- policy compliance
- override state if applicable

This gate should map most directly to real release progression readiness.

---

8. Mandatory policy questions

For every meaningful AVCS policy set, the system should be able to answer these questions clearly:

- What run types are required for this build class?
- What domains are mandatory?
- What counts as a release blocker?
- What coverage is required?
- What evidence is required?
- How fresh must supporting runs be?

- What findings can be tolerated as warnings?
- When is conditional pass allowed?
- Who can approve an exception?
- What must happen before final progression?

If policy cannot answer those questions, it is too vague.

---

9. Required-run policy

AVCS should define which run types are mandatory for which build classes or release stages.

A strong default policy might look like this:

9.1 Minimum verification path

For most meaningful builds:

- Smoke Run required after build generation
- Functional Run required before certification
- Security Run required before release decision

9.2 Full certification path

For enterprise-significant builds:

- Smoke Run
- Functional Run
- Security Run
- Performance Run
- Accessibility Run
- Full Certification Run

9.3 Final progression path

Before deployment progression:

- recent Full Certification Run or equivalent validated coverage
- Pre-Deployment Gate Run
- no unresolved release blockers
- evidence completeness confirmed

These can be refined later by build class.

---

10. Required-domain policy

Policy should define domain requirements explicitly.

Examples:

- Smoke Run requires Build Integrity and minimal startup/workflow checks
- Security-sensitive build class requires Security Posture as mandatory in all release-grade runs
- Customer-facing UI build class requires Accessibility and UI/UX validation before final gate
- Pre-Deployment Gate Run requires Deployment Readiness and policy freshness validation

This prevents partial runs from being misread as full readiness.

---

11. Coverage threshold policy

AVCS should define minimum coverage expectations by build class and run type.

Examples:

- 100% of critical workflows must be covered for final release gating
- no untested high-risk admin surfaces
- at least standard-depth validation for required auth paths
- all required domains must be covered at defined minimum depth
- deferred scope allowed only in explicitly non-critical areas

Coverage policy is what gives “full coverage” operational meaning.

---

12. Evidence policy

AVCS should define what proof is required before strong certification claims are allowed.

Examples:

- every blocker finding must have direct linked evidence
- every required domain must have domain result evidence

- final verdict must have coverage and evidence completeness summary
- critical pass claims must have direct execution proof
- stale evidence reuse must be disclosed and policy-approved

Evidence policy protects against unsupported green states.

---

13. Blocker policy

AVCS must define what kinds of issues are blocker-class by default.

Common blocker candidates:

- app fails to boot or initialize in required profile
- critical workflow fails
- auth or permission boundary failure
- exposed secret or severe security finding
- missing required environment contract for deployment
- incomplete required evidence for certification-critical result
- required high-risk scope untested
- required run missing or stale beyond allowed threshold

Policy may refine these by build class, but the blocker model must stay explicit.

---

14. Warning and tolerance policy

Not every issue should block release. AVCS policy should define tolerated issue bands.

Examples of issues that may be tolerated in some contexts:

- low-severity cosmetic UI issues in secondary surfaces
- medium-severity non-primary UX friction with explicit visibility
- low-impact supportability gaps
- isolated non-critical performance warnings within tolerance margin

But tolerance policy should never be vague. It should clearly define:

- what can be tolerated
- under what conditions
- whether it produces PASS_WITH_WARNINGS or CONDITIONAL_PASS
- whether acknowledgement is required

---

15. Conditional-pass policy

Policy should tightly control when **CONDITIONAL_PASS** is allowed.

Appropriate uses may include:

- bounded issue in non-primary surface
- acceptable short-term exception with known remediation path
- non-blocking but meaningful issue requiring explicit review
- coverage or evidence gap small enough to allow controlled exception under policy

It should not be allowed for:

- severe auth/security failure
- broken critical workflow
- major deployment-readiness blocker
- major missing evidence for required results
- broad required-domain incompleteness

Conditional pass must remain a governed exception path.

---

16. Freshness policy

AVCS should define how recent supporting runs must be to count toward a final release decision.

Examples:

- Security Run must be no older than a defined freshness window
- Performance evidence may be reused only if no relevant runtime-affecting change occurred
- Accessibility evidence may require rerun after significant UI change
- Pre-Deployment Gate Run must be current enough to represent actual release state

Freshness rules are critical because stale confidence is still bad confidence.

---

17. Rerun policy

AVCS should define when reruns are required or allowed.

Examples:

- rerun required after remediation of blocker-class issue
- partial rerun allowed for isolated domain issue if policy permits
- full rerun required if foundational domains changed
- rerun required when prior run became stale before release decision

This helps keep certification disciplined after changes.

---

18. Override policy

Overrides should be formal policy objects, not informal human habits.

Policy should define:

- which verdict states can be overridden
- which issue classes can never be overridden
- who has override authority
- whether one or multiple approvers are required
- what justification must be recorded
- whether override changes effective release state only, while original verdict remains preserved

Overrides should always be treated as exceptions, not alternate normal flow.

---

19. Gate evaluation order

AVCS should evaluate gates in a stable sequence so results remain predictable.

Recommended order:

1. run eligibility
2. required prerequisites
3. domain completion
4. evidence completeness
5. coverage sufficiency
6. blocker presence
7. policy compliance
8. freshness and reuse validity

9. final release gate

This sequence helps AVCS fail early for invalidity and reserve final gate judgment for runs that are actually certifiable.

---

20. Gate result states

Each gate should have clear states such as:

- passed
- failed
- blocked
- not applicable
- conditionally satisfied where policy permits

This is useful because a gate might fail due to:

- a real product issue
- missing evidence
- incomplete execution
- policy mismatch

Those should not all look the same.

---

21. Policy versioning

AVCS policy must be versioned or snapshotted in a way that preserves historical meaning.

That means AVCS should record:

- policy version
- effective rule set at time of run
- build-class mapping used
- gate thresholds applied
- override rules in effect

Without this, historical verdict comparison becomes unreliable.

---

22. Policy visibility to operators

Operators should be able to see the policy basis for gate outcomes.

The UI should expose:

- mandatory runs for the target build class
- required domains
- unmet gate conditions
- blocker rules triggered
- freshness failures
- override eligibility or prohibition
- final gate reasoning summary

This keeps the suite from feeling arbitrary.

---

23. Policy misuse protections

AVCS should explicitly guard against weak policy behavior.

It should prevent:

- silent weakening of blocker criteria
- PASS despite missing mandatory runs
- PASS despite missing required evidence
- shallow validation counting as mandatory coverage
- stale runs being reused without disclosure
- overrides erasing original verdict truth
- inconsistent treatment of similar build classes under same policy

These protections are necessary for certification integrity.

---

24. Canonical policy and gating statement

Use a stable summary statement for this section:

AVCS enforces release-readiness through explicit certification policy and gate evaluation across required runs, domains, coverage thresholds, evidence requirements, blocker rules, freshness constraints, and controlled override paths.

25. Section close

AVCS-14 should make one thing unmistakable:

- policy defines the certification standard
- gates enforce the standard
- blockers stop weak builds
- missing proof weakens authority
- freshness matters
- overrides are exceptions, not shortcuts

That is what gives AVCS its enforcement layer.

AVCS-15

AVCS-15 — Integration Boundaries

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** connects to the rest of Axion and to adjacent technical surfaces without losing its own responsibility boundary.

If **AVCS-14** defines policy and gating, **AVCS-15** defines the interfaces through which AVCS receives targets, consumes signals, emits certification outputs, and hands results to downstream decision points.

This section matters because AVCS should be deeply connected, but not loosely entangled. A certification system becomes unreliable when:

- it owns too much
- it depends on hidden assumptions
- its inputs are ambiguous
- its outputs are not cleanly consumable
- its boundaries with upstream and downstream systems are blurry

Integration boundaries are what keep AVCS operationally powerful without making it structurally messy.

---

2. Core integration-boundary statement

AVCS must integrate with Axion through **explicit input, output, and control boundaries** that allow it to consume build outputs, execute verification, preserve evidence and history, and emit certification results without taking over build generation, deployment execution, or unrelated platform responsibilities.

That means AVCS should be:

- well-connected
- clearly bounded

- interface-driven
- responsibility-stable

---

3. Integration-boundary principles

The AVCS integration model should follow nine principles:

3.1 Explicit inputs

AVCS should only certify clearly identified build targets and declared run contexts.

3.2 Explicit outputs

AVCS should emit structured certification outputs that other systems and operators can consume reliably.

3.3 Responsibility separation

AVCS should integrate with adjacent systems without absorbing their primary roles.

3.4 Minimal hidden coupling

Critical AVCS behavior should not depend on untracked side effects or invisible assumptions.

3.5 Policy-aware interchange

Inputs and outputs that affect certification authority should carry policy context where needed.

3.6 Append-only history compatibility

Integration behavior should preserve auditability and run history integrity.

3.7 Reusability

AVCS outputs should be reusable by release, maintenance, and historical analysis systems.

3.8 Controlled downstream effect

AVCS may inform downstream action strongly, but should not silently execute unrelated control actions.

3.9 Evolvable contracts

Integration points should be stable enough to version and expand without breaking the certification model.

These principles help AVCS stay clean as the wider system grows.

---

4. Integration position in Axion

AVCS sits in the Axion flow between build production and downstream release progression.

A clean integration view is:

Upstream systems produce build artifacts and metadata → AVCS verifies and certifies → downstream systems consume certification outputs for review, gating, maintenance, and release progression

This framing makes AVCS a transformation boundary:

- input side: build target + context
- output side: certification result + evidence + history

That is the correct integration posture.

---

5. Upstream integration boundaries

AVCS has several upstream dependencies, but each should be narrow and explicit.

AVCS-15.1 — Build Mode Integration

5.1.1 Purpose

This is the most important upstream boundary. Build Mode produces the artifacts AVCS certifies.

5.1.2 What AVCS should receive

From Build Mode, AVCS should receive or be able to resolve:

- build target ID

- build version
- artifact location or manifest
- build metadata
- build class or profile
- environment assumptions
- generation timestamp
- relevant packaging or output descriptors

5.1.3 What AVCS should not inherit

AVCS should not inherit:

- responsibility for generating the build
- responsibility for “trusting” generation success as certification success
- hidden assumptions that a completed build is deployable

5.1.4 Boundary summary

Build Mode emits certifiable outputs.
AVCS consumes them as targets for verification.

AVCS-15.2 — Build Metadata / Manifest Integration

5.2.1 Purpose

AVCS needs structured build identity and shape information to certify the right thing.

5.2.2 Examples of consumed metadata

This may include:

- manifest files

- package metadata
- route/service descriptors
- environment contracts
- build profile declarations
- version or lineage references

5.2.3 Boundary rule

Metadata should inform certification scope and target identity, but should not substitute for actual verification.

AVCS-15.3 — Policy Source Integration

5.3.1 Purpose

AVCS needs access to the effective certification policy context under which it is operating.

5.3.2 What AVCS should receive

This may include:

- global certification policy version
- build-class policy mapping
- run-type requirements
- blocker rules
- freshness rules
- override constraints

5.3.3 Boundary rule

Policy may be sourced externally or internally, but AVCS must resolve a stable policy snapshot for each run.

AVCS-15.4 — Baseline / Historical Reference Integration

5.4.1 Purpose

Regression-aware runs require access to prior certified or comparable runs.

5.4.2 What AVCS should consume

This may include:

- prior run summary
- prior findings
- prior score profile
- prior coverage profile
- prior evidence references
- selected baseline target

5.4.3 Boundary rule

Historical reference data supports comparison, but AVCS must preserve distinction between current evaluation and reused historical context.

---

6. Internal integration boundaries

Inside AVCS itself, subsystems should also have clear interfaces.

AVCS-15.5 — Run Orchestrator ↔ Domain Evaluators

6.1.1 Purpose

The Run Orchestrator should coordinate execution, while domain evaluators perform domain-specific verification.

6.1.2 Boundary rule

The orchestrator owns:

- run state
- sequencing
- scope
- lifecycle control

Domain evaluators own:

- domain-specific checks
- domain evidence creation
- domain observations
- domain result output

This separation prevents the orchestrator from becoming domain-chaotic.

AVCS-15.6 — Domain Evaluators ↔ Evidence Layer

6.2.1 Purpose

Evaluators must emit evidence without owning long-term evidence policy or history logic.

6.2.2 Boundary rule

Evaluators produce evidence artifacts and metadata.
The AVCS evidence layer owns evidence registration, storage linkage, finalization, and retrieval.

AVCS-15.7 — Findings Engine ↔ Verdict Engine

6.3.1 Purpose

The Findings Engine normalizes issues; the Verdict Engine decides certification outcome.

6.3.2 Boundary rule

The Findings Engine should not directly decide final certification state.
The Verdict Engine consumes findings, coverage, evidence status, and policy inputs to compute the run verdict.

This separation keeps decision logic explicit.

AVCS-15.8 — History Layer ↔ Active Run Layer

6.4.1 Purpose

Active execution and historical preservation should be connected but not conflated.

6.4.2 Boundary rule

The active run layer manages live state.
The history layer preserves finalized, append-only records and event lineage.

This prevents accidental mutation of historical truth.

---

7. Downstream integration boundaries

AVCS produces outputs that other systems and actors may consume.

AVCS-15.9 — Operator UI Integration

7.1.1 Purpose

The operator-facing experience is the primary human consumer of AVCS outputs.

7.1.2 What AVCS should expose

AVCS should expose to operator surfaces:

- run summaries
- domain outcomes
- findings

- evidence references
- coverage matrices
- verdict objects
- historical comparisons
- gate states
- policy reasoning summaries

7.1.3 Boundary rule

UI surfaces consume AVCS state and objects.
They should not invent certification truth outside the backend model.

AVCS-15.10 — Release / Deployment Workflow Integration

7.2.1 Purpose

Release progression systems may consume AVCS certification outputs before deployment proceeds.

7.2.2 What AVCS should emit

This may include:

- certification verdict
- gate results
- blocker state
- override-required state
- freshness status
- release recommendation summary
- evidence-pack reference

7.2.3 Boundary rule

AVCS informs release progression.
It does not silently deploy or mutate live systems itself.

This is one of the most important downstream boundaries.

AVCS-15.11 — Maintenance / Quality Systems Integration

7.3.1 Purpose

Other Axion systems may consume AVCS historical outputs to improve platform quality over time.

7.3.2 What AVCS should emit

This may include:

- recurring findings
- failure trend data
- regression patterns
- certification drift signals
- quality score histories
- coverage shrinkage indicators

7.3.3 Boundary rule

AVCS provides authoritative certification data.
Broader maintenance systems may analyze or act on that data, but they do not become AVCS.

AVCS-15.12 — Audit / Governance Integration

7.4.1 Purpose

Governance and review workflows may need access to certification history and exception records.

7.4.2 What AVCS should expose

This may include:

- append-only run history
- override history
- evidence references
- policy version context
- decision reasoning records

7.4.3 Boundary rule

AVCS should expose traceable records, not mutable narratives.

---

8. Input contract categories

AVCS should treat its upstream inputs as formal contract classes.

Recommended input contract categories:

8.1 Target Identity Contract

Defines exactly what build is under certification.

8.2 Build Artifact Contract

Defines where the certifiable output is and what shape it has.

8.3 Policy Context Contract

Defines which rules govern the run.

8.4 Run Request Contract

Defines the selected run type, scope, and initiating actor.

8.5 Baseline Contract

Defines what prior run or reference AVCS should use for comparison.

These contracts should be versionable and validated.

---

9. Output contract categories

AVCS should also expose formal output contracts.

Recommended output contract categories:

9.1 Run Summary Contract

Compact run state and major result summary.

9.2 Domain Result Contract

Per-domain outcome, evidence state, and findings summary.

9.3 Findings Contract

Structured issue objects for triage and downstream interpretation.

9.4 Coverage Contract

Coverage states across domains, workflows, surfaces, and requirements.

9.5 Verdict Contract

Controlled certification outcome with reasoning and gate state.

9.6 Evidence Inventory Contract

Structured references to supporting artifacts.

9.7 History / Audit Contract

Append-only record objects for long-term traceability.

These outputs make AVCS interoperable without sacrificing rigor.

---

10. Integration boundary rules

AVCS should follow a small number of hard boundary rules.

10.1 AVCS consumes build outputs, not build intentions

It certifies what exists, not what was meant to exist.

10.2 AVCS emits certification truth, not deployment commands

It informs the next step without becoming the executor.

10.3 AVCS does not allow hidden input mutation after run start

Certification context must stay stable once a run begins.

10.4 AVCS does not let UI layers redefine backend verdicts

Presentation must reflect the underlying certification model.

10.5 AVCS does not let downstream systems erase upstream certification history

Consumers may act on results, but historical certification truth remains append-only.

These rules are essential for structural integrity.

---

11. Integration failure handling

AVCS should explicitly define what happens when integration boundaries fail.

11.1 Missing build target data

Run should become blocked before meaningful execution.

11.2 Missing policy context

Run should not proceed as if default assumptions are enough unless explicit fallback policy exists.

11.3 Broken evidence storage linkage

Run may execute, but certification authority may degrade or block depending on required evidence class.

11.4 Missing baseline for regression run

Regression run should become blocked or downgraded according to policy.

11.5 Downstream consumer failure

AVCS should preserve its completed outputs even if a downstream system fails to ingest them.

This keeps certification resilient.

---

12. Versioning and compatibility

Integration contracts should be version-aware.

AVCS should preserve or expose:

- contract version
- policy version
- evaluator version where relevant
- output schema version
- compatibility notes if a consumer expects older shapes

This matters for long-term evolution and safe downstream use.

---

13. Security and trust at the integration boundary

Because AVCS deals with release authority, its integration boundaries should preserve trust.

That means:

- target identity should be trustworthy

- policy source should be controlled
- evidence references should be stable
- override inputs should be authenticated and traceable
- downstream gate consumers should not be able to silently reinterpret blocker states without visibility

This is not full security design, but it is an important boundary principle.

---

14. Minimum integration set for MVP

The minimum viable AVCS integration set should include:

- Build Mode target/build metadata input
- run request input
- internal policy resolution
- evidence storage linkage
- operator UI output surfaces
- verdict output object
- append-only history persistence
- release-gate output summary

That is enough to make the suite real without overbuilding.

---

15. Future integration expansion

Later AVCS versions may add deeper integration with:

- richer maintenance systems
- more advanced deployment gates
- portfolio-level quality dashboards
- automated remediation suggestion systems
- external compliance or audit export surfaces
- multi-environment certification comparisons

These can come later as long as the boundary model stays stable.

---

16. Canonical integration-boundary statement

Use a stable summary statement for this section:

AVCS integrates with Axion through explicit contracts for build targets, run requests, policy context, evidence, findings, coverage, verdicts, and history, while remaining cleanly separated from build generation, deployment execution, and unrelated platform control responsibilities.

---

17. Section close

AVCS-15 should leave the integration story clear:

- upstream systems provide certifiable targets and context
- AVCS performs verification and certification
- downstream systems consume structured certification outputs
- AVCS remains the authority on certification truth
- adjacent systems remain distinct in responsibility

That gives AVCS clean system edges instead of accidental sprawl.

AVCS-16

AVCS-16 — Minimum Viable AVCS

1. Purpose of this section

This section defines the first **real, buildable version** of **Axion Verification & Certification Suite (AVCS)**.

If the earlier sections define the full target system, **AVCS-16** defines the smallest version that is still legitimate, useful, and aligned with the purpose of AVCS.

This matters because AVCS can become very large very quickly. It touches:

- build validation
- security
- UX/UI
- performance
- accessibility
- deployment readiness
- evidence
- findings
- coverage
- verdict logic
- history
- policy
- operator UX

Trying to build all of that at once would create drag, confusion, and likely a weaker first implementation.
The MVP should therefore focus on the minimum set of capabilities required for AVCS to function as an actual certification system rather than just a placeholder dashboard.

---

2. Core MVP statement

Minimum Viable AVCS is the smallest version of the suite that can:

- target a produced build
- execute a meaningful set of certification runs
- collect evidence
- generate structured findings
- measure basic coverage
- compute a release-readiness verdict
- preserve run history
- present operators with a usable release gate view

If any of those are missing, the system may still be useful as tooling, but it is not yet a complete AVCS foundation.

---

3. MVP design principle

The MVP should follow six rules:

3.1 Real over broad

It is better to have fewer domains implemented well than many domains implemented shallowly.

3.2 Certification-first

The MVP must already support evidence-backed verdicts, not just test execution.

3.3 Operator-usable

The MVP must already include a usable control surface, not only backend mechanics.

3.4 Release-relevant

The MVP should focus first on the domains most likely to block deployment or reveal major build weakness.

3.5 Extendable

The MVP should establish contracts and structures that later domains can plug into cleanly.

3.6 Historically preservable

The MVP must already preserve runs and results so later expansion does not require rebuilding the historical model.

These rules keep the first version disciplined.

---

4. What the MVP must prove

The first release of AVCS should prove five things:

1. **AVCS can certify a real build target**
2. **AVCS can catch major blockers before release**
3. **AVCS can show why a build passed or failed**
4. **AVCS can preserve certification history**
5. **AVCS can support operator release judgment through a command surface**

That is the right bar for the first version.

---

5. MVP capability set

AVCS-16.1 — Build Target Selection

The MVP must allow operators to select a specific build kit or build output as the certification target.

Minimum required capability:

- target build ID
- target version/build reference
- visible build metadata
- clear target identity in the run manifest

Why it is required:
Without stable target selection, nothing else in AVCS is trustworthy.

AVCS-16.2 — Core Run Types

The MVP should support a focused subset of run types rather than all nine immediately.

Recommended MVP run types:

- **Smoke Run**
- **Functional Run**
- **Security Run**
- **Performance Run**
- **Full Certification Run**

These give the system:

- fast confidence
- core behavior validation
- security posture checks
- performance signals
- one real release-grade assembled run

Why this subset:
It covers the highest-value release risks early without forcing the entire suite to be complete on day one.

AVCS-16.3 — MVP Domain Set

The MVP should implement the most release-critical domains first.

Recommended MVP domains:

- **Build Integrity**
- **Functional Correctness**
- **Security Posture**
- **Performance & Stability**
- **Deployment Readiness**

These five domains are enough to support meaningful enterprise-readiness judgment in a first version.

Why these first:
They answer the most important early certification questions:

- does it boot
- does it work
- is it safe enough
- is it stable enough
- can it realistically deploy

Domains that can come in the next wave:

- UI Quality
- UX Workflow Quality

- Accessibility
- Enterprise Readiness

Those matter a lot, but they can follow once the certification backbone is solid.

AVCS-16.4 — Run Execution Backbone

The MVP must include the basic AVCS execution spine.

Required execution capabilities:

- create run manifest
- preflight validation
- resolve included domains by run type
- execute domain evaluators
- capture run state transitions
- finalize run with preserved results

Why it is required:
Without this, AVCS is not operationally real.

AVCS-16.5 — Evidence Collection

The MVP must already support evidence, even if the evidence model starts narrow.

Minimum evidence classes for MVP:

- run manifest
- domain execution logs
- core console/runtime output

- screenshots for visible failures where applicable
- security artifacts
- performance metrics
- coverage summary
- verdict summary evidence

Why it is required:
Evidence is part of AVCS identity, not a later enhancement.

AVCS-16.6 — Findings Engine

The MVP must convert meaningful signals into structured findings.

Minimum finding capabilities:

- finding ID
- domain
- severity
- impact class
- affected surface
- concise summary
- linked evidence
- certification effect

Minimum supported finding classes:

- failure findings
- risk findings

- policy findings

Why it is required:
Operators need issue objects, not raw logs.

AVCS-16.7 — Basic Coverage Model

The MVP does not need the most advanced coverage system yet, but it must have an honest one.

Minimum coverage dimensions:

- domain coverage
- critical workflow coverage
- required-surface coverage
- coverage state by item

Minimum coverage states:

- covered
- covered-failed
- partial
- blocked
- not tested

Why it is required:
The system must already be able to show what was and was not validated.

AVCS-16.8 — Certification Verdict Engine

The MVP must compute real certification outcomes.

Required verdict states:

- **PASS**
- **PASS_WITH_WARNINGS**
- **FAIL**
- **BLOCKED**

`CONDITIONAL_PASS` can be added in MVP if override policy is implemented early, but it is acceptable to defer that until the next phase if needed.

Required verdict inputs:

- domain outcomes
- blocker detection
- coverage sufficiency
- evidence completeness
- policy checks

Why it is required:
A certification suite without a verdict engine is just a test harness.

AVCS-16.9 — Run History Persistence

The MVP must persist runs and preserve them immutably after finalization.

Minimum historical records:

- run manifest
- domain outcomes
- findings

- coverage summary
- verdict
- timestamps
- target linkage

Why it is required:
This creates the foundation for audit, rerun comparison, and future regression support.

AVCS-16.10 — Operator UI Surface

The MVP must provide a usable operator-facing UI.

Minimum required UI surfaces:

- **Certification Command Center**
- **Run Detail View**
- **Findings / Failure Triage View**
- **Coverage Matrix**
- **Release Gate Panel**

These should be functional, not decorative.

Why it is required:
AVCS is meant to be operated, not only executed in the background.

---

6. MVP run behavior

6.1 Smoke Run in MVP

The MVP Smoke Run should validate:

- build target is structurally usable
- startup path works
- critical shell/routes load
- obvious crashes are caught early

6.2 Functional Run in MVP

The MVP Functional Run should validate:

- 2–5 critical workflows
- primary action success path
- important error-state handling
- core API/interaction correctness

6.3 Security Run in MVP

The MVP Security Run should validate:

- obvious auth/permission failures
- secret leakage
- dependency vulnerabilities
- unsafe exposed surfaces/configs

6.4 Performance Run in MVP

The MVP Performance Run should validate:

- startup/load timing
- major route or interaction timing
- obvious budget or regression issues

- repeated action stability at basic depth

6.5 Full Certification Run in MVP

The MVP Full Certification Run should assemble:

- all MVP domains
- evidence summary
- coverage summary
- findings summary
- final verdict

This gives the MVP one fully enterprise-shaped run path.

---

7. MVP blocker model

The MVP should already include a hard-stop blocker model.

Minimum blocker classes:

- app fails to boot
- critical workflow failure
- severe auth/permission issue
- exposed secret
- missing required deployment config
- required domain not completed
- required evidence missing for major result

Why this matters:
This prevents the MVP from becoming score-first or pass-first too early.

---

8. MVP policy model

The MVP policy layer can be narrower than the full future model, but it must exist.

Minimum MVP policy should define:

- required domains by run type
- minimum coverage requirement for Full Certification Run
- blocker classes
- evidence minimums
- verdict mapping rules
- freshness rule for major runs if applicable

What can wait:

- advanced build-class variants
- full override authority matrix
- deep policy simulation
- complex conditional-pass governance

The important thing is to have an explicit rule base from the start.

---

9. MVP scoring model

Scoring in the MVP should exist, but it can stay simple.

Recommended MVP scoring:

- domain-level readiness scores

- overall readiness score
- simple weighted model across MVP domains
- score shown only alongside verdict and blockers

What the MVP should avoid:

- overly sophisticated formulas
- false precision
- score dominating operator judgment

The MVP scoring layer should be informative, not overengineered.

---

10. MVP evidence depth

The MVP does not need the richest possible evidence model on day one, but it should already preserve enough proof to trust the verdict.

Strong MVP evidence standard:

- every domain result has at least one direct evidence anchor
- every blocker finding has direct linked evidence
- every final verdict includes a reasoning summary and supporting evidence summary
- every run has preserved execution identity and timestamps

That is enough to make the first version credible.

---

11. MVP operator experience standard

The first operator experience should already feel like a control surface, not a raw admin page.

Minimum expectations:

- run launch is straightforward
- active run state is visible
- blockers are obvious
- findings can be sorted by severity/domain
- coverage gaps are visible
- final release recommendation is unmistakable
- evidence can be opened from findings and run detail

It does not need every advanced workbench yet, but it must already feel usable.

---

12. What is explicitly out of MVP

To keep the first version disciplined, these items can be deferred:

- dedicated UI Quality workbench
- dedicated UX Workflow scoring engine
- advanced Accessibility automation breadth
- full Enterprise Readiness depth
- deep rerun lineage graphing
- formal override workflow system
- complex build-class policy variants
- multi-build portfolio dashboards
- predictive release-risk analytics
- advanced remediation tracking

These are valuable, but they should not delay the first certifiable version of AVCS.

---

13. MVP success criteria

Minimum Viable AVCS should be considered successful if it can do the following consistently:

- certify a real build target from Build Mode
- catch obvious structural, functional, security, performance, or deployment blockers
- produce structured findings with linked evidence
- show domain and workflow coverage honestly
- compute a trustworthy verdict
- preserve run history
- give operators a clear go / no-go surface

If it can do those things, it is already a meaningful AVCS foundation.

---

14. Recommended MVP release sequence

A strong implementation order would be:

Phase 1

- target selection
- run manifest
- Smoke Run
- Build Integrity domain
- Run Detail basics

Phase 2

- Functional Run
- Functional Correctness domain
- findings model
- basic coverage model

Phase 3

- Security Run
- Security Posture domain
- blocker logic
- Release Gate basics

Phase 4

- Performance Run
- Performance & Stability domain
- simple scoring

Phase 5

- Deployment Readiness domain
- Full Certification Run
- history persistence
- Command Center + Coverage Matrix + Triage UI polish

This gets you to a real AVCS fast without trying to perfect every domain at once.

---

15. Canonical MVP statement

Use a stable summary statement for this section:

Minimum Viable AVCS is the first enterprise-usable version of the suite that can target a build output, run core verification modes across high-priority assurance domains, collect evidence, generate findings, measure essential coverage, compute a certification verdict, preserve run history, and present operators with a usable release gate interface.

---

16. Section close

AVCS-16 should make the first implementation path clear:

- do not build everything at once
- build the certification backbone first
- focus on the highest-risk domains first
- make evidence, findings, coverage, and verdict real from day one
- make the operator experience usable early
- leave advanced workbenches and deep policy variants for later

That gives AVCS a practical first version that is already true to its purpose.

AVCS-17

AVCS-17 — Future Expansion

1. Purpose of this section

This section defines how **Axion Verification & Certification Suite (AVCS)** should expand after the minimum viable system is stable.

If **AVCS-16** defines the first real version, **AVCS-17** defines the growth path that turns AVCS from a strong certification backbone into a more mature, broader, and more intelligent release-quality platform.

This section matters because AVCS is naturally expandable in many directions:

- broader assurance coverage
- deeper automation
- stronger policy sophistication
- richer operator workflows
- better historical intelligence
- wider integration with adjacent Axion systems

Without a clear expansion section, the system can grow chaotically. With it, future work stays aligned with the original identity of AVCS.

---

2. Core future-expansion statement

Future AVCS development should expand the suite by increasing:

- assurance depth
- domain breadth
- policy sophistication
- operator leverage

- historical intelligence
- integration maturity

while preserving the core AVCS identity:

evidence-backed, policy-governed, operator-legible certification of build outputs before release progression

Expansion should make AVCS stronger, not blurrier.

---

3. Expansion principles

Future AVCS work should follow eight rules:

3.1 Preserve the certification core

New features should strengthen verification and certification, not distract from them.

3.2 Expand depth before novelty

It is often better to deepen weak existing domains than to add flashy but shallow new ones.

3.3 Keep evidence central

Automation and intelligence features must still remain evidence-backed.

3.4 Protect operator clarity

More power should not make the suite harder to understand.

3.5 Maintain policy discipline

New capabilities must still respect explicit gates and override controls.

3.6 Preserve append-only truth

Historical integrity must survive future complexity.

3.7 Favor reusable contracts

New subsystems should integrate through the same disciplined contracts established in the core system.

3.8 Grow by layers

Expansion should happen in controlled layers rather than one giant second rewrite.

These principles keep future AVCS growth from drifting.

---

4. Main expansion tracks

AVCS expansion should happen across a small number of strategic tracks.

Recommended tracks:

1. **Assurance Depth Expansion**
2. **Operator Experience Expansion**
3. **Policy and Governance Expansion**
4. **Historical Intelligence Expansion**
5. **Integration Expansion**
6. **Automation and Assistance Expansion**

These tracks give future work a clean structure.

---

5. Assurance Depth Expansion

The first major growth path should deepen the quality coverage of AVCS itself.

AVCS-17.1 — UI Quality Expansion

Future AVCS versions should add:

- stronger visual regression detection
- component-state snapshot comparison
- viewport-specific defect capture

- layout integrity heuristics
- richer responsive coverage maps

This would make UI Quality a first-class domain rather than a lighter add-on.

AVCS-17.2 — UX Workflow Expansion

Future versions should deepen UX validation through:

- explicit workflow-friction scoring
- dead-end and recovery-path analysis
- step-efficiency evaluation
- success-path clarity assessment
- workflow heatmaps or repeated-friction patterns

This would strengthen AVCS's ability to certify usability, not just correctness.

AVCS-17.3 — Accessibility Expansion

Future versions should add:

- broader workflow accessibility coverage
- richer accessible-state validation
- comparative accessibility regression detection
- stronger primary-action accessibility gating
- more structured accessibility evidence packs

This is an important maturity path for enterprise readiness.

AVCS-17.4 — Enterprise Readiness Expansion

Future AVCS versions should deepen:

- auditability validation
- supportability scoring
- logging quality checks
- operator remediation clarity checks
- release-hand-off artifact completeness
- role/permission documentation clarity

This would make AVCS stronger as a deployment-quality certifier, not just a product-quality evaluator.

---

6. Operator Experience Expansion

After MVP surfaces are stable, AVCS should grow into a more powerful control environment.

AVCS-17.5 — Advanced Domain Workbenches

Future versions should provide deeper workbenches for:

- Security
- Performance
- Accessibility
- Deployment Readiness
- UX

These workbenches could include richer drill-down, comparison, clustering, and policy interpretations.

AVCS-17.6 — Certification Timeline Playback

Future versions could provide a timeline-style replay of:

- execution stages
- surfaced findings
- evidence captures
- verdict assembly
- rerun and override events

This would make run review much easier for complex certifications.

AVCS-17.7 — Multi-Build / Portfolio Views

As the system scales, operators may need portfolio-level visibility across many build kits.

Future views could show:

- builds by readiness state
- blocker concentration by team or area
- recurring weak domains
- quality drift across product lines
- certification trend maps across many outputs

This becomes especially valuable once Axion is handling many kits regularly.

AVCS-17.8 — Review Notes and Decision Annotations

Future versions may support append-only operator notes such as:

- release review comments

- acceptance rationale
- remediation observations
- comparison notes
- gate-review annotations

This would improve collaboration without compromising historical integrity.

---

7. Policy and Governance Expansion

As AVCS matures, policy should become more expressive.

AVCS-17.9 — Build-Class Policy Packs

Future versions should support richer policy packs tuned to different build classes, such as:

- admin-heavy internal systems
- customer-facing applications
- security-sensitive control surfaces
- deployment-critical tooling
- workflow-intensive operator consoles

This allows certification to become more context-aware without becoming inconsistent.

AVCS-17.10 — Formal Override Workflow

Future versions may add:

- explicit override request lifecycle
- authority checks

- multi-party approval
- mandatory justification templates
- override expiry or review reminders
- clearer distinction between effective progression state and original verdict state

This is an important governance expansion once AVCS has real organizational weight.

AVCS-17.11 — Policy Simulation and Impact Preview

A mature AVCS could allow operators or system designers to simulate:

- what verdict would occur under a stricter policy
- how a build would behave under a different build-class profile
- what requirement prevented PASS
- what minimum changes would change the verdict

This would be highly useful, but should come after the base policy system is solid.

---

8. Historical Intelligence Expansion

Once history accumulates, AVCS can become much smarter.

AVCS-17.12 — Quality Trend Intelligence

Future versions should improve trend analysis across:

- verdict movement
- score drift
- recurring blockers

- domain-level weakness accumulation
- coverage shrinkage
- evidence completeness decline

This helps AVCS act as a strategic quality signal, not only a per-run gate.

AVCS-17.13 — Persistent Issue Intelligence

Future versions could identify:

- repeat offenders by workflow
- findings that reappear after “resolution”
- domain areas with chronic fragility
- recurring deployment-readiness failures
- regressions that cluster around certain change types

This would make historical data far more actionable.

AVCS-17.14 — Comparative Certification Profiles

Future versions could compare:

- build A vs build B
- current branch vs release branch
- team-level certification quality
- release-train readiness over time

This would support broader Axion quality management.

---

9. Integration Expansion

Future AVCS versions can connect more deeply to adjacent systems while still respecting the boundary model.

AVCS-17.15 — Maintenance System Integration

AVCS outputs could feed maintenance systems for:

- recurring issue prioritization
- quality debt tracking
- suggested hardening areas
- domain-level improvement backlogs

This would make AVCS a major source of system-improvement intelligence.

AVCS-17.16 — Release Workflow Integration

Future versions could offer richer downstream release hooks such as:

- stronger final gate signals
- richer verdict contracts
- policy-compliance export
- evidence-pack handoff objects
- deployment-readiness handoff summaries

Still, AVCS should remain the certifier, not the deployer.

AVCS-17.17 — External Audit / Export Surfaces

Future versions may support:

- exportable certification summaries
- audit-focused run packs
- evidence export bundles
- structured exception and override reports

This would be useful for governance-heavy environments.

---

10. Automation and Assistance Expansion

AVCS can also become more proactive over time, but carefully.

AVCS-17.18 — Smart Finding Clustering

Future versions may cluster related findings automatically to reduce triage noise.

Examples:

- many route failures tied to one auth defect
- many layout symptoms tied to one CSS regression
- repeated workflow breakage tied to one shared dependency issue

This could make the suite much easier to operate at scale.

AVCS-17.19 — Suggested Remediation Guidance

AVCS could eventually provide structured remediation hints such as:

- likely affected module or surface
- most probable cause class
- recommended next validation step

- highest-leverage fix order

This should stay suggestive, not authoritative, unless evidence strongly supports it.

AVCS-17.20 — Predictive Release Risk

A later AVCS could estimate likely release fragility based on:

- historical blocker recurrence
- coverage shrinkage
- recent domain weakness trends
- score volatility
- repeated override patterns

This would be useful, but only after AVCS has enough historical integrity to justify it.

---

11. Expansion by maturity phase

A clean future roadmap can be framed in maturity waves.

Phase A — Post-MVP Stabilization

Focus on:

- making MVP domains stronger
- improving evidence reliability
- improving blocker precision
- polishing Command Center, Run Detail, Coverage, and Release Gate flows

Phase B — Full Domain Maturity

Focus on:

- UI Quality
- UX Workflow Quality
- Accessibility depth
- Enterprise Readiness depth
- broader comparative runs

Phase C — Governance and Review Maturity

Focus on:

- formal override workflows
- richer policy packs
- deeper gate logic
- better audit and review views

Phase D — Intelligence and Scale

Focus on:

- portfolio views
- issue intelligence
- predictive quality trends
- richer remediation support
- stronger multi-build comparison

This keeps future growth orderly.

---

12. What should not happen in expansion

Future AVCS work should explicitly avoid several bad growth patterns.

AVCS should not expand by:

- turning into a generic CI clone
- replacing deployment orchestration
- becoming a vague analytics dashboard
- adding AI-flavored features without evidence grounding
- weakening blocker logic for convenience
- burying operators in too many surfaces
- prioritizing novelty over certification truth

This is important because large systems often lose identity as they expand.

---

13. Expansion readiness criteria

A future feature should be considered ready to enter AVCS only if it:

- strengthens certification quality
- has clear evidence implications
- fits the domain or policy model cleanly
- preserves operator clarity
- does not blur responsibility boundaries
- can be historically represented without breaking audit integrity

That should be the filter for future growth.

---

14. Canonical future-expansion statement

Use a stable summary statement for this section:

AVCS should expand by deepening assurance domains, enriching operator review workflows, maturing policy and override governance, strengthening historical intelligence, and improving integration with adjacent Axion systems while preserving evidence-backed, policy-governed certification discipline.

---

15. Section close

AVCS-17 should make the long-term path clear:

- MVP is only the beginning
- future work should deepen and sharpen the suite
- policy, evidence, and operator clarity must remain central
- expansion should strengthen certification, not dilute it
- AVCS can grow into a major quality authority inside Axion without losing its boundaries

That completes the full clean outline for **Axion Verification & Certification Suite (AVCS)**.

AVCS Build Plan

AVCS Build Plan

Phase Map

Phase 1 — Foundation

Establish the system backbone so AVCS can exist as a real subsystem.

Deliverables:

- AVCS system folder structure
- run manifest model
- target build selection model
- run lifecycle/state model
- base domain evaluator contract
- base evidence contract
- base finding contract
- base verdict contract
- base history record contract
- Command Center shell
- Run Detail shell

Goal:
Make AVCS capable of identifying a build, starting a run, tracking state, and storing structured outputs.

Phase 2 — Core Certification Engine

Build the minimum certification mechanics.

Deliverables:

- Smoke Run
- Functional Run
- Build Integrity evaluator
- Functional Correctness evaluator
- findings normalization pipeline
- evidence registration pipeline
- basic coverage assembly
- basic blocker logic
- verdict engine v1
- Release Gate panel v1

Goal:
Make AVCS able to perform actual verification and return a real verdict.

Phase 3 — Security + Performance Layer

Add the most important enterprise-quality enforcement domains.

Deliverables:

- Security Run
- Performance Run
- Security Posture evaluator
- Performance & Stability evaluator

- security findings model extensions
- performance metrics/evidence extensions
- domain scoring v1
- blocker rules for auth/secret/security/perf issues

Goal:
Make AVCS strong enough to catch serious release risks.

Phase 4 — Deployment + History Layer

Make the suite operationally trustworthy over time.

Deliverables:

- Deployment Readiness evaluator
- Full Certification Run
- append-only run history
- rerun linkage
- verdict history
- evidence inventory persistence
- comparison primitives
- historical run list
- history/regression view v1

Goal:
Make AVCS durable and reviewable, not just executable.

Phase 5 — Operator Experience Completion

Turn the backend certification system into a proper control center.

Deliverables:

- Coverage Matrix
- Findings / Failure Triage Board
- Evidence Explorer
- Release Gate panel polish
- domain cards/status summaries
- run filters/search
- build/run comparison surfaces
- score displays and trend indicators

Goal:
Make AVCS feel like a true operator-grade command environment.

Phase 6 — Expansion Layer

Everything beyond MVP.

Deliverables:

- UI Quality evaluator
- UX Workflow evaluator
- Accessibility evaluator
- Enterprise Readiness evaluator
- Regression Run
- Pre-Deployment Gate Run

- override workflows
- advanced policy packs
- deeper workbenches
- portfolio/history intelligence

Goal:
Grow from strong MVP to full AVCS maturity.

Recommended Repo Shape

Top-level AVCS areas

```
App/
 src/
   pages/
     avcs/
     ...

   components/
     avcs/
     ...

   hooks/
     avcs/
     ...

   types/
     avcs/
     ...

server/
 avcs/
   core/
   runs/
   evaluators/
   evidence/
   findings/
```

```
  coverage/
  verdict/
  policy/
  history/
  scoring/
  integrations/
  utils/

shared/
 avcs/
  schemas/
  contracts/
  enums/
  constants/
```

Detailed Folder / File Map

Frontend

```
App/src/pages/avcs/
 avcs-command-center.tsx
 avcs-run-detail.tsx
 avcs-coverage.tsx
 avcs-findings.tsx
 avcs-history.tsx
 avcs-release-gate.tsx

App/src/components/avcs/
 command-center/
  build-target-selector.tsx
  run-type-selector.tsx
  avcs-status-hero.tsx
  recent-runs-panel.tsx
  certification-summary-card.tsx

 run-detail/
  run-header.tsx
  run-state-timeline.tsx
  domain-status-grid.tsx
  run-evidence-summary.tsx
  run-verdict-card.tsx
```

```
coverage/
  coverage-matrix.tsx
  coverage-state-badge.tsx
  workflow-coverage-table.tsx
  domain-coverage-panel.tsx

findings/
  findings-table.tsx
  findings-filter-bar.tsx
  finding-detail-drawer.tsx
  severity-badge.tsx
  impact-badge.tsx

evidence/
  evidence-explorer.tsx
  evidence-preview-panel.tsx
  screenshot-viewer.tsx
  log-viewer.tsx
  metric-card.tsx

release-gate/
  release-gate-summary.tsx
  blocker-list.tsx
  policy-status-panel.tsx
  verdict-reasoning-card.tsx

shared/
  avcs-shell-header.tsx
  run-status-badge.tsx
  verdict-badge.tsx
  domain-chip.tsx
  freshness-indicator.tsx
```

Backend

```
server/avcs/core/
 avcs-runner.ts
 run-lifecycle.ts
 run-manifest-service.ts
 target-resolution-service.ts
 domain-plan-resolver.ts

server/avcs/runs/
```

```
 smoke-run.ts
 functional-run.ts
 security-run.ts
 performance-run.ts
 full-certification-run.ts

server/avcs/evaluators/
 build-integrity/
   build-integrity-evaluator.ts
   build-integrity-checks.ts
 functional-correctness/
   functional-correctness-evaluator.ts
   workflow-checks.ts
 security-posture/
   security-posture-evaluator.ts
   security-checks.ts
 performance-stability/
   performance-stability-evaluator.ts
   performance-checks.ts
 deployment-readiness/
   deployment-readiness-evaluator.ts
   deployment-checks.ts

server/avcs/evidence/
 evidence-registry.ts
 evidence-storage.ts
 evidence-linker.ts
 evidence-finalizer.ts

server/avcs/findings/
 findings-normalizer.ts
 findings-deduplicator.ts
 findings-classifier.ts
 findings-service.ts

server/avcs/coverage/
 coverage-assembler.ts
 domain-coverage.ts
 workflow-coverage.ts
 coverage-thresholds.ts

server/avcs/verdict/
 verdict-engine.ts
 blocker-evaluator.ts
```

```
 verdict-reasoner.ts

server/avcs/policy/
 policy-resolver.ts
 run-type-policy.ts
 blocker-policy.ts
 evidence-policy.ts
 coverage-policy.ts

server/avcs/history/
 run-history-service.ts
 event-history-service.ts
 rerun-lineage-service.ts
 comparison-service.ts

server/avcs/scoring/
 domain-score-service.ts
 readiness-score-service.ts
 scoring-weights.ts

server/avcs/integrations/
 build-mode-adapter.ts
 artifact-metadata-adapter.ts
 release-gate-adapter.ts

server/avcs/utils/
 timestamps.ts
 ids.ts
 guards.ts
 mappers.ts
```

Shared contracts

```
shared/avcs/contracts/
 run-manifest.contract.ts
 run-summary.contract.ts
 domain-result.contract.ts
 finding.contract.ts
 evidence.contract.ts
 coverage.contract.ts
 verdict.contract.ts
 history-record.contract.ts
 policy-snapshot.contract.ts
```

Artifact List

Core runtime artifacts

These are the minimum objects AVCS needs to create and preserve.

1. `run_manifest.json`
2. `run_summary.json`
3. `domain_results.json`
4. `findings.json`
5. `coverage_summary.json`
6. `evidence_inventory.json`
7. `verdict.json`
8. `policy_snapshot.json`
9. `run_history_record.json`

Domain artifacts

By MVP domain:

Build Integrity

- install/build/type/lint result record
- required-files check result
- env-contract check result
- startup viability result

Functional Correctness

- workflow result records
- API/interaction result records
- error-state result records

Security Posture

- auth boundary result record
- dependency scan result
- secret exposure result
- config exposure result

Performance & Stability

- startup timing record
- route timing record
- repeated-action timing/stability record
- budget check result

Deployment Readiness

- env completeness result
- startup path result
- readiness/health result
- deployment config result

Schema List

Must-have schemas first

1. `AVCSRunManifest`
2. `AVCSRunState`
3. `AVCSRunSummary`
4. `AVCSDomainResult`
5. `AVCSFinding`
6. `AVCSEvidenceItem`
7. `AVCSCoverageItem`
8. `AVCSVerdict`
9. `AVCSPolicySnapshot`
10. `AVCSHistoryRecord`

Important enums

1. `AVCSRunType`
2. `AVCSDomain`
3. `AVCSRunStatus`
4. `AVCSFindingSeverity`
5. `AVCSFindingImpactClass`

6. `AVCSCoverageState`
7. `AVCSVerdictState`
8. `AVCSEvidenceType`

UI Screen List

MVP screens

1. AVCS Command Center

Purpose:

- select build target
- select run type
- launch run
- view recent runs
- view latest verdict status

2. AVCS Run Detail

Purpose:

- view active or completed run
- see run state timeline
- see domain outcomes
- inspect findings and evidence summary

- see verdict summary

3. AVCS Findings / Failure Triage

Purpose:

- sort by severity/domain/impact
- inspect issue details
- open linked evidence

4. AVCS Coverage Matrix

Purpose:

- show domains/workflows/surfaces
- distinguish covered, failed, partial, blocked, not tested

5. AVCS Release Gate

Purpose:

- final readiness summary
- blocker list
- policy status
- verdict reasoning

Later screens

- AVCS History & Regression
- Security Workbench
- Performance Workbench

- Evidence Explorer full view
- Accessibility Workbench
- UI/UX Workbench

Execution Component List

Core engine components

1. Target Resolver
2. Run Manifest Builder
3. Preflight Validator
4. Domain Plan Resolver
5. Run Orchestrator
6. Domain Evaluator Interface
7. Evidence Registry
8. Findings Normalizer
9. Coverage Assembler
10. Blocker Evaluator
11. Verdict Engine
12. History Persister

Supporting components

1. Policy Resolver
2. Scoring Engine
3. Comparison Service
4. Rerun Linkage Service
5. Build Mode Adapter
6. Release Gate Adapter

MVP Domain Implementation Order

Order I recommend

1. Build Integrity

Why first:
Everything else depends on whether the target is structurally valid.

2. Functional Correctness

Why second:
This gives AVCS real product-quality meaning.

3. Security Posture

Why third:
This adds enterprise-grade blocker power fast.

4. Performance & Stability

Why fourth:
This catches a major class of release quality issues early.

5. Deployment Readiness

Why fifth:
This completes the core release gate backbone.

Suggested Build Sequence by Actual Work Items

Wave 1

- shared enums/constants
- run manifest schema
- run summary schema
- verdict schema
- backend AVCS folder scaffolding
- frontend page scaffolding

Wave 2

- target resolution
- run lifecycle states
- Command Center basic launch flow
- Run Detail basic state rendering

Wave 3

- Build Integrity evaluator
- Smoke Run

- evidence registry v1
- domain result output v1

Wave 4

- Functional evaluator
- Functional Run
- findings normalizer v1
- coverage assembler v1

Wave 5

- blocker evaluator
- verdict engine v1
- Release Gate panel v1

Wave 6

- Security evaluator/run
- Performance evaluator/run
- scoring v1

Wave 7

- Deployment Readiness evaluator
- Full Certification Run
- history persistence

- findings triage page
- coverage matrix page

AVCS schema set

AVCS schema set

1. enums
2. run manifest
3. domain result
4. finding
5. evidence
6. coverage
7. verdict
8. history record

AVCS Schema 01 — Enums

AVCS Schema 01 — Enums

Location recommended:

shared/avcs/enums/avcs.enums.ts

These enums define the **canonical vocabulary of the AVCS system**.
All schemas and logic reference these.

Run Types

```
export enum AVCSRunType {
 SMOKE = "SMOKE",
 FUNCTIONAL = "FUNCTIONAL",
 SECURITY = "SECURITY",
 PERFORMANCE = "PERFORMANCE",
 FULL_CERTIFICATION = "FULL_CERTIFICATION",
 REGRESSION = "REGRESSION",
 PRE_DEPLOYMENT_GATE = "PRE_DEPLOYMENT_GATE"
}
```

Purpose:
Defines the **types of certification runs** the suite can execute.

Domains

```
export enum AVCSDomain {
 BUILD_INTEGRITY = "BUILD_INTEGRITY",
 FUNCTIONAL_CORRECTNESS = "FUNCTIONAL_CORRECTNESS",
 SECURITY_POSTURE = "SECURITY_POSTURE",
 PERFORMANCE_STABILITY = "PERFORMANCE_STABILITY",
 UI_QUALITY = "UI_QUALITY",
 UX_WORKFLOW = "UX_WORKFLOW",
 ACCESSIBILITY = "ACCESSIBILITY",
 DEPLOYMENT_READINESS = "DEPLOYMENT_READINESS",
 ENTERPRISE_READINESS = "ENTERPRISE_READINESS"
```

```
}
```

Purpose:
Defines the **assurance domains AVCS evaluates**.

MVP uses first five.

Run Status

```
export enum AVCSRunStatus {
 CREATED = "CREATED",
 PREFLIGHT = "PREFLIGHT",
 RUNNING = "RUNNING",
 FINALIZING = "FINALIZING",
 COMPLETED = "COMPLETED",
 FAILED = "FAILED",
 BLOCKED = "BLOCKED"
}
```

Purpose:
Represents the **lifecycle state of a certification run**.

Finding Severity

```
export enum AVCSFindingSeverity {
 INFO = "INFO",
 LOW = "LOW",
 MEDIUM = "MEDIUM",
 HIGH = "HIGH",
 CRITICAL = "CRITICAL"
}
```

Purpose:
Severity classification for findings.

Finding Impact Class

```
export enum AVCSFindingImpactClass {
 FUNCTIONAL = "FUNCTIONAL",
 SECURITY = "SECURITY",
 PERFORMANCE = "PERFORMANCE",
 RELIABILITY = "RELIABILITY",
 DEPLOYMENT = "DEPLOYMENT",
 USABILITY = "USABILITY",
 COMPLIANCE = "COMPLIANCE"
}
```

Purpose:
 Indicates **what system property the issue affects**.

Coverage State

```
export enum AVCSCoverageState {
 COVERED = "COVERED",
 COVERED_FAILED = "COVERED_FAILED",
 PARTIAL = "PARTIAL",
 BLOCKED = "BLOCKED",
 NOT_TESTED = "NOT_TESTED"
}
```

Purpose:
 Used in the **Coverage Matrix** to represent validation depth.

Verdict State

```
export enum AVCSVerdictState {
 PASS = "PASS",
 PASS_WITH_WARNINGS = "PASS_WITH_WARNINGS",
 CONDITIONAL_PASS = "CONDITIONAL_PASS",
 FAIL = "FAIL",
 BLOCKED = "BLOCKED"
}
```

Purpose:
The **final certification outcome**.

Evidence Types

```
export enum AVCSEvidenceType {
 LOG = "LOG",
 SCREENSHOT = "SCREENSHOT",
 METRIC = "METRIC",
 REPORT = "REPORT",
 TRACE = "TRACE",
 ARTIFACT = "ARTIFACT",
 SUMMARY = "SUMMARY"
}
```

Purpose:
Defines types of evidence produced during runs.

Domain Status

```
export enum AVCSDomainStatus {
 NOT_STARTED = "NOT_STARTED",
 RUNNING = "RUNNING",
 PASSED = "PASSED",
 FAILED = "FAILED",
 BLOCKED = "BLOCKED",
 PARTIAL = "PARTIAL"
}
```

Purpose:
Represents **per-domain execution state**.

Run Actor Type

```
export enum AVCSActorType {
 USER = "USER",
```

```
 SYSTEM = "SYSTEM",
 AUTOMATION = "AUTOMATION"
}
```

Purpose:
Tracks who initiated the run.

Why this schema comes first

Every other AVCS schema will reference these.

For example:

RunManifest uses:

- `AVCSRunType`
- `AVCSRunStatus`

DomainResult uses:

- `AVCSDomain`
- `AVCSDomainStatus`

Finding uses:

- `AVCSFindingSeverity`
- `AVCSFindingImpactClass`

Coverage uses:

- `AVCSCoverageState`

Verdict uses:

- `AVCSVerdictState`

Evidence uses:

- `AVCSEvidenceType`

AVCS Schema 02 — Run Manifest

AVCS Schema 02 — Run Manifest

Recommended location:

shared/avcs/contracts/run-manifest.contract.ts

The **Run Manifest** defines:

- what build is being certified
- what run type is being executed
- what domains will run
- what policy snapshot applies
- who initiated the run
- when the run started
- what the initial configuration of the run was

This object is created **before execution begins** and becomes the **anchor record for the entire run lifecycle**.

TypeScript Schema

```
import {
 AVCSRunType,
 AVCSRunStatus,
 AVCSDomain,
 AVCSActorType
} from "../enums/avcs.enums";

export interface AVCSRunManifest {
 runId: string;

 target: AVCSRunTarget;
```

```
 runType: AVCSRunType;

 includedDomains: AVCSDomain[];

 actor: AVCSRunActor;

 policySnapshotId: string;

 createdAt: string;

 status: AVCSRunStatus;

 metadata?: Record<string, any>;
}
```

Target Object

Defines the **build output being certified**.

```
export interface AVCSRunTarget {
 buildId: string;

 buildVersion: string;

 artifactLocation: string;

 buildClass?: string;

 generatedAt?: string;

 metadata?: Record<string, any>;
}
```

Example values:

```
buildId: "build_7421"
buildVersion: "v1.3.0"
artifactLocation: "/builds/kit_7421"
buildClass: "web_application"
generatedAt: "2026-03-05T12:20:44Z"
```

Actor Object

Defines **who initiated the certification run**.

```
export interface AVCSRunActor {
 actorId: string;

 actorType: AVCSActorType;

 name?: string;
}
```

Examples:

User launch

```
actorId: "user_1"
actorType: USER
name: "Operator"
```

Automation launch

```
actorId: "build_pipeline"
actorType: AUTOMATION
```

Example Run Manifest


```
{
 "runId": "run_20260306_001",

 "target": {
   "buildId": "kit_7421",
   "buildVersion": "v1.3.0",
   "artifactLocation": "/artifacts/builds/kit_7421",
   "buildClass": "web_application",
   "generatedAt": "2026-03-05T12:20:44Z"
 },

 "runType": "FULL_CERTIFICATION",

 "includedDomains": [
   "BUILD_INTEGRITY",
```

```
  "FUNCTIONAL_CORRECTNESS",
  "SECURITY_POSTURE",
  "PERFORMANCE_STABILITY",
  "DEPLOYMENT_READINESS"
 ],

 "actor": {
  "actorId": "user_1",
  "actorType": "USER",
  "name": "Operator"
 },

 "policySnapshotId": "policy_2026_03_v1",

 "createdAt": "2026-03-06T04:55:11Z",

 "status": "CREATED"
}
```

Why the Run Manifest is Critical

The run manifest becomes the **root reference for everything in the run**.

All artifacts attach to it.

```
RunManifest
  │
  ├── DomainResults
  │
  ├── Findings
  │
  ├── Evidence
  │
  ├── Coverage
  │
  ├── Verdict
  │
  └── HistoryRecords
```

It ensures:

- reproducibility
- auditability
- traceability
- deterministic run context

Without this object, AVCS would not be able to reconstruct what happened during a certification run.

Runtime Role in the System

When a run starts, the system does:

1 Build target resolved
2 Run type selected
3 Policy snapshot resolved
4 Domains resolved
5 Run manifest created
6 Run execution begins

The manifest is written **before any evaluation occurs**.

Additional Best Practice (Important)

The manifest should **never be mutated except for the `status` field**.

Everything else should remain immutable so the run context stays trustworthy.

AVCS Schema 03 — Domain Result

AVCS Schema 03 — Domain Result

Recommended location:

shared/avcs/contracts/domain-result.contract.ts

The **Domain Result** is the per-domain output object for a run.

If the Run Manifest is the root object, the Domain Result is the main execution result object that feeds:

- findings
- coverage
- scoring
- blocker evaluation
- final verdict

Each included domain in a run should produce one Domain Result.

TypeScript Schema

```
import {
 AVCSDomain,
 AVCSDomainStatus,
 AVCSCoverageState
} from "../enums/avcs.enums";

export interface AVCSDomainResult {
 domainResultId: string;

 runId: string;

 domain: AVCSDomain;

 status: AVCSDomainStatus;
```

```
 startedAt?: string;

 completedAt?: string;

 summary: string;

 score?: number;

 coverageState: AVCSCoverageState;

 findings: AVCSFindingRef[];

 evidence: AVCSEvidenceRef[];

 checks: AVCSDomainCheckResult[];

 metadata?: Record<string, any>;
}
```

Finding Reference

Domain Result should not embed full findings directly.
It should reference them.

```
export interface AVCSFindingRef {
 findingId: string;
}
```

Evidence Reference

Same idea for evidence.

```
export interface AVCSEvidenceRef {
 evidenceId: string;
}
```

Domain Check Result

This is the important sub-object.

A domain is usually composed of multiple checks, not one big pass/fail.

```
export interface AVCSDomainCheckResult {
 checkId: string;

 checkKey: string;

 name: string;

 status: AVCSDomainCheckStatus;

 summary: string;

 startedAt?: string;

 completedAt?: string;

 findings?: AVCSFindingRef[];

 evidence?: AVCSEvidenceRef[];

 metadata?: Record<string, any>;
}
```

Domain Check Status Enum

Add this to your enums file.

```
export enum AVCSDomainCheckStatus {
 NOT_STARTED = "NOT_STARTED",
 RUNNING = "RUNNING",
 PASSED = "PASSED",
 FAILED = "FAILED",
 BLOCKED = "BLOCKED",
 PARTIAL = "PARTIAL",
 SKIPPED = "SKIPPED"
}
```

This gives you check-level resolution inside the domain.

Why this schema matters

A domain like `SECURITY_POSTURE` or `FUNCTIONAL_CORRECTNESS` is too broad to treat as one opaque result.

You need to see:

- what checks ran inside the domain
- which checks failed
- what evidence came from them
- what findings they produced
- what the aggregate domain status became

So the relationship is:

```
RunManifest
 └── DomainResult
     ├── DomainCheckResult[]
     ├── FindingRef[]
     └── EvidenceRef[]
```

Example: Build Integrity Domain Result


```
{
 "domainResultId": "domain_result_001",
 "runId": "run_20260306_001",
 "domain": "BUILD_INTEGRITY",
 "status": "FAILED",
 "startedAt": "2026-03-06T04:56:10Z",
 "completedAt": "2026-03-06T04:57:02Z",
 "summary": "Build Integrity failed due to missing environment contract and startup failure.",
 "score": 42,
 "coverageState": "COVERED_FAILED",
 "findings": [
   { "findingId": "finding_101" },
   { "findingId": "finding_102" }
 ],
 "evidence": [
   { "evidenceId": "evidence_201" },
   { "evidenceId": "evidence_202" }
 ],
```

```
 "checks": [
  {
    "checkId": "check_1",
    "checkKey": "required_files",
    "name": "Required Files Check",
    "status": "PASSED",
    "summary": "All required files were present."
  },
  {
    "checkId": "check_2",
    "checkKey": "env_contract",
    "name": "Environment Contract Check",
    "status": "FAILED",
    "summary": "Required environment contract file was missing.",
    "findings": [
      { "findingId": "finding_101" }
    ],
    "evidence": [
      { "evidenceId": "evidence_201" }
    ]
  },
  {
    "checkId": "check_3",
    "checkKey": "startup_viability",
    "name": "Startup Viability Check",
    "status": "FAILED",
    "summary": "Application failed to initialize using the declared startup command.",
    "findings": [
      { "findingId": "finding_102" }
    ],
    "evidence": [
      { "evidenceId": "evidence_202" }
    ]
  }
 ]
}
```

Status meaning

The top-level `status` is the aggregate status of the domain.

A good rule is:

- `PASSED` = all required checks passed
- `FAILED` = one or more required checks failed
- `BLOCKED` = domain could not complete meaningfully
- `PARTIAL` = some checks completed but domain depth was incomplete
- `RUNNING` = domain still executing

Coverage role

The `coverageState` on the Domain Result answers:

Was this domain actually exercised to meaningful depth?

Typical values:

- `COVERED`
- `COVERED_FAILED`
- `PARTIAL`
- `BLOCKED`
- `NOT_TESTED`

This is important because a domain can fail and still count as covered.

Score role

`score` should be optional because:

- MVP may compute simple domain scoring later

- some runs may not use scoring yet
- domain status and findings should still work without score

So this field is useful, but not mandatory for first implementation.

Strong implementation rule

Do not let the Domain Result become just a summary blob.

It should always preserve:

- explicit domain identity
- aggregate status
- check list
- findings refs
- evidence refs
- time boundaries

That is what makes it usable for:

- Run Detail
- Coverage Matrix
- Triage Board
- Verdict Engine

Relationship to other schemas

Domain Result depends on:

- Enums
- Run Manifest

And later it feeds:

- Finding
- Evidence
- Coverage
- Verdict
- History Record

So this is the first major execution-output schema.

Recommended small improvement

Add one more optional field if you want stronger verdict logic later:

blockerDetected?: boolean;

Updated interface fragment:

blockerDetected?: boolean;

That gives the verdict engine a fast signal before full finding resolution, though it is optional because blockers can also be derived from findings.

AVCS Schema 04 — Finding

AVCS Schema 04 — Finding

Recommended location:

shared/avcs/contracts/finding.contract.ts

The **Finding** is the core triage object in AVCS.

It turns raw failures, risks, and policy problems into a structured issue record that operators and the verdict engine can both use.

A finding should answer:

- what went wrong
- where it happened
- how serious it is
- what kind of impact it has
- what evidence supports it
- whether it affects certification

TypeScript Schema

```
import {
 AVCSDomain,
 AVCSFindingSeverity,
 AVCSFindingImpactClass
} from "../enums/avcs.enums";

export interface AVCSFinding {
 findingId: string;

 runId: string;

 domain: AVCSDomain;
```

```
 severity: AVCSFindingSeverity;

 impactClass: AVCSFindingImpactClass;

 title: string;

 summary: string;

 description?: string;

 affectedSurface: AVCSAffectedSurface;

 evidence: AVCSEvidenceRef[];

 certificationEffect: AVCSCertificationEffect;

 remediation?: string;

 metadata?: Record<string, any>;
}
```

Affected Surface

This tells AVCS where the issue lives.

```
export interface AVCSAffectedSurface {
 type: AVCSAffectedSurfaceType;

 key: string;

 label?: string;
}
```

Add this enum:

```
export enum AVCSAffectedSurfaceType {
 BUILD = "BUILD",
 WORKFLOW = "WORKFLOW",
 ROUTE = "ROUTE",
 PAGE = "PAGE",
 COMPONENT = "COMPONENT",
 API = "API",
 CONFIG = "CONFIG",
```

```
 DEPLOYMENT = "DEPLOYMENT",
 DOMAIN = "DOMAIN",
 UNKNOWN = "UNKNOWN"
}
```

Evidence Ref

If you have not centralized it yet, use the same simple ref shape here:

```
export interface AVCSEvidenceRef {
 evidenceId: string;
}
```

Certification Effect

This is critical. Severity alone is not enough.

A medium issue in a critical workflow may matter more than a high issue in a secondary surface.

```
export interface AVCSCertificationEffect {
 blocksRelease: boolean;

 requiresReview: boolean;

 affectsScore: boolean;

 notes?: string;
}
```

Example Finding

```
{
 "findingId": "finding_101",
 "runId": "run_20260306_001",
 "domain": "BUILD_INTEGRITY",
 "severity": "HIGH",
 "impactClass": "DEPLOYMENT",
 "title": "Missing environment contract",
 "summary": "Required environment contract file was not present in the build output.",
```

"description": "The build target does not include the required environment contract definition used by startup and deployment checks.",
"affectedSurface": {
"type": "CONFIG",
"key": "env.contract",
"label": "Environment Contract"
},
"evidence": [
{ "evidenceId": "evidence_201" }
],
"certificationEffect": {
"blocksRelease": true,
"requiresReview": true,
"affectsScore": true,
"notes": "Deployment-readiness path cannot be trusted without a valid environment contract."
},
"remediation": "Add the required environment contract file to the build output and rerun Build Integrity and Deployment Readiness validation."
}

Why this schema matters

The Finding is the object that powers:

- failure triage
- severity sorting
- blocker summaries
- verdict logic
- remediation workflow
- history comparison

Without it, you only have raw check failures.

With it, AVCS can present:

- clean blocker lists

- grouped issue boards
- operator-friendly explanations
- release-impact summaries

Strong design rule

Do not let findings be just log fragments.

A real finding must always have:

- domain
- severity
- impact class
- title/summary
- affected surface
- evidence link
- certification effect

That is the minimum for a usable AVCS issue object.

Recommended enum improvement

Your current `AVCSFindingImpactClass` is useful, but you also need a more direct verdict-oriented label sometimes.

If you want stronger control later, add:

```
export enum AVCSFindingDisposition {
 BLOCKER = "BLOCKER",
 WARNING = "WARNING",
```

```
 OBSERVATION = "OBSERVATION"
}
```

Then add to finding:

disposition?: AVCSFindingDisposition;

This is optional for MVP because `certificationEffect.blocksRelease` already covers the main gating need.

Relationship to other schemas

Finding depends on:

- Enums
- Run Manifest
- Domain Result

And it feeds:

- Triage Board
- Coverage/Verdict explanations
- History persistence
- Release Gate summaries

AVCS Schema 05 — Evidence

AVCS Schema 05 — Evidence

Recommended location:

shared/avcs/contracts/evidence.contract.ts

The **Evidence** object is the proof layer of AVCS.

It supports:

- domain results
- findings
- coverage claims
- verdict reasoning
- history and audit trails

If the Finding is the triage object, Evidence is the thing that makes the finding trustworthy.

TypeScript Schema

```
import {
 AVCSDomain,
 AVCSEvidenceType
} from "../enums/avcs.enums";

export interface AVCSEvidenceItem {
 evidenceId: string;

 runId: string;

 type: AVCSEvidenceType;

 domain?: AVCSDomain;

 label: string;
```

```
 summary: string;

 storageRef: string;

 source: AVCSEvidenceSource;

 linkedFindingIds?: string[];

 linkedCheckIds?: string[];

 createdAt: string;

 metadata?: Record<string, any>;
}
```

Evidence Source

This identifies where the evidence came from.

```
export interface AVCSEvidenceSource {
 sourceType: AVCSEvidenceSourceType;

 sourceKey: string;

 sourceLabel?: string;
}
```

Add this enum:

```
export enum AVCSEvidenceSourceType {
 DOMAIN_EVALUATOR = "DOMAIN_EVALUATOR",
 CHECK = "CHECK",
 RUNNER = "RUNNER",
 SYSTEM = "SYSTEM",
 USER = "USER"
}
```

Example Evidence Items

Log evidence

```
{
```

```
 "evidenceId": "evidence_201",
 "runId": "run_20260306_001",
 "type": "LOG",
 "domain": "BUILD_INTEGRITY",
 "label": "Environment contract check log",
 "summary": "Log output from the environment contract validation step.",
 "storageRef": "/avcs/runs/run_20260306_001/evidence/env_contract_check.log",
 "source": {
   "sourceType": "CHECK",
   "sourceKey": "env_contract",
   "sourceLabel": "Environment Contract Check"
 },
 "linkedFindingIds": ["finding_101"],
 "linkedCheckIds": ["check_2"],
 "createdAt": "2026-03-06T04:56:28Z"
}
```


Screenshot evidence


```
{
 "evidenceId": "evidence_305",
 "runId": "run_20260306_001",
 "type": "SCREENSHOT",
 "domain": "FUNCTIONAL_CORRECTNESS",
 "label": "Workflow failure screenshot",
 "summary": "Captured UI state after submit failure in the project creation workflow.",
 "storageRef": "/avcs/runs/run_20260306_001/evidence/project_create_failure.png",
 "source": {
   "sourceType": "CHECK",
   "sourceKey": "project_create_workflow",
   "sourceLabel": "Project Create Workflow"
 },
 "linkedFindingIds": ["finding_221"],
 "linkedCheckIds": ["check_14"],
 "createdAt": "2026-03-06T05:02:10Z"
}
```


Metric evidence


```
{
 "evidenceId": "evidence_412",
 "runId": "run_20260306_001",
 "type": "METRIC",
 "domain": "PERFORMANCE_STABILITY",
 "label": "Dashboard load timing",
```

"summary": "Measured initial dashboard load time exceeded target threshold.",
"storageRef": "/avcs/runs/run_20260306_001/evidence/dashboard_load_metric.json",
"source": {
"sourceType": "CHECK",
"sourceKey": "dashboard_load_time",
"sourceLabel": "Dashboard Load Time Check"
},
"linkedFindingIds": ["finding_341"],
"linkedCheckIds": ["check_27"],
"createdAt": "2026-03-06T05:08:43Z"
}

Why this schema matters

AVCS cannot make trusted claims without evidence.

This schema gives the system a standard way to represent proof across many artifact types:

- logs
- screenshots
- metrics
- reports
- traces
- summary artifacts

That lets you build:

- Evidence Explorer
- finding drill-down
- verdict reasoning panels
- append-only history records

Strong design rule

Do not store raw evidence inline inside findings or domain results.

Always reference it by `evidenceId`.

That keeps the system:

- normalized
- queryable
- easier to persist
- easier to audit
- easier to display in multiple views

So the pattern should be:

DomainResult -> EvidenceRef[]
Finding -> EvidenceRef[]
Verdict -> EvidenceRef[] or summary references

Not giant embedded blobs.

Recommended enum expansion

Your current `AVCSEvidenceType` is good, but for AVCS you should probably expand it slightly now so you do not outgrow it fast.

Use this instead:

```
export enum AVCSEvidenceType {
 LOG = "LOG",
 SCREENSHOT = "SCREENSHOT",
 METRIC = "METRIC",
 REPORT = "REPORT",
 TRACE = "TRACE",
 ARTIFACT = "ARTIFACT",
 SUMMARY = "SUMMARY",
```

```
 JSON = "JSON",
 CONSOLE = "CONSOLE"
}
```

Why:

- `JSON` is useful for captured structured outputs
- `CONSOLE` is useful because console evidence often becomes a first-class thing in frontend and runtime testing

---

Recommended small improvement

Add an optional integrity/finalization field now so history and audit become easier later:

finalized?: boolean;

Updated interface fragment:

finalized?: boolean;

That helps you distinguish:

- evidence collected during active execution
- evidence frozen as part of finalized run history

Relationship to other schemas

Evidence depends on:

- Enums
- Run Manifest

And it is referenced by:

- Domain Result
- Finding
- Coverage
- Verdict
- History Record

So this is one of the most cross-cutting schemas in the system.

AVCS Schema 06 — Coverage

AVCS Schema 06 — Coverage

Recommended location:

shared/avcs/contracts/coverage.contract.ts

The **Coverage** object defines what AVCS actually validated during a run.

This schema is what prevents the system from saying “fully tested” when only a few checks ran.

It should answer:

- what surface was in scope
- what kind of surface it was
- what domain it belonged to
- whether it was actually exercised
- what the result state was
- what evidence supports that claim

TypeScript Schema

```
import {
 AVCSDomain,
 AVCSCoverageState
} from "../enums/avcs.enums";

export interface AVCSCoverageItem {
 coverageId: string;

 runId: string;

 domain: AVCSDomain;

 scopeType: AVCSCoverageScopeType;
```

```
 scopeKey: string;

 scopeLabel: string;

 state: AVCSCoverageState;

 depth: AVCSCoverageDepth;

 required: boolean;

 linkedCheckIds?: string[];

 linkedFindingIds?: string[];

 linkedEvidenceIds?: string[];

 summary: string;

 metadata?: Record<string, any>;
}
```

Coverage Scope Type

This tells AVCS what kind of thing was covered.

```
export enum AVCSCoverageScopeType {
 DOMAIN = "DOMAIN",
 WORKFLOW = "WORKFLOW",
 ROUTE = "ROUTE",
 PAGE = "PAGE",
 COMPONENT = "COMPONENT",
 API = "API",
 CONFIG = "CONFIG",
 DEPLOYMENT_STEP = "DEPLOYMENT_STEP",
 REQUIREMENT = "REQUIREMENT",
 RISK_AREA = "RISK_AREA"
}
```

Coverage Depth

This tells AVCS how deeply the surface was exercised.

```
export enum AVCSCoverageDepth {
 SHALLOW = "SHALLOW",
 STANDARD = "STANDARD",
 DEEP = "DEEP"
}
```

This is important because:

- route touched once is not the same as
- workflow fully executed with error/recovery coverage

Example Coverage Items

Workflow coverage


```
{
 "coverageId": "coverage_101",
 "runId": "run_20260306_001",
 "domain": "FUNCTIONAL_CORRECTNESS",
 "scopeType": "WORKFLOW",
 "scopeKey": "project_create_workflow",
 "scopeLabel": "Project Create Workflow",
 "state": "COVERED_FAILED",
 "depth": "STANDARD",
 "required": true,
 "linkedCheckIds": ["check_14"],
 "linkedFindingIds": ["finding_221"],
 "linkedEvidenceIds": ["evidence_305"],
 "summary": "Workflow was executed through the primary submit path but failed during final
creation step."
}
```


Domain coverage


```
{
 "coverageId": "coverage_201",
 "runId": "run_20260306_001",
 "domain": "SECURITY_POSTURE",
 "scopeType": "DOMAIN",
 "scopeKey": "security_posture",
 "scopeLabel": "Security Posture Domain",
```

"state": "COVERED",
"depth": "STANDARD",
"required": true,
"linkedCheckIds": ["check_31", "check_32", "check_33"],
"linkedEvidenceIds": ["evidence_501", "evidence_502"],
"summary": "Required security checks completed successfully for the selected run scope."
}

Required surface not tested

{
"coverageId": "coverage_301",
"runId": "run_20260306_001",
"domain": "DEPLOYMENT_READINESS",
"scopeType": "DEPLOYMENT_STEP",
"scopeKey": "readiness_endpoint",
"scopeLabel": "Readiness Endpoint Check",
"state": "NOT_TESTED",
"depth": "SHALLOW",
"required": true,
"summary": "Required readiness endpoint validation did not execute in this run."
}

Why this schema matters

This is the schema that powers:

- Coverage Matrix
- release sufficiency checks
- “not tested” visibility
- required-scope enforcement
- comparison against previous runs

Without it, AVCS only knows:

- what failed

But not:

- what was never exercised

That is a huge difference in a certification system.

Strong design rule

Coverage must stay separate from results.

Do not confuse:

- **coverage** = was it exercised?
  with
- **result** = what happened when it was exercised?

That is why this schema uses states like:

- `COVERED`
- `COVERED_FAILED`
- `PARTIAL`
- `BLOCKED`
- `NOT_TESTED`

A failed workflow still counts as covered.
An untouched workflow does not.

Recommended small improvement

Add one optional field for policy/gate logic later:

gateRelevant?: boolean;

Updated fragment:

gateRelevant?: boolean;

This helps distinguish:

- coverage that matters directly to release gating
  from
- informational coverage that is useful but not gate-critical

---

Relationship to other schemas

Coverage depends on:

- Enums
- Run Manifest

And it references:

- check IDs from Domain Result
- finding IDs from Finding
- evidence IDs from Evidence

It feeds:

- Coverage Matrix UI
- policy/gating
- verdict engine
- history/comparison

Recommended enum update

Since Coverage now adds two enums, your central enums file should include:

```
export enum AVCSCoverageScopeType {
 DOMAIN = "DOMAIN",
 WORKFLOW = "WORKFLOW",
 ROUTE = "ROUTE",
 PAGE = "PAGE",
 COMPONENT = "COMPONENT",
 API = "API",
 CONFIG = "CONFIG",
 DEPLOYMENT_STEP = "DEPLOYMENT_STEP",
 REQUIREMENT = "REQUIREMENT",
 RISK_AREA = "RISK_AREA"
}

export enum AVCSCoverageDepth {
 SHALLOW = "SHALLOW",
 STANDARD = "STANDARD",
 DEEP = "DEEP"
}
```

AVCS Schema 07 — Verdict

AVCS Schema 07 — Verdict

Recommended location:

shared/avcs/contracts/verdict.contract.ts

The **Verdict** is the final certification decision object for a run.

It is the schema that answers:

- did the build pass certification
- is release blocked
- were warnings present
- was coverage sufficient
- was evidence complete
- what reasoning drove the final decision

This is the object the **Release Gate Panel** should center on.

TypeScript Schema

```
import { AVCSVerdictState } from "../enums/avcs.enums";

export interface AVCSVerdict {
 verdictId: string;

 runId: string;

 state: AVCSVerdictState;

 blockerCount: number;

 warningCount: number;

 coverageSufficient: boolean;
```

```
 evidenceComplete: boolean;

 score?: AVCSVerdictScoreSummary;

 linkedFindingIds: string[];

 linkedEvidenceIds?: string[];

 reasoning: string;

 decidedAt: string;

 metadata?: Record<string, any>;
}
```

Score Summary

Keep score nested so the verdict can exist cleanly with or without scoring.

```
export interface AVCSVerdictScoreSummary {
 overallScore: number;

 rating: AVCSVerdictRating;

 domainScores?: AVCSVerdictDomainScore[];
}
```

Verdict Rating

This gives you a readable readiness band.

```
export enum AVCSVerdictRating {
 STRONG = "STRONG",
 ACCEPTABLE = "ACCEPTABLE",
 FRAGILE = "FRAGILE",
 INSUFFICIENT = "INSUFFICIENT"
}
```

Domain Score Entry

```
export interface AVCSVerdictDomainScore {
 domain: string;

 score: number;
}
```

You can later tighten `domain: string` to `domain: AVCSDomain` if you want domain typing here too. I would recommend that.

Better version:

```
import { AVCSDomain } from "../enums/avcs.enums";

export interface AVCSVerdictDomainScore {
 domain: AVCSDomain;

 score: number;
}
```

Recommended upgraded Verdict schema

This is the version I would actually use:

```
import {
 AVCSVerdictState,
 AVCSDomain
} from "../enums/avcs.enums";

export interface AVCSVerdict {
 verdictId: string;

 runId: string;

 state: AVCSVerdictState;

 blockerCount: number;

 warningCount: number;

 coverageSufficient: boolean;

 evidenceComplete: boolean;
```

```
 score?: AVCSVerdictScoreSummary;

 linkedFindingIds: string[];

 linkedEvidenceIds?: string[];

 reasoning: string;

 decidedAt: string;

 metadata?: Record<string, any>;
}

export interface AVCSVerdictScoreSummary {
 overallScore: number;

 rating: AVCSVerdictRating;

 domainScores?: AVCSVerdictDomainScore[];
}

export interface AVCSVerdictDomainScore {
 domain: AVCSDomain;

 score: number;
}

export enum AVCSVerdictRating {
 STRONG = "STRONG",
 ACCEPTABLE = "ACCEPTABLE",
 FRAGILE = "FRAGILE",
 INSUFFICIENT = "INSUFFICIENT"
}
```

Example Verdict

```
{
 "verdictId": "verdict_001",
 "runId": "run_20260306_001",
 "state": "FAIL",
 "blockerCount": 2,
 "warningCount": 1,
 "coverageSufficient": true,
```

"evidenceComplete": true,
"score": {
  "overallScore": 58,
  "rating": "INSUFFICIENT",
  "domainScores": [
    { "domain": "BUILD_INTEGRITY", "score": 42 },
    { "domain": "FUNCTIONAL_CORRECTNESS", "score": 61 },
    { "domain": "SECURITY_POSTURE", "score": 74 },
    { "domain": "PERFORMANCE_STABILITY", "score": 66 },
    { "domain": "DEPLOYMENT_READINESS", "score": 39 }
  ]
},
"linkedFindingIds": [
  "finding_101",
  "finding_102",
  "finding_221"
],
"linkedEvidenceIds": [
  "evidence_201",
  "evidence_202",
  "evidence_305"
],
"reasoning": "Run failed because Build Integrity and Deployment Readiness produced blocker-class findings. Coverage and evidence were sufficient, but release cannot proceed while required deployment conditions remain unresolved.",
"decidedAt": "2026-03-06T05:12:45Z"
}

Why this schema matters

This is the schema that feeds:

- Release Gate Panel
- Command Center latest certification status
- build readiness summaries
- deployment gating integrations
- historical verdict comparison

- score trend views

It is the single clean object that expresses the final result of the run.

Strong design rule

Do not make the verdict too “smart” by stuffing the whole world into it.

The verdict should stay a **decision summary object**, not a duplicate of:

- all findings
- all evidence
- all coverage items
- all domain results

That is why it should reference those things rather than embed them fully.

The verdict should answer:

- what was the decision
- why was it made
- what major issues drove it
- how strong was the run

That’s it.

Important distinction

The verdict should not try to replace policy state entirely.

For example:

- `coverageSufficient`
- `evidenceComplete`

are good summary booleans.

But the full policy logic that produced them belongs in:

- policy resolver
- gate evaluator
- coverage assembler
- evidence completeness evaluator

The verdict is the output, not the whole decision engine.

Recommended small improvement

Add an optional override flag now so you do not have to reshape the schema later.

overrideRequired?: boolean;

Updated fragment:

overrideRequired?: boolean;

This will be useful when you fully implement:

- `CONDITIONAL_PASS`
- formal override workflows
- pre-deployment gate handling

Recommended stronger version with override flag

```
import {
 AVCSVerdictState,
 AVCSDomain
} from "../enums/avcs.enums";

export interface AVCSVerdict {
 verdictId: string;

 runId: string;

 state: AVCSVerdictState;

 blockerCount: number;

 warningCount: number;

 coverageSufficient: boolean;

 evidenceComplete: boolean;

 overrideRequired?: boolean;

 score?: AVCSVerdictScoreSummary;

 linkedFindingIds: string[];

 linkedEvidenceIds?: string[];

 reasoning: string;

 decidedAt: string;

 metadata?: Record<string, any>;
}

export interface AVCSVerdictScoreSummary {
 overallScore: number;

 rating: AVCSVerdictRating;

 domainScores?: AVCSVerdictDomainScore[];
}

export interface AVCSVerdictDomainScore {
```

```
 domain: AVCSDomain;

 score: number;
}

export enum AVCSVerdictRating {
 STRONG = "STRONG",
 ACCEPTABLE = "ACCEPTABLE",
 FRAGILE = "FRAGILE",
 INSUFFICIENT = "INSUFFICIENT"
}
```

Relationship to other schemas

Verdict depends on:

- Enums
- Findings
- Evidence
- Coverage
- Domain Results

And it feeds:

- Release Gate Panel
- Command Center
- history records
- downstream release gating integrations

So this is the final decision object, but not the historical record yet.

AVCS Schema 08 — History Record

AVCS Schema 08 — History Record

Recommended location:

shared/avcs/contracts/history-record.contract.ts

The **History Record** is the append-only audit object for AVCS.

It preserves what happened over time during and after a run:

- run creation
- state transitions
- domain completion
- finding creation
- verdict finalization
- rerun linkage
- override or review events later

This is the schema that gives AVCS durable memory and auditability.

TypeScript Schema

```
import { AVCSRunStatus, AVCSVerdictState } from "../enums/avcs.enums";

export interface AVCSHistoryRecord {
 historyId: string;

 runId: string;

 eventType: AVCSHistoryEventType;

 timestamp: string;

 actor?: AVCSHistoryActor;
```

```
 summary: string;

 details?: Record<string, any>;

 relatedObject?: AVCSHistoryRelatedObject;

 metadata?: Record<string, any>;
}
```

History Event Type

This defines what kind of historical event occurred.

```
export enum AVCSHistoryEventType {
 RUN_CREATED = "RUN_CREATED",
 RUN_STATUS_CHANGED = "RUN_STATUS_CHANGED",
 DOMAIN_STARTED = "DOMAIN_STARTED",
 DOMAIN_COMPLETED = "DOMAIN_COMPLETED",
 FINDING_CREATED = "FINDING_CREATED",
 EVIDENCE_REGISTERED = "EVIDENCE_REGISTERED",
 COVERAGE_ASSEMBLED = "COVERAGE_ASSEMBLED",
 VERDICT_FINALIZED = "VERDICT_FINALIZED",
 RERUN_CREATED = "RERUN_CREATED",
 OVERRIDE_APPLIED = "OVERRIDE_APPLIED",
 NOTE_ADDED = "NOTE_ADDED"
}
```

History Actor

This records who or what caused the event.

```
export interface AVCSHistoryActor {
 actorId: string;

 actorType: string;

 name?: string;
}
```

You can later tighten `actorType` to `AVCSActorType` if you want full enum reuse. I would recommend doing that.

Better version:

```
import { AVCSActorType } from "../enums/avcs.enums";

export interface AVCSHistoryActor {
 actorId: string;

 actorType: AVCSActorType;

 name?: string;
}
```

Related Object

This links the history event to the main thing it affected.

```
export interface AVCSHistoryRelatedObject {
 objectType: AVCSHistoryObjectType;

 objectId: string;
}
```

Add this enum:

```
export enum AVCSHistoryObjectType {
 RUN = "RUN",
 DOMAIN_RESULT = "DOMAIN_RESULT",
 FINDING = "FINDING",
 EVIDENCE = "EVIDENCE",
 COVERAGE = "COVERAGE",
 VERDICT = "VERDICT"
}
```

Recommended stronger typed version

```
import { AVCSActorType } from "../enums/avcs.enums";

export interface AVCSHistoryActor {
 actorId: string;
```

```
 actorType: AVCSActorType;

 name?: string;
}
```

Example History Records

Run created


```
{
 "historyId": "history_001",
 "runId": "run_20260306_001",
 "eventType": "RUN_CREATED",
 "timestamp": "2026-03-06T04:55:11Z",
 "actor": {
   "actorId": "user_1",
   "actorType": "USER",
   "name": "Operator"
 },
 "summary": "Certification run was created.",
 "relatedObject": {
   "objectType": "RUN",
   "objectId": "run_20260306_001"
 }
}
```


Domain completed


```
{
 "historyId": "history_014",
 "runId": "run_20260306_001",
 "eventType": "DOMAIN_COMPLETED",
 "timestamp": "2026-03-06T04:57:02Z",
 "summary": "Build Integrity domain completed with FAILED status.",
 "details": {
   "domain": "BUILD_INTEGRITY",
   "status": "FAILED",
   "blockerDetected": true
 },
 "relatedObject": {
   "objectType": "DOMAIN_RESULT",
   "objectId": "domain_result_001"
 }
```

```
}
```

Finding created

```
{
 "historyId": "history_021",
 "runId": "run_20260306_001",
 "eventType": "FINDING_CREATED",
 "timestamp": "2026-03-06T04:56:29Z",
 "summary": "High severity finding created for missing environment contract.",
 "details": {
   "findingId": "finding_101",
   "severity": "HIGH",
   "blocksRelease": true
 },
 "relatedObject": {
   "objectType": "FINDING",
   "objectId": "finding_101"
 }
}
```

Verdict finalized

```
{
 "historyId": "history_055",
 "runId": "run_20260306_001",
 "eventType": "VERDICT_FINALIZED",
 "timestamp": "2026-03-06T05:12:45Z",
 "summary": "Certification verdict finalized as FAIL.",
 "details": {
   "verdictState": "FAIL",
   "blockerCount": 2,
   "warningCount": 1
 },
 "relatedObject": {
   "objectType": "VERDICT",
   "objectId": "verdict_001"
 }
}
```

Why this schema matters

Without History Record, AVCS can tell you only the current result.

With it, AVCS can tell you:

- what happened first
- what changed during execution
- when blockers appeared
- when findings were created
- when verdict was finalized
- what rerun followed this run later

This is what makes:

- audit trails
- timelines
- rerun lineage
- override visibility
- append-only governance

actually possible.

Strong design rule

Do not use history records as mutable snapshots.

A history record should always be:

- append-only
- event-shaped
- timestamped

- linked to a run
- linked to a related object when possible

That means if a run changes state from `RUNNING` to `FAILED`, you do not edit the old record.

You append a new event.

Recommended important improvement

You should add one specialized event payload for run status changes instead of leaving it vague in `details`.

That gives stronger typing.

Add this interface

```
export interface AVCSRunStatusChangeDetails {
 fromStatus?: AVCSRunStatus;

 toStatus: AVCSRunStatus;
}
```

And verdict details

```
export interface AVCSVerdictFinalizedDetails {
 verdictState: AVCSVerdictState;

 blockerCount: number;

 warningCount: number;
}
```

You can still keep `details?: Record<string, any>` for flexibility, but these typed shapes are worth using in service code.

Recommended upgraded full version

```
import {
 AVCSActorType,
```

```
 AVCSRunStatus,
 AVCSVerdictState
} from "../enums/avcs.enums";

export interface AVCSHistoryRecord {
 historyId: string;

 runId: string;

 eventType: AVCSHistoryEventType;

 timestamp: string;

 actor?: AVCSHistoryActor;

 summary: string;

 details?: Record<string, any>;

 relatedObject?: AVCSHistoryRelatedObject;

 metadata?: Record<string, any>;
}

export enum AVCSHistoryEventType {
 RUN_CREATED = "RUN_CREATED",
 RUN_STATUS_CHANGED = "RUN_STATUS_CHANGED",
 DOMAIN_STARTED = "DOMAIN_STARTED",
 DOMAIN_COMPLETED = "DOMAIN_COMPLETED",
 FINDING_CREATED = "FINDING_CREATED",
 EVIDENCE_REGISTERED = "EVIDENCE_REGISTERED",
 COVERAGE_ASSEMBLED = "COVERAGE_ASSEMBLED",
 VERDICT_FINALIZED = "VERDICT_FINALIZED",
 RERUN_CREATED = "RERUN_CREATED",
 OVERRIDE_APPLIED = "OVERRIDE_APPLIED",
 NOTE_ADDED = "NOTE_ADDED"
}

export interface AVCSHistoryActor {
 actorId: string;

 actorType: AVCSActorType;

 name?: string;
```

```
}

export interface AVCSHistoryRelatedObject {
 objectType: AVCSHistoryObjectType;

 objectId: string;
}

export enum AVCSHistoryObjectType {
 RUN = "RUN",
 DOMAIN_RESULT = "DOMAIN_RESULT",
 FINDING = "FINDING",
 EVIDENCE = "EVIDENCE",
 COVERAGE = "COVERAGE",
 VERDICT = "VERDICT"
}

export interface AVCSRunStatusChangeDetails {
 fromStatus?: AVCSRunStatus;

 toStatus: AVCSRunStatus;
}

export interface AVCSVerdictFinalizedDetails {
 verdictState: AVCSVerdictState;

 blockerCount: number;

 warningCount: number;
}
```

Relationship to other schemas

History Record depends on:

- Enums
- Run Manifest
- Verdict
- Findings

- Domain Results
- Evidence
- Coverage

And it feeds:

- History view
- audit trail
- rerun lineage
- override visibility
- governance export

This is the final core schema in the base AVCS chain.

AVCS Type Script

AVCS Final Schema Pack

Recommended structure

```
shared/avcs/
 enums/
   avcs.enums.ts
 types/
   avcs.shared.ts
 contracts/
   run-manifest.contract.ts
   domain-result.contract.ts
   finding.contract.ts
   evidence.contract.ts
   coverage.contract.ts
   verdict.contract.ts
   history-record.contract.ts
```

---

1. `shared/avcs/enums/avcs.enums.ts`

```
export enum AVCSRunType {
 SMOKE = "SMOKE",
 FUNCTIONAL = "FUNCTIONAL",
 SECURITY = "SECURITY",
 PERFORMANCE = "PERFORMANCE",
 FULL_CERTIFICATION = "FULL_CERTIFICATION",
 REGRESSION = "REGRESSION",
 PRE_DEPLOYMENT_GATE = "PRE_DEPLOYMENT_GATE",
}

export enum AVCSDomain {
 BUILD_INTEGRITY = "BUILD_INTEGRITY",
 FUNCTIONAL_CORRECTNESS = "FUNCTIONAL_CORRECTNESS",
 SECURITY_POSTURE = "SECURITY_POSTURE",
 PERFORMANCE_STABILITY = "PERFORMANCE_STABILITY",
 UI_QUALITY = "UI_QUALITY",
 UX_WORKFLOW = "UX_WORKFLOW",
 ACCESSIBILITY = "ACCESSIBILITY",
 DEPLOYMENT_READINESS = "DEPLOYMENT_READINESS",
```

```
 ENTERPRISE_READINESS = "ENTERPRISE_READINESS",
}

export enum AVCSRunStatus {
 CREATED = "CREATED",
 PREFLIGHT = "PREFLIGHT",
 RUNNING = "RUNNING",
 FINALIZING = "FINALIZING",
 COMPLETED = "COMPLETED",
 FAILED = "FAILED",
 BLOCKED = "BLOCKED",
}

export enum AVCSDomainStatus {
 NOT_STARTED = "NOT_STARTED",
 RUNNING = "RUNNING",
 PASSED = "PASSED",
 FAILED = "FAILED",
 BLOCKED = "BLOCKED",
 PARTIAL = "PARTIAL",
}

export enum AVCSDomainCheckStatus {
 NOT_STARTED = "NOT_STARTED",
 RUNNING = "RUNNING",
 PASSED = "PASSED",
 FAILED = "FAILED",
 BLOCKED = "BLOCKED",
 PARTIAL = "PARTIAL",
 SKIPPED = "SKIPPED",
}

export enum AVCSFindingSeverity {
 INFO = "INFO",
 LOW = "LOW",
 MEDIUM = "MEDIUM",
 HIGH = "HIGH",
 CRITICAL = "CRITICAL",
}

export enum AVCSFindingImpactClass {
 FUNCTIONAL = "FUNCTIONAL",
 SECURITY = "SECURITY",
 PERFORMANCE = "PERFORMANCE",
```

```
 RELIABILITY = "RELIABILITY",
 DEPLOYMENT = "DEPLOYMENT",
 USABILITY = "USABILITY",
 COMPLIANCE = "COMPLIANCE",
}

export enum AVCSFindingDisposition {
 BLOCKER = "BLOCKER",
 WARNING = "WARNING",
 OBSERVATION = "OBSERVATION",
}

export enum AVCSCoverageState {
 COVERED = "COVERED",
 COVERED_FAILED = "COVERED_FAILED",
 PARTIAL = "PARTIAL",
 BLOCKED = "BLOCKED",
 NOT_TESTED = "NOT_TESTED",
}

export enum AVCSCoverageScopeType {
 DOMAIN = "DOMAIN",
 WORKFLOW = "WORKFLOW",
 ROUTE = "ROUTE",
 PAGE = "PAGE",
 COMPONENT = "COMPONENT",
 API = "API",
 CONFIG = "CONFIG",
 DEPLOYMENT_STEP = "DEPLOYMENT_STEP",
 REQUIREMENT = "REQUIREMENT",
 RISK_AREA = "RISK_AREA",
}

export enum AVCSCoverageDepth {
 SHALLOW = "SHALLOW",
 STANDARD = "STANDARD",
 DEEP = "DEEP",
}

export enum AVCSVerdictState {
 PASS = "PASS",
 PASS_WITH_WARNINGS = "PASS_WITH_WARNINGS",
 CONDITIONAL_PASS = "CONDITIONAL_PASS",
 FAIL = "FAIL",
```

```
 BLOCKED = "BLOCKED",
}

export enum AVCSVerdictRating {
 STRONG = "STRONG",
 ACCEPTABLE = "ACCEPTABLE",
 FRAGILE = "FRAGILE",
 INSUFFICIENT = "INSUFFICIENT",
}

export enum AVCSEvidenceType {
 LOG = "LOG",
 SCREENSHOT = "SCREENSHOT",
 METRIC = "METRIC",
 REPORT = "REPORT",
 TRACE = "TRACE",
 ARTIFACT = "ARTIFACT",
 SUMMARY = "SUMMARY",
 JSON = "JSON",
 CONSOLE = "CONSOLE",
}

export enum AVCSEvidenceSourceType {
 DOMAIN_EVALUATOR = "DOMAIN_EVALUATOR",
 CHECK = "CHECK",
 RUNNER = "RUNNER",
 SYSTEM = "SYSTEM",
 USER = "USER",
}

export enum AVCSActorType {
 USER = "USER",
 SYSTEM = "SYSTEM",
 AUTOMATION = "AUTOMATION",
}

export enum AVCSAffectedSurfaceType {
 BUILD = "BUILD",
 WORKFLOW = "WORKFLOW",
 ROUTE = "ROUTE",
 PAGE = "PAGE",
 COMPONENT = "COMPONENT",
 API = "API",
 CONFIG = "CONFIG",
```

```
 DEPLOYMENT = "DEPLOYMENT",
 DOMAIN = "DOMAIN",
 UNKNOWN = "UNKNOWN",
}

export enum AVCSHistoryEventType {
 RUN_CREATED = "RUN_CREATED",
 RUN_STATUS_CHANGED = "RUN_STATUS_CHANGED",
 DOMAIN_STARTED = "DOMAIN_STARTED",
 DOMAIN_COMPLETED = "DOMAIN_COMPLETED",
 FINDING_CREATED = "FINDING_CREATED",
 EVIDENCE_REGISTERED = "EVIDENCE_REGISTERED",
 COVERAGE_ASSEMBLED = "COVERAGE_ASSEMBLED",
 VERDICT_FINALIZED = "VERDICT_FINALIZED",
 RERUN_CREATED = "RERUN_CREATED",
 OVERRIDE_APPLIED = "OVERRIDE_APPLIED",
 NOTE_ADDED = "NOTE_ADDED",
}

export enum AVCSHistoryObjectType {
 RUN = "RUN",
 DOMAIN_RESULT = "DOMAIN_RESULT",
 FINDING = "FINDING",
 EVIDENCE = "EVIDENCE",
 COVERAGE = "COVERAGE",
 VERDICT = "VERDICT",
}
```

---

2. `shared/avcs/types/avcs.shared.ts`

```
import {
 AVCSActorType,
 AVCSAffectedSurfaceType,
 AVCSEvidenceSourceType,
} from "../enums/avcs.enums";

export type AVCSMetadata = Record<string, unknown>;

export interface AVCSFindingRef {
 findingId: string;
}
```

```
export interface AVCSEvidenceRef {
 evidenceId: string;
}

export interface AVCSRunActor {
 actorId: string;
 actorType: AVCSActorType;
 name?: string;
}

export interface AVCSHistoryActor {
 actorId: string;
 actorType: AVCSActorType;
 name?: string;
}

export interface AVCSAffectedSurface {
 type: AVCSAffectedSurfaceType;
 key: string;
 label?: string;
}

export interface AVCSEvidenceSource {
 sourceType: AVCSEvidenceSourceType;
 sourceKey: string;
 sourceLabel?: string;
}
```

---

3. `shared/avcs/contracts/run-manifest.contract.ts`

```
import {
 AVCSDomain,
 AVCSRunStatus,
 AVCSRunType,
} from "../enums/avcs.enums";
import { AVCSMetadata, AVCSRunActor } from "../types/avcs.shared";
```

```
export interface AVCSRunManifest {
 runId: string;
 target: AVCSRunTarget;
 runType: AVCSRunType;
 includedDomains: AVCSDomain[];
 actor: AVCSRunActor;
 policySnapshotId: string;
 createdAt: string;
 status: AVCSRunStatus;
 metadata?: AVCSMetadata;
}

export interface AVCSRunTarget {
 buildId: string;
 buildVersion: string;
 artifactLocation: string;
 buildClass?: string;
 generatedAt?: string;
 metadata?: AVCSMetadata;
}
```

---

4. shared/avcs/contracts/domain-result.contract.ts

```
import {
 AVCSDomain,
 AVCSCoverageState,
 AVCSDomainCheckStatus,
 AVCSDomainStatus,
} from "../enums/avcs.enums";
import {
 AVCSEvidenceRef,
 AVCSFindingRef,
 AVCSMetadata,
} from "../types/avcs.shared";

export interface AVCSDomainResult {
 domainResultId: string;
```

```
 runId: string;
 domain: AVCSDomain;
 status: AVCSDomainStatus;
 startedAt?: string;
 completedAt?: string;
 summary: string;
 score?: number;
 coverageState: AVCSCoverageState;
 blockerDetected?: boolean;
 findings: AVCSFindingRef[];
 evidence: AVCSEvidenceRef[];
 checks: AVCSDomainCheckResult[];
 metadata?: AVCSMetadata;
}

export interface AVCSDomainCheckResult {
 checkId: string;
 checkKey: string;
 name: string;
 status: AVCSDomainCheckStatus;
 summary: string;
 startedAt?: string;
 completedAt?: string;
 findings?: AVCSFindingRef[];
 evidence?: AVCSEvidenceRef[];
 metadata?: AVCSMetadata;
}
```

---

5. `shared/avcs/contracts/finding.contract.ts`

```
import {
 AVCSDomain,
 AVCSFindingDisposition,
 AVCSFindingImpactClass,
 AVCSFindingSeverity,
} from "../enums/avcs.enums";
import {
```

```
 AVCSAffectedSurface,
 AVCSEvidenceRef,
 AVCSMetadata,
} from "../types/avcs.shared";

export interface AVCSFinding {
 findingId: string;
 runId: string;
 domain: AVCSDomain;
 severity: AVCSFindingSeverity;
 impactClass: AVCSFindingImpactClass;
 disposition?: AVCSFindingDisposition;
 title: string;
 summary: string;
 description?: string;
 affectedSurface: AVCSAffectedSurface;
 evidence: AVCSEvidenceRef[];
 certificationEffect: AVCSCertificationEffect;
 remediation?: string;
 metadata?: AVCSMetadata;
}

export interface AVCSCertificationEffect {
 blocksRelease: boolean;
 requiresReview: boolean;
 affectsScore: boolean;
 notes?: string;
}
```

---

6. shared/avcs/contracts/evidence.contract.ts

```
import {
 AVCSDomain,
 AVCSEvidenceType,
} from "../enums/avcs.enums";
import {
 AVCSEvidenceSource,
```

```
 AVCSMetadata,
} from "../types/avcs.shared";

export interface AVCSEvidenceItem {
 evidenceId: string;
 runId: string;
 type: AVCSEvidenceType;
 domain?: AVCSDomain;
 label: string;
 summary: string;
 storageRef: string;
 source: AVCSEvidenceSource;
 linkedFindingIds?: string[];
 linkedCheckIds?: string[];
 createdAt: string;
 finalized?: boolean;
 metadata?: AVCSMetadata;
}
```

---

7. shared/avcs/contracts/coverage.contract.ts

```
import {
 AVCSCoverageDepth,
 AVCSCoverageScopeType,
 AVCSCoverageState,
 AVCSDomain,
} from "../enums/avcs.enums";
import { AVCSMetadata } from "../types/avcs.shared";

export interface AVCSCoverageItem {
 coverageId: string;
 runId: string;
 domain: AVCSDomain;
 scopeType: AVCSCoverageScopeType;
 scopeKey: string;
 scopeLabel: string;
 state: AVCSCoverageState;
```

```
 depth: AVCSCoverageDepth;
 required: boolean;
 gateRelevant?: boolean;
 linkedCheckIds?: string[];
 linkedFindingIds?: string[];
 linkedEvidenceIds?: string[];
 summary: string;
 metadata?: AVCSMetadata;
}
```

---

8. shared/avcs/contracts/verdict.contract.ts

```
import {
 AVCSDomain,
 AVCSVerdictRating,
 AVCSVerdictState,
} from "../enums/avcs.enums";
import { AVCSMetadata } from "../types/avcs.shared";

export interface AVCSVerdict {
 verdictId: string;
 runId: string;
 state: AVCSVerdictState;
 blockerCount: number;
 warningCount: number;
 coverageSufficient: boolean;
 evidenceComplete: boolean;
 overrideRequired?: boolean;
 score?: AVCSVerdictScoreSummary;
 linkedFindingIds: string[];
 linkedEvidenceIds?: string[];
 reasoning: string;
 decidedAt: string;
 metadata?: AVCSMetadata;
}

export interface AVCSVerdictScoreSummary {
```

```
 overallScore: number;
 rating: AVCSVerdictRating;
 domainScores?: AVCSVerdictDomainScore[];
}

export interface AVCSVerdictDomainScore {
 domain: AVCSDomain;
 score: number;
}
```

---

9. shared/avcs/contracts/history-record.contract.ts

```
import {
 AVCSHistoryEventType,
 AVCSHistoryObjectType,
 AVCSRunStatus,
 AVCSVerdictState,
} from "../enums/avcs.enums";
import {
 AVCSHistoryActor,
 AVCSMetadata,
} from "../types/avcs.shared";

export interface AVCSHistoryRecord {
 historyId: string;
 runId: string;
 eventType: AVCSHistoryEventType;
 timestamp: string;
 actor?: AVCSHistoryActor;
 summary: string;
 details?: Record<string, unknown>;
 relatedObject?: AVCSHistoryRelatedObject;
 metadata?: AVCSMetadata;
}

export interface AVCSHistoryRelatedObject {
 objectType: AVCSHistoryObjectType;
```

```
 objectId: string;
}

export interface AVCSRunStatusChangeDetails {
 fromStatus?: AVCSRunStatus;
 toStatus: AVCSRunStatus;
}

export interface AVCSVerdictFinalizedDetails {
 verdictState: AVCSVerdictState;
 blockerCount: number;
 warningCount: number;
}
```

What changed from the draft version

This cleaned version fixes the important structural issues:

- shared refs moved into one place
- actor types unified
- metadata typing unified
- duplicated evidence/finding refs removed
- enums consolidated into one canonical file
- imports now form a clean dependency direction
- verdict scoring is typed correctly against `AVCSDomain`

The dependency chain is now clean:

enums -> shared types -> contracts

Not:

contracts importing each other in circles

That matters a lot once the repo gets bigger.

AVCS Runtime 01 — Run Lifecycle

AVCS Runtime 01 — Run Lifecycle

Recommended location:

server/avcs/core/run-lifecycle.ts

This file defines:

- allowed run state transitions
- lifecycle helpers
- guards against invalid state movement
- the canonical execution path of an AVCS run

Without this, the runner can drift into invalid states like:

- jumping from `CREATED` straight to `COMPLETED`
- re-entering `PREFLIGHT` after finalization
- writing verdicts before execution is done
- marking blocked runs as normal completed runs

So this file is the first real control-plane spine for AVCS.

What the lifecycle should control

An AVCS run should move through a tightly controlled sequence:

```
CREATED
 -> PREFLIGHT
 -> RUNNING
 -> FINALIZING
 -> COMPLETED
```

And it may exit early into:

FAILED
BLOCKED

That means:

- `COMPLETED`, `FAILED`, and `BLOCKED` are terminal states
- `FINALIZING` is the only state that should lead into `COMPLETED`
- `BLOCKED` is for non-certifiable or incomplete runs
- `FAILED` is for runs that completed enough evaluation to conclude failure, or execution failure severe enough to terminate the run

---

File content

```
import { AVCSRunStatus } from "../../../shared/avcs/enums/avcs.enums";

const ALLOWED_TRANSITIONS: Record<AVCSRunStatus, AVCSRunStatus[]> = {
 [AVCSRunStatus.CREATED]: [
   AVCSRunStatus.PREFLIGHT,
   AVCSRunStatus.BLOCKED,
   AVCSRunStatus.FAILED,
 ],

 [AVCSRunStatus.PREFLIGHT]: [
   AVCSRunStatus.RUNNING,
   AVCSRunStatus.BLOCKED,
   AVCSRunStatus.FAILED,
 ],

 [AVCSRunStatus.RUNNING]: [
   AVCSRunStatus.FINALIZING,
   AVCSRunStatus.BLOCKED,
   AVCSRunStatus.FAILED,
 ],

 [AVCSRunStatus.FINALIZING]: [
   AVCSRunStatus.COMPLETED,
```

```
  AVCSRunStatus.BLOCKED,
  AVCSRunStatus.FAILED,
 ],

 [AVCSRunStatus.COMPLETED]: [],
 [AVCSRunStatus.FAILED]: [],
 [AVCSRunStatus.BLOCKED]: [],
};

export function canTransitionRunStatus(
 from: AVCSRunStatus,
 to: AVCSRunStatus
): boolean {
 return ALLOWED_TRANSITIONS[from].includes(to);
}

export function assertValidRunStatusTransition(
 from: AVCSRunStatus,
 to: AVCSRunStatus
): void {
 if (!canTransitionRunStatus(from, to)) {
  throw new Error(
    `Invalid AVCS run status transition: ${from} -> ${to}`
  );
 }
}

export function isTerminalRunStatus(status: AVCSRunStatus): boolean {
 return (
  status === AVCSRunStatus.COMPLETED ||
  status === AVCSRunStatus.FAILED ||
  status === AVCSRunStatus.BLOCKED
 );
}

export function isActiveRunStatus(status: AVCSRunStatus): boolean {
 return !isTerminalRunStatus(status);
}

export function getNextHappyPathStatus(
 status: AVCSRunStatus
): AVCSRunStatus | null {
 switch (status) {
  case AVCSRunStatus.CREATED:
```

```
    return AVCSRunStatus.PREFLIGHT;
  case AVCSRunStatus.PREFLIGHT:
    return AVCSRunStatus.RUNNING;
  case AVCSRunStatus.RUNNING:
    return AVCSRunStatus.FINALIZING;
  case AVCSRunStatus.FINALIZING:
    return AVCSRunStatus.COMPLETED;
  default:
    return null;
 }
}

export function getAllowedTransitions(
 status: AVCSRunStatus
): AVCSRunStatus[] {
 return [...ALLOWED_TRANSITIONS[status]];
}
```

Why this shape is correct

This gives you five important things immediately:

1. Explicit transition law

The system knows exactly what moves are valid.

2. Terminal-state protection

Once a run is done, it cannot silently restart.

3. Cleaner runner logic

Your runner can call:

- `assertValidRunStatusTransition`
- `getNextHappyPathStatus`

instead of hardcoding status movement everywhere.

4. Better history writing

Every status change can be validated before a history event is appended.

5. Easier UI truth

The UI can reliably treat:

- `COMPLETED`
- `FAILED`
- `BLOCKED`

as terminal and stable.

Strong rule for this file

Do not let arbitrary services mutate `manifest.status` directly without passing through lifecycle validation.

The safe pattern is:

```
assertValidRunStatusTransition(currentStatus, nextStatus);
manifest.status = nextStatus;
```

Not:

```
manifest.status = AVCSRunStatus.COMPLETED;
```

in random places.

That will save you a lot of future drift.

Recommended improvement

Add a helper to classify terminal outcome meaning.

```
export function isSuccessfulTerminalStatus(status: AVCSRunStatus): boolean {
 return status === AVCSRunStatus.COMPLETED;
}
```

```
export function isFailureTerminalStatus(status: AVCSRunStatus): boolean {
 return (
   status === AVCSRunStatus.FAILED ||
   status === AVCSRunStatus.BLOCKED
 );
}
```

This is useful later for:

- Command Center summaries
- run lists
- history comparison
- quick status badges

Recommended second improvement

If you want stronger operator visibility later, add a lifecycle stage label helper now.

```
export function getRunStatusLabel(status: AVCSRunStatus): string {
 switch (status) {
   case AVCSRunStatus.CREATED:
     return "Created";
   case AVCSRunStatus.PREFLIGHT:
     return "Preflight";
   case AVCSRunStatus.RUNNING:
     return "Running";
   case AVCSRunStatus.FINALIZING:
     return "Finalizing";
   case AVCSRunStatus.COMPLETED:
     return "Completed";
   case AVCSRunStatus.FAILED:
     return "Failed";
   case AVCSRunStatus.BLOCKED:
     return "Blocked";
   default:
     return status;
 }
```

}

Useful for UI badges and timelines.

How this fits the next runtime files

This file is used by:

`run-manifest-service.ts`

to create and update manifests safely

`domain-plan-resolver.ts`

to know whether a run is ready to move into execution

`avcs-runner.ts`

to control state progression during the run

`history-service.ts`

to append validated status-change events

AVCS Runtime 02 — Domain Plan Resolver

AVCS Runtime 02 — Domain Plan Resolver

Recommended location:

server/avcs/core/domain-plan-resolver.ts

This file is what turns a run type into a real execution plan.

Without it, `SMOKE`, `FUNCTIONAL`, and `FULL_CERTIFICATION` are just labels.
With it, AVCS can answer:

- which domains are included
- which domains are required
- in what order they run
- what depth they should run at
- which domains are gate-relevant

This is one of the most important runtime files in the whole system.

What this file should do

Input:

- run type
- optional build class
- optional policy overrides

Output:

- ordered domain plan entries

Each domain plan entry should say:

- domain
- execution order
- required or not
- coverage depth
- gate relevance

This gives the runner a clean plan to execute.

File content

```
import {
 AVCSDomain,
 AVCSRunType,
 AVCSCoverageDepth,
} from "../../../shared/avcs/enums/avcs.enums";

export interface AVCSDomainPlanEntry {
 domain: AVCSDomain;
 order: number;
 required: boolean;
 depth: AVCSCoverageDepth;
 gateRelevant: boolean;
}

export interface AVCSDomainPlan {
 runType: AVCSRunType;
 entries: AVCSDomainPlanEntry[];
}

export interface AVCSDomainPlanContext {
 buildClass?: string;
}
```

```
const RUN_TYPE_DOMAIN_PLANS: Record<AVCSRunType, Omit<AVCSDomainPlanEntry,
"order">[]> = {
 [AVCSRunType.SMOKE]: [
   {
     domain: AVCSDomain.BUILD_INTEGRITY,
     required: true,
     depth: AVCSCoverageDepth.SHALLOW,
     gateRelevant: true,
   },
   {
     domain: AVCSDomain.FUNCTIONAL_CORRECTNESS,
     required: true,
     depth: AVCSCoverageDepth.SHALLOW,
     gateRelevant: true,
   },
 ],

 [AVCSRunType.FUNCTIONAL]: [
   {
     domain: AVCSDomain.BUILD_INTEGRITY,
     required: true,
     depth: AVCSCoverageDepth.STANDARD,
     gateRelevant: true,
   },
   {
     domain: AVCSDomain.FUNCTIONAL_CORRECTNESS,
     required: true,
     depth: AVCSCoverageDepth.STANDARD,
     gateRelevant: true,
   },
 ],

 [AVCSRunType.SECURITY]: [
   {
     domain: AVCSDomain.BUILD_INTEGRITY,
     required: true,
     depth: AVCSCoverageDepth.SHALLOW,
     gateRelevant: true,
   },
   {
     domain: AVCSDomain.SECURITY_POSTURE,
     required: true,
     depth: AVCSCoverageDepth.STANDARD,
```

```
    gateRelevant: true,
  },
],

[AVCSRunType.PERFORMANCE]: [
  {
    domain: AVCSDomain.BUILD_INTEGRITY,
    required: true,
    depth: AVCSCoverageDepth.SHALLOW,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.PERFORMANCE_STABILITY,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
],

[AVCSRunType.FULL_CERTIFICATION]: [
  {
    domain: AVCSDomain.BUILD_INTEGRITY,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.FUNCTIONAL_CORRECTNESS,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.SECURITY_POSTURE,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.PERFORMANCE_STABILITY,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
```

```
  {
    domain: AVCSDomain.DEPLOYMENT_READINESS,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
],

[AVCSRunType.REGRESSION]: [
  {
    domain: AVCSDomain.FUNCTIONAL_CORRECTNESS,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.SECURITY_POSTURE,
    required: false,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.PERFORMANCE_STABILITY,
    required: false,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
],

[AVCSRunType.PRE_DEPLOYMENT_GATE]: [
  {
    domain: AVCSDomain.BUILD_INTEGRITY,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.FUNCTIONAL_CORRECTNESS,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.SECURITY_POSTURE,
```

```
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.PERFORMANCE_STABILITY,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true,
  },
  {
    domain: AVCSDomain.DEPLOYMENT_READINESS,
    required: true,
    depth: AVCSCoverageDepth.DEEP,
    gateRelevant: true,
  },
],
};

export function resolveDomainPlan(
 runType: AVCSRunType,
 _context?: AVCSDomainPlanContext
): AVCSDomainPlan {
 const basePlan = RUN_TYPE_DOMAIN_PLANS[runType];

 if (!basePlan) {
   throw new Error(`No domain plan configured for run type: ${runType}`);
 }

 const entries: AVCSDomainPlanEntry[] = basePlan.map((entry, index) => ({
   ...entry,
   order: index + 1,
 }));

 return {
   runType,
   entries,
 };
}

export function getIncludedDomains(plan: AVCSDomainPlan): AVCSDomain[] {
 return plan.entries.map((entry) => entry.domain);
}
```

```
export function getRequiredDomains(plan: AVCSDomainPlan): AVCSDomain[] {
 return plan.entries
   .filter((entry) => entry.required)
   .map((entry) => entry.domain);
}

export function getGateRelevantDomains(plan: AVCSDomainPlan): AVCSDomain[] {
 return plan.entries
   .filter((entry) => entry.gateRelevant)
   .map((entry) => entry.domain);
}

export function getOrderedDomainPlanEntries(
 plan: AVCSDomainPlan
): AVCSDomainPlanEntry[] {
 return [...plan.entries].sort((a, b) => a.order - b.order);
}
```

Why this shape is correct

This gives AVCS a clean separation between:

- **run type**
- **domain selection**
- **domain execution order**
- **coverage expectation**
- **gate relevance**

That is exactly what you want.

The runner does not need to guess.

It can just do:

1. resolve plan
2. iterate plan entries in order

3. run evaluator for each domain
4. enforce required/gate logic later

Why the order matters

The default order here is intentional.

Foundation-first order

You generally want:

1. `BUILD_INTEGRITY`
2. `FUNCTIONAL_CORRECTNESS`
3. `SECURITY_POSTURE`
4. `PERFORMANCE_STABILITY`
5. `DEPLOYMENT_READINESS`

Reason:

- structural sanity first
- core behavior second
- safety third
- runtime quality fourth
- deployability at the end once the rest is known

This makes failures easier to interpret and helps avoid wasting time running later checks on obviously broken builds.

Strong design rule

Do not scatter domain inclusion logic across the codebase.

Bad:

```
if (runType === AVCSRunType.SMOKE) {
 // maybe run build integrity here
}
```

in one service, and then different logic somewhere else.

Good:

```
const plan = resolveDomainPlan(runType);
```

Then every other service consumes the same plan.

That keeps AVCS deterministic.

What this unlocks immediately

This file gives you:

1. Manifest population

`includedDomains` can come directly from the plan.

2. Runner sequencing

The runner knows what order to execute evaluators.

3. Coverage expectations

Each domain already has a planned depth.

4. Gate logic

You know which domains are required and gate-relevant.

5. UI hints

Command Center can show what domains a selected run type will include before the run starts.

Recommended improvement

Right now the context is unused:

_context?: AVCSDomainPlanContext

Keep it anyway.

That is the right move because later you will want build-class expansion such as:

- admin console builds requiring deeper security
- customer-facing builds requiring accessibility
- deployment-critical builds requiring deeper deployment readiness

So later you can add build-class-specific adjustments without changing the API.

Example future direction:

```
if (_context?.buildClass === "customer_facing_web") {
 // add ACCESSIBILITY and UI_QUALITY
}
```

That is exactly why the context belongs here now.

Recommended second improvement

Add a helper to validate required evaluator coverage.

```
export function assertDomainPlanIsExecutable(
 plan: AVCSDomainPlan,
 availableDomains: AVCSDomain[]
): void {
 const missing = plan.entries
   .map((entry) => entry.domain)
   .filter((domain) => !availableDomains.includes(domain));

 if (missing.length > 0) {
   throw new Error(
```

```
    `Domain plan cannot execute. Missing evaluators for domains: ${missing.join(", ")}`
  );
 }
}
```

This will be very useful once you create evaluator registration.

Example output

For:

resolveDomainPlan(AVCSRunType.FULL_CERTIFICATION)

You get:

```
{
 runType: AVCSRunType.FULL_CERTIFICATION,
 entries: [
   {
     domain: AVCSDomain.BUILD_INTEGRITY,
     order: 1,
     required: true,
     depth: AVCSCoverageDepth.STANDARD,
     gateRelevant: true
   },
   {
     domain: AVCSDomain.FUNCTIONAL_CORRECTNESS,
     order: 2,
     required: true,
     depth: AVCSCoverageDepth.STANDARD,
     gateRelevant: true
   },
   {
     domain: AVCSDomain.SECURITY_POSTURE,
     order: 3,
     required: true,
     depth: AVCSCoverageDepth.STANDARD,
     gateRelevant: true
   },
   {
     domain: AVCSDomain.PERFORMANCE_STABILITY,
     order: 4,
```

```
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true
  },
  {
    domain: AVCSDomain.DEPLOYMENT_READINESS,
    order: 5,
    required: true,
    depth: AVCSCoverageDepth.STANDARD,
    gateRelevant: true
  }
]
}
```

That is exactly the kind of clean execution plan you want.

AVCS Runtime 03 — Run Manifest Service

AVCS Runtime 03 — Run Manifest Service

Recommended location:

server/avcs/core/run-manifest-service.ts

This is the first service that creates a **real AVCS run object**.

It is responsible for turning:

- selected build target
- selected run type
- actor
- policy snapshot
- resolved domain plan

into a valid `AVCSRunManifest`.

This service is important because manifest creation should happen in **one canonical place**. You do not want random parts of the system building run manifests differently.

What this service should do

It should:

- validate required manifest inputs
- resolve the domain plan from the selected run type
- populate `includedDomains`
- assign `CREATED` as the initial status
- generate a run ID

- stamp `createdAt`
- return a valid `AVCSRunManifest`

This becomes the official entry point for run creation.

File content

```
import { randomUUID } from "crypto";

import { AVCSRunManifest, AVCSRunTarget } from
"../../../shared/avcs/contracts/run-manifest.contract";
import { AVCSRunType, AVCSRunStatus } from "../../../shared/avcs/enums/avcs.enums";
import { AVCSRunActor } from "../../../shared/avcs/types/avcs.shared";
import { resolveDomainPlan, AVCSDomainPlanContext } from "./domain-plan-resolver";

export interface CreateAVCSRunManifestInput {
 target: AVCSRunTarget;
 runType: AVCSRunType;
 actor: AVCSRunActor;
 policySnapshotId: string;
 metadata?: Record<string, unknown>;
 planContext?: AVCSDomainPlanContext;
}

export function createAVCSRunManifest(
 input: CreateAVCSRunManifestInput
): AVCSRunManifest {
 validateCreateRunManifestInput(input);

 const plan = resolveDomainPlan(input.runType, input.planContext);

 return {
   runId: generateAVCSRunId(),
   target: input.target,
   runType: input.runType,
   includedDomains: plan.entries.map((entry) => entry.domain),
   actor: input.actor,
   policySnapshotId: input.policySnapshotId,
   createdAt: new Date().toISOString(),
```

```
  status: AVCSRunStatus.CREATED,
  metadata: input.metadata,
 };
}

export function validateCreateRunManifestInput(
 input: CreateAVCSRunManifestInput
): void {
 if (!input.target) {
  throw new Error("AVCS run manifest creation requires a target.");
 }

 if (!input.target.buildId) {
  throw new Error("AVCS run manifest target requires buildId.");
 }

 if (!input.target.buildVersion) {
  throw new Error("AVCS run manifest target requires buildVersion.");
 }

 if (!input.target.artifactLocation) {
  throw new Error("AVCS run manifest target requires artifactLocation.");
 }

 if (!input.runType) {
  throw new Error("AVCS run manifest creation requires runType.");
 }

 if (!input.actor) {
  throw new Error("AVCS run manifest creation requires actor.");
 }

 if (!input.actor.actorId) {
  throw new Error("AVCS run manifest actor requires actorId.");
 }

 if (!input.actor.actorType) {
  throw new Error("AVCS run manifest actor requires actorType.");
 }

 if (!input.policySnapshotId) {
  throw new Error("AVCS run manifest creation requires policySnapshotId.");
 }
}
```

```
export function generateAVCSRunId(): string {
 return `avcs_run_${randomUUID()}`;
}
```

Why this shape is correct

This keeps the manifest creation path:

- centralized
- deterministic
- validated
- plan-aware

That is exactly what you want.

Instead of letting UI handlers, runners, or controllers assemble the manifest themselves, everything goes through:

createAVCSRunManifest(...)

That makes the system much harder to drift.

What this service is deliberately not doing

It is **not**:

- mutating lifecycle state beyond initial `CREATED`
- running preflight
- persisting the manifest
- writing history records
- creating domain results

- starting execution

That separation is good.

This service should do one thing well:

create a valid manifest

Example usage

```
import { AVCSRunType, AVCSActorType } from "../../../shared/avcs/enums/avcs.enums";
import { createAVCSRunManifest } from "./run-manifest-service";

const manifest = createAVCSRunManifest({
 target: {
   buildId: "kit_7421",
   buildVersion: "v1.3.0",
   artifactLocation: "/artifacts/builds/kit_7421",
   buildClass: "web_application",
   generatedAt: "2026-03-05T12:20:44Z",
 },
 runType: AVCSRunType.FULL_CERTIFICATION,
 actor: {
   actorId: "user_1",
   actorType: AVCSActorType.USER,
   name: "Operator",
 },
 policySnapshotId: "policy_2026_03_v1",
});
```

Result:

```
{
 runId: "avcs_run_...",
 target: { ... },
 runType: AVCSRunType.FULL_CERTIFICATION,
 includedDomains: [
   AVCSDomain.BUILD_INTEGRITY,
   AVCSDomain.FUNCTIONAL_CORRECTNESS,
   AVCSDomain.SECURITY_POSTURE,
   AVCSDomain.PERFORMANCE_STABILITY,
   AVCSDomain.DEPLOYMENT_READINESS
```

```
 ],
 actor: { ... },
 policySnapshotId: "policy_2026_03_v1",
 createdAt: "2026-03-07T...",
 status: AVCSRunStatus.CREATED
}
```

Strong design rule

Do not let callers supply `includedDomains` directly for normal run creation.

Why:

- that bypasses the domain plan
- weakens policy consistency
- makes run meaning unstable

The correct flow is:

runType -> domain plan resolver -> includedDomains

Not:

caller manually invents includedDomains

You can support custom scoped runs later, but that should be a separate controlled path.

Recommended improvement

Add a helper that returns both the manifest and the resolved plan, because the caller will often need both.

```
import { AVCSDomainPlan } from "./domain-plan-resolver";

export interface CreateAVCSRunManifestResult {
 manifest: AVCSRunManifest;
 plan: AVCSDomainPlan;
}
```

```
export function createAVCSRunManifestWithPlan(
 input: CreateAVCSRunManifestInput
): CreateAVCSRunManifestResult {
 validateCreateRunManifestInput(input);

 const plan = resolveDomainPlan(input.runType, input.planContext);

 const manifest: AVCSRunManifest = {
   runId: generateAVCSRunId(),
   target: input.target,
   runType: input.runType,
   includedDomains: plan.entries.map((entry) => entry.domain),
   actor: input.actor,
   policySnapshotId: input.policySnapshotId,
   createdAt: new Date().toISOString(),
   status: AVCSRunStatus.CREATED,
   metadata: input.metadata,
 };

 return { manifest, plan };
}
```

This is often better than resolving the plan twice in two different places.

Recommended stronger final version

This is the version I would actually use:

```
import { randomUUID } from "crypto";

import { AVCSRunManifest, AVCSRunTarget } from
"../../../shared/avcs/contracts/run-manifest.contract";
import { AVCSRunStatus, AVCSRunType } from "../../../shared/avcs/enums/avcs.enums";
import { AVCSRunActor } from "../../../shared/avcs/types/avcs.shared";
import {
 resolveDomainPlan,
 AVCSDomainPlan,
 AVCSDomainPlanContext,
} from "./domain-plan-resolver";

export interface CreateAVCSRunManifestInput {
```

```
 target: AVCSRunTarget;
 runType: AVCSRunType;
 actor: AVCSRunActor;
 policySnapshotId: string;
 metadata?: Record<string, unknown>;
 planContext?: AVCSDomainPlanContext;
}

export interface CreateAVCSRunManifestResult {
 manifest: AVCSRunManifest;
 plan: AVCSDomainPlan;
}

export function createAVCSRunManifestWithPlan(
 input: CreateAVCSRunManifestInput
): CreateAVCSRunManifestResult {
 validateCreateRunManifestInput(input);

 const plan = resolveDomainPlan(input.runType, input.planContext);

 const manifest: AVCSRunManifest = {
   runId: generateAVCSRunId(),
   target: input.target,
   runType: input.runType,
   includedDomains: plan.entries.map((entry) => entry.domain),
   actor: input.actor,
   policySnapshotId: input.policySnapshotId,
   createdAt: new Date().toISOString(),
   status: AVCSRunStatus.CREATED,
   metadata: input.metadata,
 };

 return { manifest, plan };
}

export function createAVCSRunManifest(
 input: CreateAVCSRunManifestInput
): AVCSRunManifest {
 return createAVCSRunManifestWithPlan(input).manifest;
}

export function validateCreateRunManifestInput(
 input: CreateAVCSRunManifestInput
): void {
```

```
 if (!input.target) {
   throw new Error("AVCS run manifest creation requires a target.");
 }

 if (!input.target.buildId) {
   throw new Error("AVCS run manifest target requires buildId.");
 }

 if (!input.target.buildVersion) {
   throw new Error("AVCS run manifest target requires buildVersion.");
 }

 if (!input.target.artifactLocation) {
   throw new Error("AVCS run manifest target requires artifactLocation.");
 }

 if (!input.runType) {
   throw new Error("AVCS run manifest creation requires runType.");
 }

 if (!input.actor) {
   throw new Error("AVCS run manifest creation requires actor.");
 }

 if (!input.actor.actorId) {
   throw new Error("AVCS run manifest actor requires actorId.");
 }

 if (!input.actor.actorType) {
   throw new Error("AVCS run manifest actor requires actorType.");
 }

 if (!input.policySnapshotId) {
   throw new Error("AVCS run manifest creation requires policySnapshotId.");
 }
}

export function generateAVCSRunId(): string {
 return `avcs_run_${randomUUID()}`;
}
```

AVCS Runtime 04 — AVCS Runner Skeleton

AVCS Runtime 04 — AVCS Runner Skeleton

Recommended location:

server/avcs/core/avcs-runner.ts

This is the first file that makes AVCS feel like a real system.

Up to now, you have:

- contracts
- lifecycle rules
- domain planning
- manifest creation

Now you need the orchestrator that actually:

- creates the run
- moves it through the lifecycle
- executes preflight
- runs domains in order
- finalizes the run

This first version should stay intentionally lean. It is a **runner skeleton**, not the full engine yet.

What this file should do

The runner skeleton should:

1. create the manifest
2. move status from `CREATED` -> `PREFLIGHT`
3. run preflight
4. move status from `PREFLIGHT` -> `RUNNING`
5. execute each domain in plan order
6. move status from `RUNNING` -> `FINALIZING`
7. assemble a placeholder verdict stage
8. move status to terminal state

For now, it can use:

- placeholder preflight
- placeholder evaluator execution
- placeholder verdict finalization

That is fine. The goal is to establish the backbone correctly.

File content

```
import { AVCSRunManifest } from "../../../shared/avcs/contracts/run-manifest.contract";
import { AVCSDomainResult } from "../../../shared/avcs/contracts/domain-result.contract";
import { AVCSVerdict } from "../../../shared/avcs/contracts/verdict.contract";
import {
 AVCSDomain,
 AVCSRunStatus,
 AVCSVerdictState,
 AVCSDomainStatus,
 AVCSCoverageState,
} from "../../../shared/avcs/enums/avcs.enums";
import {
```

```
 createAVCSRunManifestWithPlan,
 CreateAVCSRunManifestInput,
} from "./run-manifest-service";
import {
 assertValidRunStatusTransition,
 isTerminalRunStatus,
} from "./run-lifecycle";
import {
 AVCSDomainPlan,
 AVCSDomainPlanEntry,
} from "./domain-plan-resolver";

export interface AVCSRunExecutionResult {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
 verdict: AVCSVerdict;
}

export interface AVCSPreflightResult {
 ok: boolean;
 reason?: string;
}

export interface AVCSDomainEvaluator {
 domain: AVCSDomain;
 evaluate: (
   manifest: AVCSRunManifest,
   entry: AVCSDomainPlanEntry
 ) => Promise<AVCSDomainResult>;
}

export class AVCSRunner {
 private readonly evaluators: Map<AVCSDomain, AVCSDomainEvaluator>;

 constructor(evaluators: AVCSDomainEvaluator[] = []) {
   this.evaluators = new Map(
     evaluators.map((evaluator) => [evaluator.domain, evaluator])
   );
 }

 async run(
   input: CreateAVCSRunManifestInput
 ): Promise<AVCSRunExecutionResult> {
   const { manifest, plan } = createAVCSRunManifestWithPlan(input);
```

```
const domainResults: AVCSDomainResult[] = [];

await this.transitionManifestStatus(manifest, AVCSRunStatus.PREFLIGHT);

const preflight = await this.runPreflight(manifest, plan);
if (!preflight.ok) {
  await this.transitionManifestStatus(manifest, AVCSRunStatus.BLOCKED);

  const verdict = this.createBlockedVerdict(
    manifest,
    preflight.reason ?? "Preflight failed."
  );

  return {
    manifest,
    domainResults,
    verdict,
  };
}

await this.transitionManifestStatus(manifest, AVCSRunStatus.RUNNING);

for (const entry of plan.entries) {
  const result = await this.executeDomain(manifest, entry);
  domainResults.push(result);

  if (result.status === AVCSDomainStatus.BLOCKED) {
    await this.transitionManifestStatus(manifest, AVCSRunStatus.BLOCKED);

    const verdict = this.createBlockedVerdict(
      manifest,
      `Run blocked during ${entry.domain}.`
    );

    return {
      manifest,
      domainResults,
      verdict,
    };
  }
}

await this.transitionManifestStatus(manifest, AVCSRunStatus.FINALIZING);
```

```
  const verdict = await this.finalizeVerdict(manifest, domainResults);

  await this.transitionManifestStatus(
    manifest,
    verdict.state === AVCSVerdictState.BLOCKED
      ? AVCSRunStatus.BLOCKED
      : AVCSRunStatus.COMPLETED
  );

  return {
    manifest,
    domainResults,
    verdict,
  };
}

protected async runPreflight(
  manifest: AVCSRunManifest,
  _plan: AVCSDomainPlan
): Promise<AVCSPreflightResult> {
  if (!manifest.target.artifactLocation) {
    return {
      ok: false,
      reason: "Missing artifact location.",
    };
  }

  return { ok: true };
}

protected async executeDomain(
  manifest: AVCSRunManifest,
  entry: AVCSDomainPlanEntry
): Promise<AVCSDomainResult> {
  const evaluator = this.evaluators.get(entry.domain);

  if (!evaluator) {
    return this.createBlockedDomainResult(
      manifest.runId,
      entry.domain,
      `No evaluator registered for domain ${entry.domain}.`
    );
  }
```

```
  return evaluator.evaluate(manifest, entry);
}

protected async finalizeVerdict(
  manifest: AVCSRunManifest,
  domainResults: AVCSDomainResult[]
): Promise<AVCSVerdict> {
  const blockedCount = domainResults.filter(
    (result) => result.status === AVCSDomainStatus.BLOCKED
  ).length;

  const failedCount = domainResults.filter(
    (result) => result.status === AVCSDomainStatus.FAILED
  ).length;

  const state =
    blockedCount > 0
      ? AVCSVerdictState.BLOCKED
      : failedCount > 0
      ? AVCSVerdictState.FAIL
      : AVCSVerdictState.PASS;

  return {
    verdictId: `verdict_${manifest.runId}`,
    runId: manifest.runId,
    state,
    blockerCount: failedCount + blockedCount,
    warningCount: 0,
    coverageSufficient: true,
    evidenceComplete: true,
    linkedFindingIds: [],
    linkedEvidenceIds: [],
    reasoning:
      state === AVCSVerdictState.PASS
        ? "All executed domains completed without failure."
        : state === AVCSVerdictState.FAIL
        ? "One or more domains failed during execution."
        : "Run was blocked before certification could complete.",
    decidedAt: new Date().toISOString(),
  };
}

protected createBlockedVerdict(
  manifest: AVCSRunManifest,
```

```
  reason: string
): AVCSVerdict {
  return {
    verdictId: `verdict_${manifest.runId}`,
    runId: manifest.runId,
    state: AVCSVerdictState.BLOCKED,
    blockerCount: 1,
    warningCount: 0,
    coverageSufficient: false,
    evidenceComplete: false,
    linkedFindingIds: [],
    linkedEvidenceIds: [],
    reasoning: reason,
    decidedAt: new Date().toISOString(),
  };
}

protected createBlockedDomainResult(
  runId: string,
  domain: AVCSDomain,
  summary: string
): AVCSDomainResult {
  return {
    domainResultId: `domain_result_${runId}_${domain}`,
    runId,
    domain,
    status: AVCSDomainStatus.BLOCKED,
    startedAt: new Date().toISOString(),
    completedAt: new Date().toISOString(),
    summary,
    coverageState: AVCSCoverageState.BLOCKED,
    blockerDetected: true,
    findings: [],
    evidence: [],
    checks: [],
  };
}

protected async transitionManifestStatus(
  manifest: AVCSRunManifest,
  nextStatus: AVCSRunStatus
): Promise<void> {
  if (isTerminalRunStatus(manifest.status)) {
    throw new Error(
```

```
      `Cannot transition terminal AVCS run status: ${manifest.status} -> ${nextStatus}`
    );
  }

  assertValidRunStatusTransition(manifest.status, nextStatus);
  manifest.status = nextStatus;
 }
}
```

Why this shape is correct

This runner skeleton gives you the correct control flow without prematurely overbuilding.

It establishes:

- one canonical run entry point
- lifecycle-safe status transitions
- explicit preflight stage
- ordered domain execution
- evaluator registration model
- finalization stage
- terminal verdict return object

That is the right first orchestrator.

What is intentionally placeholder right now

Three areas are intentionally simple:

Preflight

Right now it only checks artifact location.

Later this should validate:

- target existence
- policy availability
- evaluator availability
- baseline availability for regression runs
- evidence storage readiness

Domain execution

Right now it delegates to registered evaluators and blocks if missing.

Later this should also:

- append history events
- collect evidence
- normalize findings
- assemble coverage incrementally

Verdict finalization

Right now it uses simple failed/blocked counting.

Later this should use:

- findings
- blocker logic
- coverage sufficiency
- evidence completeness
- score
- policy gates

That is exactly how it should evolve.

Strong design rule

Do not let the runner directly know domain-specific logic.

The runner should know:

- sequencing
- lifecycle
- orchestration

The evaluator should know:

- how to test a domain

That separation is what keeps the system scalable.

So this is good:

```
const evaluator = this.evaluators.get(entry.domain);
return evaluator.evaluate(manifest, entry);
```

And this is bad:

```
if (entry.domain === AVCSDomain.SECURITY_POSTURE) {
 // security logic here
}
```

inside the runner.

Important note about terminal status

Right now the runner maps verdict states to run statuses like this:

- `BLOCKED` verdict -> `BLOCKED` run status

- everything else -> `COMPLETED`

That is correct.

Why?
Because run status describes **execution lifecycle**, not certification outcome.

A run that finishes and yields `FAIL` still completed execution successfully, so its run status should be `COMPLETED`, while its verdict state is `FAIL`.

That distinction is important.

Recommended improvement

Add evaluator registration guards so you can validate plan executability before running.

Add this method:

```
protected assertPlanExecutable(plan: AVCSDomainPlan): void {
 const missing = plan.entries
   .map((entry) => entry.domain)
   .filter((domain) => !this.evaluators.has(domain));

 if (missing.length > 0) {
   throw new Error(
     `AVCS runner missing evaluators for domains: ${missing.join(", ")}`
   );
 }
}
```

Then call it after manifest creation and before preflight:

```
this.assertPlanExecutable(plan);
```

That gives you an early hard failure instead of a blocked domain later if you prefer stricter runtime behavior.

For MVP, either approach is acceptable:

- strict fail early

- block missing evaluator at domain execution

I slightly prefer fail early for internal developer clarity.

Recommended second improvement

Add lightweight lifecycle hooks now, even if they are no-ops.

```
protected async onRunCreated(_manifest: AVCSRunManifest): Promise<void> {}
protected async onStatusChanged(
 _manifest: AVCSRunManifest,
 _from: AVCSRunStatus,
 _to: AVCSRunStatus
): Promise<void> {}
protected async onDomainCompleted(
 _result: AVCSDomainResult
): Promise<void> {}
protected async onVerdictFinalized(
 _verdict: AVCSVerdict
): Promise<void> {}
```

Why this matters:
later you can use them for:

- history persistence
- logging
- event streaming
- UI live updates
- metrics

without rewriting the runner.

Example of how this would be used

```
const runner = new AVCSRunner([
```

```
 buildIntegrityEvaluator,
 functionalCorrectnessEvaluator,
 securityPostureEvaluator,
 performanceStabilityEvaluator,
 deploymentReadinessEvaluator,
]);

const result = await runner.run({
 target: {
   buildId: "kit_7421",
   buildVersion: "v1.3.0",
   artifactLocation: "/artifacts/builds/kit_7421",
   buildClass: "web_application",
 },
 runType: AVCSRunType.FULL_CERTIFICATION,
 actor: {
   actorId: "user_1",
   actorType: AVCSActorType.USER,
   name: "Operator",
 },
 policySnapshotId: "policy_2026_03_v1",
});
```

Returned:

- `manifest`
- `domainResults`
- `verdict`

That is enough for the first Run Detail page.

AVCS Runtime 05 — First Domain Evaluator

AVCS Runtime 05 — First Domain Evaluator

Build Integrity Evaluator

Recommended location:

server/avcs/evaluators/build-integrity/build-integrity-evaluator.ts

This is the right first evaluator because **Build Integrity** is the foundation domain.

Before AVCS can judge functionality, security, performance, or deployment readiness, it needs to know whether the build target is structurally valid enough to test.

This evaluator should answer:

- do required files exist
- is the artifact location usable
- is the env contract present
- is the startup path declared
- is the build structurally testable

Design goal

Do **not** make the first evaluator too magical.

The first version should be:

- deterministic
- simple
- easy to extend

- easy to debug

So the first Build Integrity evaluator should be a check runner with 3–4 explicit checks, each returning:

- check status
- summary
- optional finding refs later
- optional evidence refs later

File content

```
import fs from "fs/promises";
import path from "path";

import { AVCSRunManifest } from "../../../../shared/avcs/contracts/run-manifest.contract";
import {
 AVCSDomainResult,
 AVCSDomainCheckResult,
} from "../../../../shared/avcs/contracts/domain-result.contract";
import {
 AVCSDomain,
 AVCSDomainStatus,
 AVCSDomainCheckStatus,
 AVCSCoverageState,
 AVCSCoverageDepth,
} from "../../../../shared/avcs/enums/avcs.enums";
import { AVCSDomainPlanEntry } from "../../core/domain-plan-resolver";
import { AVCSDomainEvaluator } from "../../core/avcs-runner";

export class BuildIntegrityEvaluator implements AVCSDomainEvaluator {
 public readonly domain = AVCSDomain.BUILD_INTEGRITY;

 async evaluate(
   manifest: AVCSRunManifest,
   entry: AVCSDomainPlanEntry
 ): Promise<AVCSDomainResult> {
```

```
  const startedAt = new Date().toISOString();
  const checks: AVCSDomainCheckResult[] = [];

  checks.push(await this.runArtifactLocationCheck(manifest));
  checks.push(await this.runRequiredFilesCheck(manifest));
  checks.push(await this.runEnvironmentContractCheck(manifest));
  checks.push(await this.runStartupDeclarationCheck(manifest));

  const completedAt = new Date().toISOString();

  const failedChecks = checks.filter(
    (check) => check.status === AVCSDomainCheckStatus.FAILED
  );
  const blockedChecks = checks.filter(
    (check) => check.status === AVCSDomainCheckStatus.BLOCKED
  );
  const partialChecks = checks.filter(
    (check) => check.status === AVCSDomainCheckStatus.PARTIAL
  );

  let status: AVCSDomainStatus;
  let coverageState: AVCSCoverageState;
  let blockerDetected = false;
  let summary: string;

  if (blockedChecks.length > 0) {
    status = AVCSDomainStatus.BLOCKED;
    coverageState = AVCSCoverageState.BLOCKED;
    blockerDetected = true;
    summary = `Build Integrity blocked. ${blockedChecks.length} check(s) blocked execution.`;
  } else if (failedChecks.length > 0) {
    status = AVCSDomainStatus.FAILED;
    coverageState = AVCSCoverageState.COVERED_FAILED;
    blockerDetected = true;
    summary = `Build Integrity failed. ${failedChecks.length} check(s) failed.`;
  } else if (partialChecks.length > 0) {
    status = AVCSDomainStatus.PARTIAL;
    coverageState = AVCSCoverageState.PARTIAL;
    summary = `Build Integrity completed partially. ${partialChecks.length} check(s) were
partial.`;
  } else {
    status = AVCSDomainStatus.PASSED;
    coverageState = AVCSCoverageState.COVERED;
    summary = "Build Integrity passed all required checks.";
```

```
  }

  return {
    domainResultId: `domain_result_${manifest.runId}_${this.domain}`,
    runId: manifest.runId,
    domain: this.domain,
    status,
    startedAt,
    completedAt,
    summary,
    coverageState,
    blockerDetected,
    findings: [],
    evidence: [],
    checks,
    metadata: {
      plannedDepth: entry.depth ?? AVCSCoverageDepth.STANDARD,
      required: entry.required,
      gateRelevant: entry.gateRelevant,
    },
  };
}

private async runArtifactLocationCheck(
  manifest: AVCSRunManifest
): Promise<AVCSDomainCheckResult> {
  const startedAt = new Date().toISOString();
  const artifactLocation = manifest.target.artifactLocation;

  try {
    const stats = await fs.stat(artifactLocation);

    if (!stats.isDirectory()) {
      return this.makeCheckResult({
        checkKey: "artifact_location",
        name: "Artifact Location Check",
        status: AVCSDomainCheckStatus.FAILED,
        summary: "Artifact location exists but is not a directory.",
        startedAt,
      });
    }

    return this.makeCheckResult({
      checkKey: "artifact_location",
```

```
      name: "Artifact Location Check",
      status: AVCSDomainCheckStatus.PASSED,
      summary: "Artifact location exists and is readable.",
      startedAt,
    });
  } catch {
    return this.makeCheckResult({
      checkKey: "artifact_location",
      name: "Artifact Location Check",
      status: AVCSDomainCheckStatus.BLOCKED,
      summary: "Artifact location does not exist or cannot be accessed.",
      startedAt,
    });
  }
}

private async runRequiredFilesCheck(
  manifest: AVCSRunManifest
): Promise<AVCSDomainCheckResult> {
  const startedAt = new Date().toISOString();
  const base = manifest.target.artifactLocation;

  const requiredFiles = ["package.json"];
  const missing: string[] = [];

  for (const relativeFile of requiredFiles) {
    const fullPath = path.join(base, relativeFile);
    try {
      await fs.access(fullPath);
    } catch {
      missing.push(relativeFile);
    }
  }

  if (missing.length > 0) {
    return this.makeCheckResult({
      checkKey: "required_files",
      name: "Required Files Check",
      status: AVCSDomainCheckStatus.FAILED,
      summary: `Missing required file(s): ${missing.join(", ")}.`,
      startedAt,
      metadata: { missing },
    });
  }
```

```
  return this.makeCheckResult({
    checkKey: "required_files",
    name: "Required Files Check",
    status: AVCSDomainCheckStatus.PASSED,
    summary: "All required files were present.",
    startedAt,
    metadata: { requiredFiles },
  });
}

private async runEnvironmentContractCheck(
  manifest: AVCSRunManifest
): Promise<AVCSDomainCheckResult> {
  const startedAt = new Date().toISOString();
  const base = manifest.target.artifactLocation;

  const candidateFiles = [
    "env.contract.json",
    ".env.example",
    ".env.schema.json",
  ];

  const found: string[] = [];

  for (const relativeFile of candidateFiles) {
    const fullPath = path.join(base, relativeFile);
    try {
      await fs.access(fullPath);
      found.push(relativeFile);
    } catch {
      // ignore
    }
  }

  if (found.length === 0) {
    return this.makeCheckResult({
      checkKey: "environment_contract",
      name: "Environment Contract Check",
      status: AVCSDomainCheckStatus.FAILED,
      summary:
        "No environment contract file was found. Expected one of env.contract.json, .env.example,
or .env.schema.json.",
      startedAt,
```

```
      metadata: { candidates: candidateFiles },
    });
  }

  return this.makeCheckResult({
    checkKey: "environment_contract",
    name: "Environment Contract Check",
    status: AVCSDomainCheckStatus.PASSED,
    summary: `Environment contract surface found: ${found.join(", ")}.`,
    startedAt,
    metadata: { found },
  });
}

private async runStartupDeclarationCheck(
  manifest: AVCSRunManifest
): Promise<AVCSDomainCheckResult> {
  const startedAt = new Date().toISOString();
  const packageJsonPath = path.join(
    manifest.target.artifactLocation,
    "package.json"
  );

  try {
    const raw = await fs.readFile(packageJsonPath, "utf8");
    const parsed = JSON.parse(raw) as {
      scripts?: Record<string, string>;
    };

    const scripts = parsed.scripts ?? {};
    const startupScript =
      scripts.start ?? scripts.dev ?? scripts.build ?? null;

    if (!startupScript) {
      return this.makeCheckResult({
        checkKey: "startup_declaration",
        name: "Startup Declaration Check",
        status: AVCSDomainCheckStatus.FAILED,
        summary:
          "No startup-relevant script was found in package.json (expected start, dev, or build).",
        startedAt,
      });
    }
```

```
    return this.makeCheckResult({
      checkKey: "startup_declaration",
      name: "Startup Declaration Check",
      status: AVCSDomainCheckStatus.PASSED,
      summary: "Startup-relevant script declaration was found in package.json.",
      startedAt,
      metadata: { startupScript },
    });
  } catch {
    return this.makeCheckResult({
      checkKey: "startup_declaration",
      name: "Startup Declaration Check",
      status: AVCSDomainCheckStatus.BLOCKED,
      summary: "Unable to read or parse package.json for startup validation.",
      startedAt,
    });
  }
}

private makeCheckResult(input: {
  checkKey: string;
  name: string;
  status: AVCSDomainCheckStatus;
  summary: string;
  startedAt: string;
  metadata?: Record<string, unknown>;
}): AVCSDomainCheckResult {
  return {
    checkId: `${this.domain}_${input.checkKey}`,
    checkKey: input.checkKey,
    name: input.name,
    status: input.status,
    summary: input.summary,
    startedAt: input.startedAt,
    completedAt: new Date().toISOString(),
    findings: [],
    evidence: [],
    metadata: input.metadata,
  };
}
}
```

Why this evaluator is the right first one

It gives you a real domain with real filesystem-based validation, but without forcing the whole evidence/finding stack to exist yet.

It checks four useful things:

1. **Artifact location exists**
2. **Required files exist**
3. **Environment contract exists**
4. **Startup declaration exists**

That is enough to make `BUILD_INTEGRITY` meaningful immediately.

What this unlocks right now

Once you register this evaluator in the runner, AVCS can perform a real `SMOKE` or `FULL_CERTIFICATION` partial execution.

Example:

```
const runner = new AVCSRunner([
 new BuildIntegrityEvaluator(),
]);
```

Now `SMOKE` runs can at least do something real for the first domain.

Strong design rule

Keep check functions **small and explicit**.

This is good:

- `runArtifactLocationCheck`

- `runRequiredFilesCheck`
- `runEnvironmentContractCheck`
- `runStartupDeclarationCheck`

This is bad:

- one giant `evaluateEverythingAboutBuildIntegrity()` blob

Small checks make it much easier to:

- attach evidence later
- attach findings later
- debug failures
- display check-level detail in Run Detail

Important note

Right now this evaluator returns:

- empty `findings`
- empty `evidence`

That is fine for this stage.

Do **not** force findings/evidence too early if the contracts and persistence hooks for them are not wired yet.

The right progression is:

1. get the evaluator producing real domain results

2. add findings mapping
3. add evidence registration
4. enrich verdict logic

That order is much cleaner.

Recommended next improvement

The next thing to add to this evaluator after it works is a lightweight finding mapper.

Example pattern:

- missing env contract → `HIGH` deployment finding
- missing package.json → `CRITICAL` build/deployment finding
- missing startup script → `HIGH` deployment finding

But do that only after your finding creation service exists.

Recommended second improvement

You will soon want a shared check utility type so every evaluator doesn't reinvent status aggregation.

Later you should extract something like:

aggregateDomainStatusFromChecks(checks)

But do **not** extract it yet until you have 2–3 evaluators and the pattern is stable.

Right now duplication here is acceptable because it helps you discover the real abstraction.

AVCS Runtime 06 — Verdict Engine v1

AVCS Runtime 06 — Verdict Engine v1

Recommended location:

server/avcs/verdict/verdict-engine.ts

This is the right next extraction.

Your runner currently has embedded placeholder verdict logic. That is fine for scaffolding, but it will become a mess quickly once you add:

- findings
- blockers
- coverage sufficiency
- evidence completeness
- scoring
- policy rules

The verdict engine should be its own service now, while the logic is still small.

What Verdict Engine v1 should do

For v1, it should compute a verdict from:

- domain results
- blocked domain count
- failed domain count
- basic coverage sufficiency
- basic evidence completeness

It should return a valid `AVCSVerdict`.

For now, it does **not** need:

- finding-aware blocker mapping
- scoring
- policy packs
- override handling
- conditional pass logic

Those come later.

The goal of v1 is to create a clean seam between:

- orchestration
  and
- certification decision logic

File content

```
import { AVCSDomainResult } from "../../../shared/avcs/contracts/domain-result.contract";
import { AVCSRunManifest } from "../../../shared/avcs/contracts/run-manifest.contract";
import { AVCSVerdict } from "../../../shared/avcs/contracts/verdict.contract";
import {
 AVCSDomainStatus,
 AVCSVerdictState,
} from "../../../shared/avcs/enums/avcs.enums";

export interface AVCSVerdictEngineInput {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
}

export class AVCSVerdictEngine {
 compute(input: AVCSVerdictEngineInput): AVCSVerdict {
```

```
const { manifest, domainResults } = input;

const blockedResults = domainResults.filter(
  (result) => result.status === AVCSDomainStatus.BLOCKED
);

const failedResults = domainResults.filter(
  (result) => result.status === AVCSDomainStatus.FAILED
);

const partialResults = domainResults.filter(
  (result) => result.status === AVCSDomainStatus.PARTIAL
);

const coverageSufficient = this.isCoverageSufficient(
  manifest,
  domainResults
);

const evidenceComplete = this.isEvidenceComplete(
  manifest,
  domainResults
);

const state = this.determineVerdictState({
  blockedCount: blockedResults.length,
  failedCount: failedResults.length,
  partialCount: partialResults.length,
  coverageSufficient,
  evidenceComplete,
});

return {
  verdictId: `verdict_${manifest.runId}`,
  runId: manifest.runId,
  state,
  blockerCount: blockedResults.length + failedResults.length,
  warningCount: partialResults.length,
  coverageSufficient,
  evidenceComplete,
  linkedFindingIds: [],
  linkedEvidenceIds: [],
  reasoning: this.buildReasoning({
    state,
```

```
      blockedResults,
      failedResults,
      partialResults,
      coverageSufficient,
      evidenceComplete,
    }),
    decidedAt: new Date().toISOString(),
  };
}

protected determineVerdictState(input: {
  blockedCount: number;
  failedCount: number;
  partialCount: number;
  coverageSufficient: boolean;
  evidenceComplete: boolean;
}): AVCSVerdictState {
  const {
    blockedCount,
    failedCount,
    partialCount,
    coverageSufficient,
    evidenceComplete,
  } = input;

  if (!coverageSufficient || !evidenceComplete || blockedCount > 0) {
    return AVCSVerdictState.BLOCKED;
  }

  if (failedCount > 0) {
    return AVCSVerdictState.FAIL;
  }

  if (partialCount > 0) {
    return AVCSVerdictState.PASS_WITH_WARNINGS;
  }

  return AVCSVerdictState.PASS;
}

protected isCoverageSufficient(
  manifest: AVCSRunManifest,
  domainResults: AVCSDomainResult[]
): boolean {
```

```
  const includedDomains = manifest.includedDomains;
  const completedDomains = new Set(domainResults.map((r) => r.domain));

  return includedDomains.every((domain) => completedDomains.has(domain));
}

protected isEvidenceComplete(
  _manifest: AVCSRunManifest,
  _domainResults: AVCSDomainResult[]
): boolean {
  // v1 intentionally permissive until evidence registration is wired.
  return true;
}

protected buildReasoning(input: {
  state: AVCSVerdictState;
  blockedResults: AVCSDomainResult[];
  failedResults: AVCSDomainResult[];
  partialResults: AVCSDomainResult[];
  coverageSufficient: boolean;
  evidenceComplete: boolean;
}): string {
  const {
    state,
    blockedResults,
    failedResults,
    partialResults,
    coverageSufficient,
    evidenceComplete,
  } = input;

  if (state === AVCSVerdictState.BLOCKED) {
    const reasons: string[] = [];

    if (!coverageSufficient) {
      reasons.push("required domain coverage was incomplete");
    }

    if (!evidenceComplete) {
      reasons.push("required evidence was incomplete");
    }

    if (blockedResults.length > 0) {
      reasons.push(
```

```
        `blocked domain results were present (${blockedResults
          .map((r) => r.domain)
          .join(", ")})`
      );
    }

    return `Run was blocked because ${reasons.join("; ")}.`;
  }

  if (state === AVCSVerdictState.FAIL) {
    return `Run failed because one or more domains failed (${failedResults
      .map((r) => r.domain)
      .join(", ")}).`;
  }

  if (state === AVCSVerdictState.PASS_WITH_WARNINGS) {
    return `Run passed with warnings because partial domain results were present
(${partialResults
      .map((r) => r.domain)
      .join(", ")}).`;
  }

  return "All included domains completed successfully with sufficient coverage and no blocking
conditions.";
 }
}
```

Why this shape is correct

This gives you a dedicated service for certification decision logic without overengineering it.

It already separates:

- verdict state selection
- coverage sufficiency evaluation
- evidence completeness evaluation
- reasoning generation

That means you now have a clean place to expand later.

Verdict rules in v1

The current decision model is:

`BLOCKED`

If:

- coverage is insufficient
- evidence is incomplete
- any domain is blocked

`FAIL`

If:

- at least one domain failed
- and no blocked condition exists

`PASS_WITH_WARNINGS`

If:

- no blocked domains
- no failed domains
- but at least one domain is partial

`PASS`

If:

- all included domains completed successfully

- no blocked/failed/partial conditions
- coverage sufficient
- evidence complete

That is a solid MVP decision model.

Strong design rule

Do not let the runner decide verdict logic anymore.

The runner should do this:

const verdict = verdictEngine.compute({ manifest, domainResults });

And stop there.

That keeps the runner focused on:

- lifecycle
- sequencing
- evaluator invocation

while the verdict engine owns:

- certification outcome logic

That separation matters a lot.

How to wire it into the runner

In `server/avcs/core/avcs-runner.ts`, replace the embedded finalization logic.

Add import

```
import { AVCSVerdictEngine } from "../verdict/verdict-engine";
```

Add field

```
private readonly verdictEngine: AVCSVerdictEngine;
```

Update constructor

```
constructor(
 evaluators: AVCSDomainEvaluator[] = [],
 verdictEngine: AVCSVerdictEngine = new AVCSVerdictEngine()
) {
 this.evaluators = new Map(
   evaluators.map((evaluator) => [evaluator.domain, evaluator])
 );
 this.verdictEngine = verdictEngine;
}
```

Replace `finalizeVerdict`

Replace:

```
const verdict = await this.finalizeVerdict(manifest, domainResults);
```

with:

```
const verdict = this.verdictEngine.compute({
 manifest,
 domainResults,
});
```

Remove the old embedded `finalizeVerdict()` method

You will no longer need it.

Why `isEvidenceComplete()` is permissive right now

Because your evidence layer is not wired yet.

If you made evidence strict now, every run would become artificially blocked.

So v1 should deliberately return:

return true;

until:

- evidence registry exists
- domain evaluators emit evidence
- finalization can verify required proof

That is the correct staged build order.

Recommended improvement

Add a structured summary object internally so later expansion is easier.

You do not need to expose it yet, but you can add this helper:

```
protected summarizeDomainOutcomes(domainResults: AVCSDomainResult[]) {
 return {
   blocked: domainResults.filter((r) => r.status === AVCSDomainStatus.BLOCKED),
   failed: domainResults.filter((r) => r.status === AVCSDomainStatus.FAILED),
   partial: domainResults.filter((r) => r.status === AVCSDomainStatus.PARTIAL),
   passed: domainResults.filter((r) => r.status === AVCSDomainStatus.PASSED),
 };
}
```

That will make future logic cleaner when you add:

- scoring
- finding-linked blockers
- policy gates

Not required yet, but a good next refinement.

Recommended second improvement

Make coverage sufficiency stricter later.

Right now it only checks:

- every included domain produced a domain result

Later it should also check:

- required domains specifically completed
- blocked domains do not count as sufficient
- partial domains may or may not count depending on policy
- workflow coverage exists, not just domain presence

For v1, the current check is appropriate.

Example outcome

If Full Certification includes:

- Build Integrity = PASSED
- Functional Correctness = PASSED
- Security Posture = FAILED
- Performance & Stability = PASSED
- Deployment Readiness = PASSED

Then the verdict engine returns:

- `state = FAIL`
- `blockerCount = 1`

- `warningCount = 0`

Reasoning:

> Run failed because one or more domains failed (SECURITY_POSTURE).

That is exactly the kind of clean output you want.

AVCS Runtime 07 — History Hook Layer

AVCS Runtime 07 — History Hook Layer

Recommended location:

server/avcs/history/history-hook-service.ts

This is the right next step.

At this point AVCS can:

- define runs
- plan domains
- execute domains
- compute a verdict

But it still does not **remember what happened** in an append-only way.

The History Hook Layer is the first audit spine. It gives the runner a clean place to emit lifecycle events without hardwiring persistence details into orchestration logic.

What this layer should do

It should provide a simple interface for recording events such as:

- run created
- run status changed
- domain started
- domain completed
- finding created later
- evidence registered later

- coverage assembled later
- verdict finalized
- rerun created later
- override applied later

For now, the first version should focus on the events the current runner can actually emit:

1. `RUN_CREATED`
2. `RUN_STATUS_CHANGED`
3. `DOMAIN_COMPLETED`
4. `VERDICT_FINALIZED`

That is enough to make AVCS auditable immediately.

File content

```
import {
 AVCSHistoryRecord,
 AVCSHistoryRelatedObject,
 AVCSRunStatusChangeDetails,
 AVCSVerdictFinalizedDetails,
} from "../../../shared/avcs/contracts/history-record.contract";
import { AVCSDomainResult } from "../../../shared/avcs/contracts/domain-result.contract";
import { AVCSRunManifest } from "../../../shared/avcs/contracts/run-manifest.contract";
import { AVCSVerdict } from "../../../shared/avcs/contracts/verdict.contract";
import {
 AVCSHistoryEventType,
 AVCSHistoryObjectType,
 AVCSRunStatus,
} from "../../../shared/avcs/enums/avcs.enums";
import { AVCSHistoryActor } from "../../../shared/avcs/types/avcs.shared";

export interface AVCSHistoryStore {
```

```
 append(record: AVCSHistoryRecord): Promise<void>;
}

export class InMemoryAVCSHistoryStore implements AVCSHistoryStore {
 private readonly records: AVCSHistoryRecord[] = [];

 async append(record: AVCSHistoryRecord): Promise<void> {
   this.records.push(record);
 }

 getAll(): AVCSHistoryRecord[] {
   return [...this.records];
 }

 getByRunId(runId: string): AVCSHistoryRecord[] {
   return this.records.filter((record) => record.runId === runId);
 }
}

export class AVCSHistoryHookService {
 constructor(private readonly store: AVCSHistoryStore) {}

 async recordRunCreated(manifest: AVCSRunManifest): Promise<void> {
   await this.store.append({
     historyId: this.generateHistoryId(manifest.runId, "run_created"),
     runId: manifest.runId,
     eventType: AVCSHistoryEventType.RUN_CREATED,
     timestamp: new Date().toISOString(),
     actor: this.toHistoryActor(manifest.actor),
     summary: "Certification run was created.",
     relatedObject: this.relatedObject(
       AVCSHistoryObjectType.RUN,
       manifest.runId
     ),
     details: {
       runType: manifest.runType,
       includedDomains: manifest.includedDomains,
       policySnapshotId: manifest.policySnapshotId,
       buildId: manifest.target.buildId,
       buildVersion: manifest.target.buildVersion,
     },
   });
 }
```

```
async recordRunStatusChanged(
  manifest: AVCSRunManifest,
  fromStatus: AVCSRunStatus,
  toStatus: AVCSRunStatus
): Promise<void> {
  const details: AVCSRunStatusChangeDetails = {
    fromStatus,
    toStatus,
  };

  await this.store.append({
    historyId: this.generateHistoryId(
      manifest.runId,
      `status_${fromStatus}_to_${toStatus}`
    ),
    runId: manifest.runId,
    eventType: AVCSHistoryEventType.RUN_STATUS_CHANGED,
    timestamp: new Date().toISOString(),
    actor: this.toHistoryActor(manifest.actor),
    summary: `Run status changed from ${fromStatus} to ${toStatus}.`,
    relatedObject: this.relatedObject(
      AVCSHistoryObjectType.RUN,
      manifest.runId
    ),
    details,
  });
}

async recordDomainCompleted(
  manifest: AVCSRunManifest,
  result: AVCSDomainResult
): Promise<void> {
  await this.store.append({
    historyId: this.generateHistoryId(
      manifest.runId,
      `domain_completed_${result.domain}`
    ),
    runId: manifest.runId,
    eventType: AVCSHistoryEventType.DOMAIN_COMPLETED,
    timestamp: new Date().toISOString(),
    actor: this.toHistoryActor(manifest.actor),
    summary: `${result.domain} domain completed with status ${result.status}.`,
    relatedObject: this.relatedObject(
      AVCSHistoryObjectType.DOMAIN_RESULT,
```

```
      result.domainResultId
    ),
    details: {
      domain: result.domain,
      status: result.status,
      coverageState: result.coverageState,
      blockerDetected: result.blockerDetected ?? false,
      checkCount: result.checks.length,
    },
  });
}

async recordVerdictFinalized(
  manifest: AVCSRunManifest,
  verdict: AVCSVerdict
): Promise<void> {
  const details: AVCSVerdictFinalizedDetails = {
    verdictState: verdict.state,
    blockerCount: verdict.blockerCount,
    warningCount: verdict.warningCount,
  };

  await this.store.append({
    historyId: this.generateHistoryId(manifest.runId, "verdict_finalized"),
    runId: manifest.runId,
    eventType: AVCSHistoryEventType.VERDICT_FINALIZED,
    timestamp: new Date().toISOString(),
    actor: this.toHistoryActor(manifest.actor),
    summary: `Certification verdict finalized as ${verdict.state}.`,
    relatedObject: this.relatedObject(
      AVCSHistoryObjectType.VERDICT,
      verdict.verdictId
    ),
    details: {
      ...details,
      coverageSufficient: verdict.coverageSufficient,
      evidenceComplete: verdict.evidenceComplete,
      overrideRequired: verdict.overrideRequired ?? false,
    },
  });
}

private toHistoryActor(actor: {
  actorId: string;
```

```
  actorType: AVCSHistoryActor["actorType"];
  name?: string;
}): AVCSHistoryActor {
  return {
    actorId: actor.actorId,
    actorType: actor.actorType,
    name: actor.name,
  };
}

private relatedObject(
  objectType: AVCSHistoryObjectType,
  objectId: string
): AVCSHistoryRelatedObject {
  return {
    objectType,
    objectId,
  };
}

private generateHistoryId(runId: string, suffix: string): string {
  return `history_${runId}_${suffix}_${Date.now()}`;
}
}
```

Why this shape is correct

This gives AVCS a clean hook layer with:

- a small persistence interface
- a default in-memory implementation
- explicit event-writing methods
- append-only behavior
- no runner-specific persistence logic leaking everywhere

That is exactly what you want.

The runner should not know how history is stored.
It should only know how to say:

- record run created
- record status changed
- record domain completed
- record verdict finalized

Why an interface matters now

This line is important:

```
export interface AVCSHistoryStore {
 append(record: AVCSHistoryRecord): Promise<void>;
}
```

That lets you start with:

- in-memory store for development/tests

and later swap to:

- file-backed store
- database-backed store
- append-only event log
- audit storage adapter

without changing runner orchestration.

That is the right abstraction boundary.

Strong design rule

History hooks should be **append-only event writers**, not mutable state updaters.

Good:

recordRunStatusChanged(...)

Bad:

updateRunHistoryToLatest(...)

You want event-shaped history, not rewrite-shaped history.

That is what preserves audit truth.

How to wire it into the runner

Now update `server/avcs/core/avcs-runner.ts`.

Add import

```
import {
 AVCSHistoryHookService,
 InMemoryAVCSHistoryStore,
} from "../history/history-hook-service";
```

Add field

```
private readonly historyHooks: AVCSHistoryHookService;
```

Update constructor

```
constructor(
 evaluators: AVCSDomainEvaluator[] = [],
 verdictEngine: AVCSVerdictEngine = new AVCSVerdictEngine(),
 historyHooks: AVCSHistoryHookService = new AVCSHistoryHookService(
   new InMemoryAVCSHistoryStore()
 )
) {
 this.evaluators = new Map(
   evaluators.map((evaluator) => [evaluator.domain, evaluator])
 );
 this.verdictEngine = verdictEngine;
 this.historyHooks = historyHooks;
}
```

Record run created

Right after:

```
const { manifest, plan } = createAVCSRunManifestWithPlan(input);
```

add:

```
await this.historyHooks.recordRunCreated(manifest);
```

Update status transition helper

Change `transitionManifestStatus` to record history.

```
protected async transitionManifestStatus(
 manifest: AVCSRunManifest,
 nextStatus: AVCSRunStatus
): Promise<void> {
 if (isTerminalRunStatus(manifest.status)) {
   throw new Error(
     `Cannot transition terminal AVCS run status: ${manifest.status} -> ${nextStatus}`
   );
 }

 const previousStatus = manifest.status;

 assertValidRunStatusTransition(previousStatus, nextStatus);
 manifest.status = nextStatus;

 await this.historyHooks.recordRunStatusChanged(
   manifest,
   previousStatus,
   nextStatus
 );
}
```

Record domain completion

Inside the domain loop, after pushing result:

```
await this.historyHooks.recordDomainCompleted(manifest, result);
```

Record verdict finalized

After computing verdict:

```
await this.historyHooks.recordVerdictFinalized(manifest, verdict);
```

What this gives you immediately

Once wired, every run now produces an append-only event trail like:

1. `RUN_CREATED`
2. `RUN_STATUS_CHANGED CREATED -> PREFLIGHT`
3. `RUN_STATUS_CHANGED PREFLIGHT -> RUNNING`
4. `DOMAIN_COMPLETED BUILD_INTEGRITY`
5. `DOMAIN_COMPLETED FUNCTIONAL_CORRECTNESS`
6. `...`
7. `RUN_STATUS_CHANGED RUNNING -> FINALIZING`
8. `VERDICT_FINALIZED`
9. `RUN_STATUS_CHANGED FINALIZING -> COMPLETED`

That is already enough to power:

- a basic History panel
- a run timeline
- audit visibility
- rerun lineage later

Recommended improvement

Add a retrieval service later instead of bloating the hook layer.

Do not turn this file into:

- query engine
- filtering engine
- comparison engine

Keep it as:

- append hooks
- maybe basic store abstraction

Later create:

- `run-history-service.ts`
- `comparison-service.ts`

That separation will hold up better.

Recommended second improvement

You will soon want a dedicated `recordDomainStarted()` hook too, but it is not necessary yet.

Why it can wait:

- your current runner only really needs completed domain milestones to be useful

Why you will want it later:

- live execution timeline
- progress tracking

- long-running domain visibility

For now, `DOMAIN_COMPLETED` is enough.

AVCS Runtime 08 — Findings Service v1

AVCS Runtime 08 — Findings Service v1

Recommended location:

server/avcs/findings/findings-service.ts

This is the right next step.

Right now AVCS can:

- run domains
- return domain results
- compute a verdict
- write history

But it still does not produce proper **issue objects**.
That means the system knows something failed, but it does not yet know how to express:

- what failed
- how severe it is
- what surface it affected
- whether it blocks release
- what remediation should happen

That is exactly what the Findings Service should do.

What Findings Service v1 should do

For v1, it should:

- inspect domain results

- inspect failed or blocked checks
- convert them into `AVCSFinding` objects
- attach severity, impact class, disposition, affected surface, and certification effect
- return a normalized findings list

It should start with one concrete mapper:

- **Build Integrity findings**

That is enough to make the system real without trying to solve every domain at once.

File content

```
import { AVCSDomainResult } from "../../../shared/avcs/contracts/domain-result.contract";
import { AVCSRunManifest } from "../../../shared/avcs/contracts/run-manifest.contract";
import {
 AVCSCertificationEffect,
 AVCSFinding,
} from "../../../shared/avcs/contracts/finding.contract";
import {
 AVCSDomain,
 AVCSDomainCheckStatus,
 AVCSAffectedSurfaceType,
 AVCSFindingDisposition,
 AVCSFindingImpactClass,
 AVCSFindingSeverity,
} from "../../../shared/avcs/enums/avcs.enums";

export interface AVCSFindingsServiceInput {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
}

export class AVCSFindingsService {
 buildFindings(input: AVCSFindingsServiceInput): AVCSFinding[] {
   const findings: AVCSFinding[] = [];
```

```
  for (const domainResult of input.domainResults) {
    switch (domainResult.domain) {
      case AVCSDomain.BUILD_INTEGRITY:
        findings.push(
          ...this.buildIntegrityFindings(input.manifest, domainResult)
        );
        break;

      default:
        break;
    }
  }

  return findings;
}

protected buildIntegrityFindings(
  manifest: AVCSRunManifest,
  domainResult: AVCSDomainResult
): AVCSFinding[] {
  const findings: AVCSFinding[] = [];

  for (const check of domainResult.checks) {
    if (
      check.status !== AVCSDomainCheckStatus.FAILED &&
      check.status !== AVCSDomainCheckStatus.BLOCKED
    ) {
      continue;
    }

    switch (check.checkKey) {
      case "artifact_location":
        findings.push(
          this.makeFinding({
            manifest,
            domain: AVCSDomain.BUILD_INTEGRITY,
            severity: AVCSFindingSeverity.CRITICAL,
            impactClass: AVCSFindingImpactClass.DEPLOYMENT,
            disposition: AVCSFindingDisposition.BLOCKER,
            title: "Artifact location is unavailable",
            summary: check.summary,
            description:
              "The declared artifact location could not be accessed, which prevents the build from being evaluated reliably.",
```

```
            affectedSurface: {
              type: AVCSAffectedSurfaceType.BUILD,
              key: manifest.target.artifactLocation,
              label: "Artifact Location",
            },
            certificationEffect: {
              blocksRelease: true,
              requiresReview: true,
              affectsScore: true,
              notes:
                "Build cannot be certified if the artifact location is unavailable.",
            },
            remediation:
              "Verify the artifact path exists, is readable by AVCS, and points to the correct build
output.",
          })
        );
        break;

      case "required_files":
        findings.push(
          this.makeFinding({
            manifest,
            domain: AVCSDomain.BUILD_INTEGRITY,
            severity: AVCSFindingSeverity.CRITICAL,
            impactClass: AVCSFindingImpactClass.DEPLOYMENT,
            disposition: AVCSFindingDisposition.BLOCKER,
            title: "Required build files are missing",
            summary: check.summary,
            description:
              "One or more required files were missing from the build output, making the artifact
structurally incomplete.",
            affectedSurface: {
              type: AVCSAffectedSurfaceType.BUILD,
              key: "required_files",
              label: "Required Files",
            },
            certificationEffect: {
              blocksRelease: true,
              requiresReview: true,
              affectsScore: true,
              notes:
                "A build missing required files cannot be considered structurally valid.",
            },
```

```
            remediation:
              "Ensure the build output includes all required files before rerunning Build Integrity
validation.",
          })
        );
        break;

      case "environment_contract":
        findings.push(
          this.makeFinding({
            manifest,
            domain: AVCSDomain.BUILD_INTEGRITY,
            severity: AVCSFindingSeverity.HIGH,
            impactClass: AVCSFindingImpactClass.DEPLOYMENT,
            disposition: AVCSFindingDisposition.BLOCKER,
            title: "Environment contract is missing",
            summary: check.summary,
            description:
              "No environment contract surface was found for the build output, making deployment
and startup assumptions unclear.",
            affectedSurface: {
              type: AVCSAffectedSurfaceType.CONFIG,
              key: "environment_contract",
              label: "Environment Contract",
            },
            certificationEffect: {
              blocksRelease: true,
              requiresReview: true,
              affectsScore: true,
              notes:
                "Deployment-readiness cannot be trusted without an environment contract surface.",
            },
            remediation:
              "Add an environment contract file such as env.contract.json, .env.example, or
.env.schema.json.",
          })
        );
        break;

      case "startup_declaration":
        findings.push(
          this.makeFinding({
            manifest,
            domain: AVCSDomain.BUILD_INTEGRITY,
```

```
            severity: AVCSFindingSeverity.HIGH,
            impactClass: AVCSFindingImpactClass.DEPLOYMENT,
            disposition: AVCSFindingDisposition.BLOCKER,
            title: "Startup declaration is missing or unreadable",
            summary: check.summary,
            description:
              "AVCS could not validate a startup-relevant script declaration for the build artifact.",
            affectedSurface: {
              type: AVCSAffectedSurfaceType.CONFIG,
              key: "startup_declaration",
              label: "Startup Declaration",
            },
            certificationEffect: {
              blocksRelease: true,
              requiresReview: true,
              affectsScore: true,
              notes:
                "A build without a valid startup declaration cannot be trusted for deployment
progression.",
            },
            remediation:
              "Ensure package.json is readable and contains a start, dev, or build script appropriate
for the target build.",
          })
        );
        break;

      default:
        findings.push(
          this.makeFinding({
            manifest,
            domain: AVCSDomain.BUILD_INTEGRITY,
            severity: AVCSFindingSeverity.MEDIUM,
            impactClass: AVCSFindingImpactClass.RELIABILITY,
            disposition: AVCSFindingDisposition.WARNING,
            title: `Build Integrity issue in ${check.name}`,
            summary: check.summary,
            description:
              "A Build Integrity check reported a non-passing result that should be reviewed.",
            affectedSurface: {
              type: AVCSAffectedSurfaceType.DOMAIN,
              key: check.checkKey,
              label: check.name,
            },
```

```
          certificationEffect: {
            blocksRelease: false,
            requiresReview: true,
            affectsScore: true,
          },
          remediation:
            "Review the failed check details and correct the underlying build integrity issue.",
        })
      );
      break;
    }
  }

  return findings;
}

protected makeFinding(input: {
  manifest: AVCSRunManifest;
  domain: AVCSDomain;
  severity: AVCSFindingSeverity;
  impactClass: AVCSFindingImpactClass;
  disposition: AVCSFindingDisposition;
  title: string;
  summary: string;
  description?: string;
  affectedSurface: {
    type: AVCSAffectedSurfaceType;
    key: string;
    label?: string;
  };
  certificationEffect: AVCSCertificationEffect;
  remediation?: string;
}): AVCSFinding {
  return {
    findingId: this.generateFindingId(
      input.manifest.runId,
      input.domain,
      input.title
    ),
    runId: input.manifest.runId,
    domain: input.domain,
    severity: input.severity,
    impactClass: input.impactClass,
    disposition: input.disposition,
```

```
    title: input.title,
    summary: input.summary,
    description: input.description,
    affectedSurface: input.affectedSurface,
    evidence: [],
    certificationEffect: input.certificationEffect,
    remediation: input.remediation,
    metadata: {},
  };
}

protected generateFindingId(
  runId: string,
  domain: AVCSDomain,
  title: string
): string {
  const slug = title
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, "_")
    .replace(/^_+|_+$/g, "")
    .slice(0, 48);

  return `finding_${runId}_${domain}_${slug}`;
}
}
```

Why this shape is correct

This creates a clean Findings Service that is:

- domain-aware
- mapper-based
- easy to extend
- independent from the runner
- independent from the verdict engine

That is exactly the right architecture.

The runner should not know how to describe issues.
The findings service should.

What v1 intentionally does not do yet

It does **not** yet:

- attach evidence IDs
- persist findings
- write finding history records
- deduplicate overlapping findings
- map every domain

That is correct for this stage.

The goal right now is to establish the proper issue-object layer with one concrete domain implementation.

Strong design rule

Do not generate findings inside the evaluator forever.

A little duplication early is fine, but long-term the evaluator should output:

- check results
- status
- metadata

And the Findings Service should translate those outputs into operator-facing issue objects.

Why:

- keeps evaluators focused on evaluation
- keeps issue language consistent
- makes disposition/severity policy easier to tune later
- avoids domain logic being scattered across multiple services

That separation is the right long-term move.

How to wire this into the runner

You now need the runner to produce findings alongside domain results.

Step 1: update runner result type

In `server/avcs/core/avcs-runner.ts`:

```
import { AVCSFinding } from "../../../shared/avcs/contracts/finding.contract";
```

Then update:

```
export interface AVCSRunExecutionResult {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
 findings: AVCSFinding[];
 verdict: AVCSVerdict;
}
```

Step 2: inject Findings Service

Add import:

```
import { AVCSFindingsService } from "../findings/findings-service";
```

Add field:

```
private readonly findingsService: AVCSFindingsService;
```

Update constructor:

```
constructor(
```

```
 evaluators: AVCSDomainEvaluator[] = [],
 verdictEngine: AVCSVerdictEngine = new AVCSVerdictEngine(),
 historyHooks: AVCSHistoryHookService = new AVCSHistoryHookService(
   new InMemoryAVCSHistoryStore()
 ),
 findingsService: AVCSFindingsService = new AVCSFindingsService()
) {
 this.evaluators = new Map(
   evaluators.map((evaluator) => [evaluator.domain, evaluator])
 );
 this.verdictEngine = verdictEngine;
 this.historyHooks = historyHooks;
 this.findingsService = findingsService;
}
```

Step 3: build findings before verdict

After domain execution and before verdict computation:

```
const findings = this.findingsService.buildFindings({
 manifest,
 domainResults,
});
```

Step 4: return findings in all branches

Blocked preflight branch:

```
return {
 manifest,
 domainResults,
 findings: [],
 verdict,
};
```

Blocked domain branch:
 build findings from whatever results exist so far:

```
const findings = this.findingsService.buildFindings({
 manifest,
 domainResults,
});

return {
 manifest,
```

```
 domainResults,
 findings,
 verdict,
};
```

Final branch:

```
const findings = this.findingsService.buildFindings({
 manifest,
 domainResults,
});

const verdict = this.verdictEngine.compute({
 manifest,
 domainResults,
 findings,
});
```

and return them.

Very important follow-up change

Now that verdict logic can be issue-aware, you should upgrade the verdict engine input.

In `server/avcs/verdict/verdict-engine.ts`, add:

```
import { AVCSFinding } from "../../../shared/avcs/contracts/finding.contract";
```

Update input:

```
export interface AVCSVerdictEngineInput {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
 findings: AVCSFinding[];
}
```

Then use findings to compute blocker/warning counts instead of only domain counts.

That is the next refinement inside the same step.

Recommended Verdict Engine adjustment

Here is the core change you should make.

Replace blocker/warning logic with finding-aware logic

Inside `compute()`:

```
const blockerFindings = findings.filter(
 (finding) => finding.certificationEffect.blocksRelease
);

const warningFindings = findings.filter(
 (finding) =>
   !finding.certificationEffect.blocksRelease &&
   finding.certificationEffect.requiresReview
);
```

Then:

- `blockerCount = blockerFindings.length`
- `warningCount = warningFindings.length`

And in verdict state selection:

- blocker findings should force at least `FAIL` or `BLOCKED` depending on completeness

That is much closer to true AVCS behavior.

AVCS Runtime 09 — Evidence Service v1

AVCS Runtime 09 — Evidence Service v1

Recommended location:

server/avcs/evidence/evidence-service.ts

This is the correct next step.

Right now AVCS can:

- execute runs
- produce domain results
- generate findings
- compute verdicts
- write history

But findings still have empty evidence arrays, and verdict evidence is still effectively placeholder. That means AVCS can describe issues, but it cannot yet **prove** them in a structured way.

The Evidence Service is the first real proof layer.

What Evidence Service v1 should do

For v1, it should:

- inspect domain results and check results
- generate structured `AVCSEvidenceItem` records
- attach evidence IDs to checks where possible
- attach evidence IDs to findings where possible
- return a normalized evidence list

This first version should stay practical and narrow.

It does **not** need to store screenshots or traces yet.
It just needs to make AVCS evidence-shaped and evidence-linked.

The best v1 evidence sources are:

- domain summary evidence
- failed/blocked check evidence
- check metadata as structured JSON evidence

That is enough to make the system meaningfully certification-oriented.

File content

```
import { AVCSDomainResult } from "../../../shared/avcs/contracts/domain-result.contract";
import { AVCSEvidenceItem } from "../../../shared/avcs/contracts/evidence.contract";
import { AVCSFinding } from "../../../shared/avcs/contracts/finding.contract";
import { AVCSRunManifest } from "../../../shared/avcs/contracts/run-manifest.contract";
import {
 AVCSDomainCheckStatus,
 AVCSEvidenceSourceType,
 AVCSEvidenceType,
} from "../../../shared/avcs/enums/avcs.enums";

export interface AVCSEvidenceServiceInput {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
 findings: AVCSFinding[];
}

export interface AVCSEvidenceBuildResult {
 evidence: AVCSEvidenceItem[];
 findings: AVCSFinding[];
 domainResults: AVCSDomainResult[];
}

export class AVCSEvidenceService {
 buildEvidence(input: AVCSEvidenceServiceInput): AVCSEvidenceBuildResult {
```

```
const evidence: AVCSEvidenceItem[] = [];
const findings = input.findings.map((finding) => ({
  ...finding,
  evidence: [...finding.evidence],
}));

const domainResults = input.domainResults.map((domainResult) => ({
  ...domainResult,
  evidence: [...domainResult.evidence],
  checks: domainResult.checks.map((check) => ({
    ...check,
    evidence: check.evidence ? [...check.evidence] : [],
  })),
}));

for (const domainResult of domainResults) {
  const domainSummaryEvidence = this.makeDomainSummaryEvidence(
    input.manifest,
    domainResult
  );

  evidence.push(domainSummaryEvidence);
  domainResult.evidence.push({ evidenceId: domainSummaryEvidence.evidenceId });

  for (const check of domainResult.checks) {
    if (
      check.status !== AVCSDomainCheckStatus.FAILED &&
      check.status !== AVCSDomainCheckStatus.BLOCKED &&
      check.status !== AVCSDomainCheckStatus.PARTIAL
    ) {
      continue;
    }

    const checkEvidence = this.makeCheckEvidence(
      input.manifest,
      domainResult,
      check
    );

    evidence.push(checkEvidence);

    if (!check.evidence) {
      check.evidence = [];
    }
```

```
      check.evidence.push({ evidenceId: checkEvidence.evidenceId });
      domainResult.evidence.push({ evidenceId: checkEvidence.evidenceId });

      const matchedFindings = findings.filter((finding) =>
        this.doesFindingMatchCheck(finding, check.checkKey)
      );

      for (const finding of matchedFindings) {
        finding.evidence.push({ evidenceId: checkEvidence.evidenceId });
        checkEvidence.linkedFindingIds = [
          ...(checkEvidence.linkedFindingIds ?? []),
          finding.findingId,
        ];
      }
    }
  }

  return {
    evidence,
    findings,
    domainResults,
  };
}

protected makeDomainSummaryEvidence(
  manifest: AVCSRunManifest,
  domainResult: AVCSDomainResult
): AVCSEvidenceItem {
  return {
    evidenceId: this.generateEvidenceId(
      manifest.runId,
      domainResult.domain,
      "domain_summary"
    ),
    runId: manifest.runId,
    type: AVCSEvidenceType.SUMMARY,
    domain: domainResult.domain,
    label: `${domainResult.domain} domain summary`,
    summary: domainResult.summary,
    storageRef: this.makeInlineStorageRef(
      manifest.runId,
      domainResult.domain,
      "domain_summary"
```

```
    ),
    source: {
      sourceType: AVCSEvidenceSourceType.DOMAIN_EVALUATOR,
      sourceKey: domainResult.domain,
      sourceLabel: `${domainResult.domain} Evaluator`,
    },
    linkedFindingIds: [],
    linkedCheckIds: [],
    createdAt: new Date().toISOString(),
    finalized: false,
    metadata: {
      status: domainResult.status,
      coverageState: domainResult.coverageState,
      blockerDetected: domainResult.blockerDetected ?? false,
    },
  };
}

protected makeCheckEvidence(
  manifest: AVCSRunManifest,
  domainResult: AVCSDomainResult,
  check: AVCSDomainResult["checks"][number]
): AVCSEvidenceItem {
  return {
    evidenceId: this.generateEvidenceId(
      manifest.runId,
      domainResult.domain,
      check.checkKey
    ),
    runId: manifest.runId,
    type: AVCSEvidenceType.JSON,
    domain: domainResult.domain,
    label: `${check.name} evidence`,
    summary: check.summary,
    storageRef: this.makeInlineStorageRef(
      manifest.runId,
      domainResult.domain,
      check.checkKey
    ),
    source: {
      sourceType: AVCSEvidenceSourceType.CHECK,
      sourceKey: check.checkKey,
      sourceLabel: check.name,
    },
```

```
    linkedFindingIds: [],
    linkedCheckIds: [check.checkId],
    createdAt: new Date().toISOString(),
    finalized: false,
    metadata: {
      status: check.status,
      summary: check.summary,
      checkMetadata: check.metadata ?? {},
      domainStatus: domainResult.status,
    },
  };
}

protected doesFindingMatchCheck(
  finding: AVCSFinding,
  checkKey: string
): boolean {
  const surfaceKey = finding.affectedSurface.key;

  return (
    surfaceKey === checkKey ||
    surfaceKey.includes(checkKey) ||
    checkKey.includes(surfaceKey)
  );
}

protected generateEvidenceId(
  runId: string,
  domain: string,
  suffix: string
): string {
  return `evidence_${runId}_${domain}_${suffix}`;
}

protected makeInlineStorageRef(
  runId: string,
  domain: string,
  suffix: string
): string {
  return `inline://avcs/${runId}/${domain}/${suffix}`;
}
}
```

Why this shape is correct

This gives AVCS a dedicated proof-building layer that:

- normalizes evidence creation
- links evidence to checks
- links evidence to findings
- enriches domain results with evidence refs
- does not force real file storage yet

That is exactly the right v1 move.

You are making evidence **structural** before making it **heavy**.

Why `inline://` storage refs are acceptable for v1

Right now, you do not yet have:

- evidence storage registry
- file-backed artifact writing
- screenshot capture
- trace persistence

So for v1, a synthetic storage reference like:

inline://avcs/runId/domain/check

is fine.

It gives every evidence item:

- a stable ID

- a stable storage reference shape
- a path to future persistence

Later you can swap this to:

- filesystem paths
- object storage keys
- DB blob references
- artifact registry paths

without changing the rest of the AVCS evidence model.

Strong design rule

Do not let evaluators create heavyweight evidence persistence directly yet.

For now, evaluators should emit:

- check summaries
- check metadata
- statuses

Then Evidence Service turns that into structured evidence.

Why:

- keeps evaluators simpler
- keeps evidence formatting consistent
- gives you one central place to improve evidence rules later

That separation is the right design.

How to wire this into the runner

Now update `server/avcs/core/avcs-runner.ts`.

Step 1: import service and contracts

```
import { AVCSEvidenceItem } from "../../../shared/avcs/contracts/evidence.contract";
import { AVCSEvidenceService } from "../evidence/evidence-service";
```

Step 2: update execution result type

```
export interface AVCSRunExecutionResult {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
 findings: AVCSFinding[];
 evidence: AVCSEvidenceItem[];
 verdict: AVCSVerdict;
}
```

Step 3: inject Evidence Service

Add field:

```
private readonly evidenceService: AVCSEvidenceService;
```

Update constructor:

```
constructor(
 evaluators: AVCSDomainEvaluator[] = [],
 verdictEngine: AVCSVerdictEngine = new AVCSVerdictEngine(),
 historyHooks: AVCSHistoryHookService = new AVCSHistoryHookService(
   new InMemoryAVCSHistoryStore()
 ),
 findingsService: AVCSFindingsService = new AVCSFindingsService(),
 evidenceService: AVCSEvidenceService = new AVCSEvidenceService()
) {
 this.evaluators = new Map(
   evaluators.map((evaluator) => [evaluator.domain, evaluator])
 );
 this.verdictEngine = verdictEngine;
 this.historyHooks = historyHooks;
 this.findingsService = findingsService;
 this.evidenceService = evidenceService;
```

```
}
```

Step 4: build findings, then evidence, then verdict

After domain execution:

```
const rawFindings = this.findingsService.buildFindings({
 manifest,
 domainResults,
});

const evidenceBuild = this.evidenceService.buildEvidence({
 manifest,
 domainResults,
 findings: rawFindings,
});

const findings = evidenceBuild.findings;
const enrichedDomainResults = evidenceBuild.domainResults;
const evidence = evidenceBuild.evidence;
```

Then pass `enrichedDomainResults` and `findings` to verdict engine.

Step 5: return evidence everywhere

Preflight blocked branch:

```
return {
 manifest,
 domainResults,
 findings: [],
 evidence: [],
 verdict,
};
```

Blocked domain branch:
 use evidence service on whatever exists so far.

Final branch:
 return full evidence set.

Required Verdict Engine update

Now your verdict engine should become evidence-aware for real.

In `server/avcs/verdict/verdict-engine.ts`, update the input:

```
import { AVCSFinding } from "../../../shared/avcs/contracts/finding.contract";
import { AVCSEvidenceItem } from "../../../shared/avcs/contracts/evidence.contract";

export interface AVCSVerdictEngineInput {
 manifest: AVCSRunManifest;
 domainResults: AVCSDomainResult[];
 findings: AVCSFinding[];
 evidence: AVCSEvidenceItem[];
}
```

Then replace the placeholder `isEvidenceComplete()` logic with something simple but real.

Verdict Engine v1.1 evidence rule

A good first evidence completeness rule is:

- every blocker finding must have at least one evidence ref
- every failed or blocked domain must have at least one evidence ref

Example implementation:

```
protected isEvidenceComplete(
 _manifest: AVCSRunManifest,
 domainResults: AVCSDomainResult[],
 findings: AVCSFinding[]
): boolean {
 const blockerFindings = findings.filter(
   (finding) => finding.certificationEffect.blocksRelease
 );

 const blockerFindingsHaveEvidence = blockerFindings.every(
   (finding) => finding.evidence.length > 0
 );

 const failedOrBlockedDomainsHaveEvidence = domainResults
   .filter(
     (result) =>
```

```
      result.status === AVCSDomainStatus.FAILED ||
      result.status === AVCSDomainStatus.BLOCKED
    )
    .every((result) => result.evidence.length > 0);

  return blockerFindingsHaveEvidence && failedOrBlockedDomainsHaveEvidence;
}
```

That is a real certification rule already.

What this gives you immediately

Once wired, AVCS can now produce:

- `AVCSFinding[]` with evidence refs
- `AVCSEvidenceItem[]`
- domain results enriched with evidence refs
- verdict logic that can actually consider proof sufficiency

That is a major jump in maturity.

At that point, the backend result object becomes genuinely certification-shaped.

Recommended improvement

Later, you should add evidence linking by explicit check-to-finding relation instead of fuzzy matching.

Right now this matcher:

```
surfaceKey === checkKey ||
surfaceKey.includes(checkKey) ||
checkKey.includes(surfaceKey)
```

is acceptable for v1, but not ideal long term.

The stronger future version is:

- finding metadata carries originating `checkKey`
- evidence service links exactly by `checkKey`

That will be cleaner once your findings service grows.

For now, the v1 matcher is enough.